PROIECTAREA CU MICROPROCESOARE

C3 2025-2026 Sem. 2

CURS 10 Sapt 10 29 aprilie 2026 16:00-19:00

serban@upit.ro

Regulament disciplina

Nota finala este formata din activitatile:

- Laborator 15%
- Lucrare control 15% **13 mai 2026**
- Prezentare si sustinere proiect 20%
- Examen 50% **16 iunie 2026**
- Bonus prezenta – 10%

Conditii pentru promovare:

- Nota de la laborator trebuie sa fie minim 5 (prezenta obligatorie la toate sedintele de laborator);
- Nota proiect minim 5 (conditie obligatorie pentru intrare in examen);
- Nota de la examen trebuie sa fie minim 5;
- Note de minim 5 la Lucrarea de control.

In cazul reluarii disciplinei intr-un alt an universitar, activitatile nepromovate trebuie parcurse din nou.

Continut disciplina

Familia de MCU Intel 8051 (hw & sw)
Circuite programabile I/O de tip port paralel
Circuite programabile I/O de tip timer
Circuite programabile I/O de tip USART
Circuite programabile I/O pentru controlul intreruperilor
Alte circuite specializate I/O programabile
Alte familii de MCU
Realizarea microsistemelor cu MCU (hw & sw)
Wereable
Harvesting

Bibliografie

Kenneth AYALA, *The 8051 Microcontroller: Architecture, Programming and Applications*, West Publishing Company, 1991

Applied Logic Engineering, *8051 Interfacing and Applications*, 1991

Jack GANSSLE, *The Art of designing Embedded Systems, 2nd ed.*, Newnes, Elsevier, 2008

Ken ARNOLD, *Embedded Controller Hardware Design*, LLH Technology Publishing, 2001

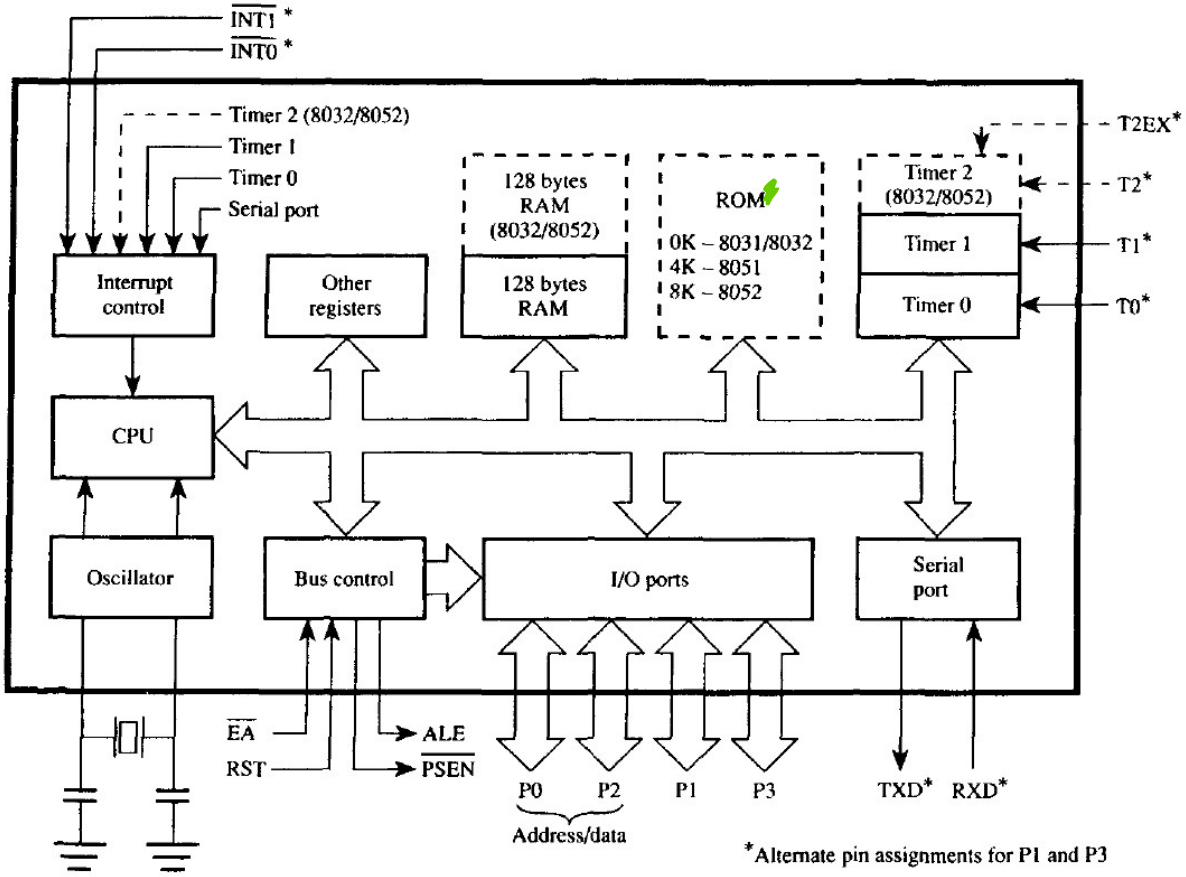
MOSTEK, *Z80 Processor - Technical Manual*

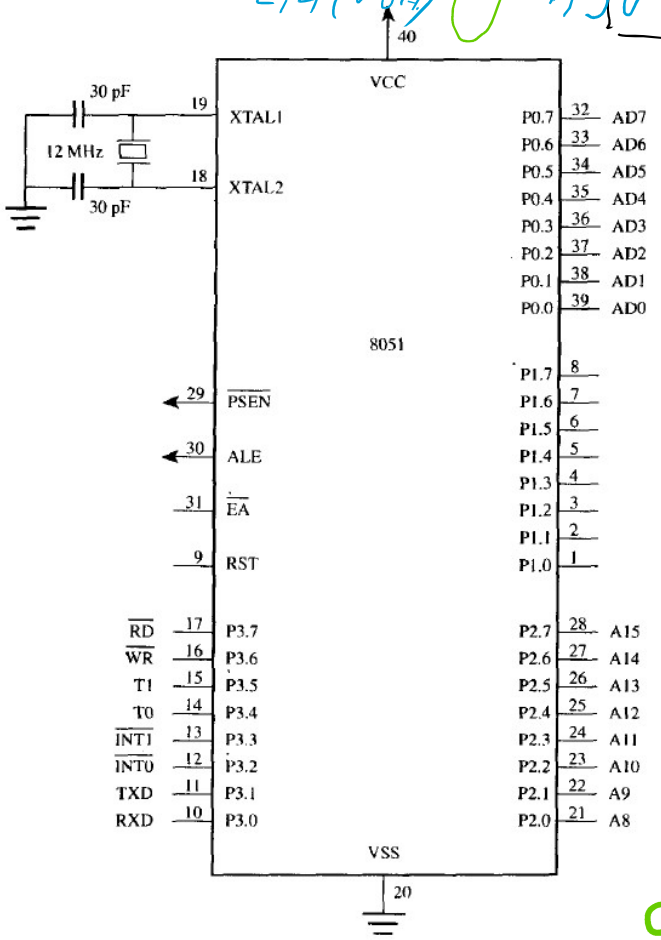
Ramesh Gaonkar, *Z-80 Microprocessor Architecture, Interfacing, Programming and Design*,

Prentice Hall, 3 ed., 2000

Documentatii de firme (Intel, Zilog)

MCU 8051





0,4 (V_{OL}) ∩ 0,9 (V_{IL})

2,4 (V_{OH}) ∪ 1,9 V

MCU 8051

Parameter	V_{oh}	I_{oh}	V_{ol}	I_{ol}	V_{il}	I_{il}	V_{ih}	I_{ih}	P_t
CMOS	2.4 V	−60 μA	.45 V	1.6 mA	.9 V	|10 μA|	1.9 V	|10 μA|	50 mW
NMOS	2.4 V	−80 μA	.45 V	1.6 mA	.8 V	−800 μA	2.0 V	10 μA	800 mW
	V_{OH}	I_{OH}	V_{OL}	I_{OL}	V_{IL}	I_{IL}	V_{IH}	I_{IH}	

CMOS

Marja de zgomot

$|V_{OH} - V_{IH}| = 0,5V$

$|V_{IL} - V_{OL}| = 0,45V$

Pinii $XTAL_1$ și $XTAL_2$ –intrari pe care se conectează în exterior un cristal de cuarț în ritmul căruia microcontroller-ul operează.

Pinul RESET – intrarea pentru inițializarea hardware a controller-ului; dupa RESET registrul PC este initializat cu 0000h.

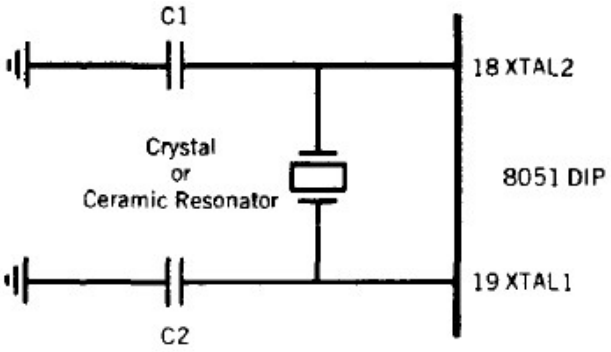
Pinul $\overline{EA}$ (external address) – este o intrare care permite configurarea controller-ului. Astfel, dacă $\overline{EA}$ =0 controller-ul formează magistrale externe de adrese și date la care se pot conecta în exterior alte resurse suplimentare (circuite de memorie, circuite de I/O), iar dacă $\overline{EA}$=1 nu se formează magistrale externe și controller-ul operează doar cu resursele interne.

Pinul $\overline{PSEN}$ ($\overline{PSEL}$) (Program Select Enable) – este o ieșire activată de controller în momentul când operează cu spațiul adreselor memoriei de programe (internă sau externă).

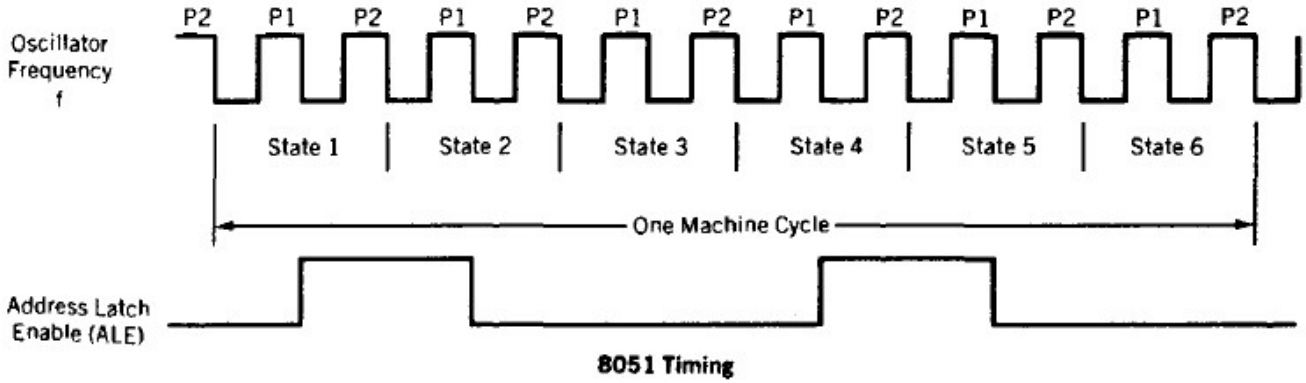
Pinul ALE (Address Latch Enable) – semnalul permite demultiplexarea magistralei comune de adrese și date generată de către procesor atunci când $\overline{EA}$=0. În această situație porturile paralele P_0 și P_2 își pierd funcționalitatea de port paralel și P_0 formează o magistală multiplexată de adrese și date cu liniile denumite AD_0 – AD_7, iar P_2 generează octetul cel mai semnificativ (ocms) al magistralei de adrese, A_8 – A_{15}.

Pinii $\overline{RD}$ și $\overline{WR}$ – sunt semnale de ieșire care se activează la lucrul controller-ului cu spațiul adreselor memoriei exerne de date, fapt care presupune că $\overline{EA}$=0; $\overline{RD}$=0 la citirea memoriei externe de date, iar $\overline{WR}$ =0 la scrierea memoriei externe de date; în mod normal pinii pe care apar cele două semnale fac parte din portul paralel P3, dar își schimbă funcționalitatea când $\overline{EA}$=0. Cele două semnale nu se activează la lucrul cu memoria de programe, dar permit decodificarea externă a spațiului memoriei de date.

CIRCUIT EXTERN OSCILATOR LA MCU 8051

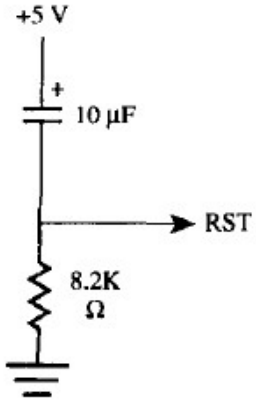


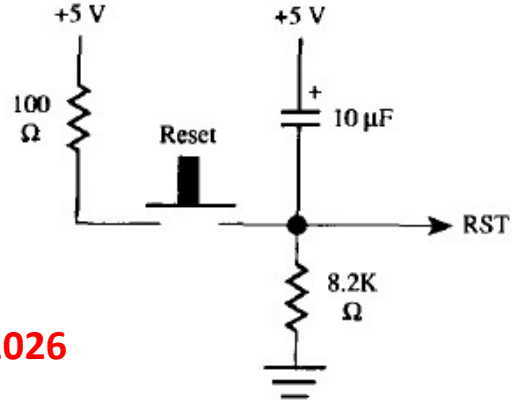
CICLU MASINA 8051



8051 Timing

CIRCUITE EXTERNE DE RESET LA MCU 8051



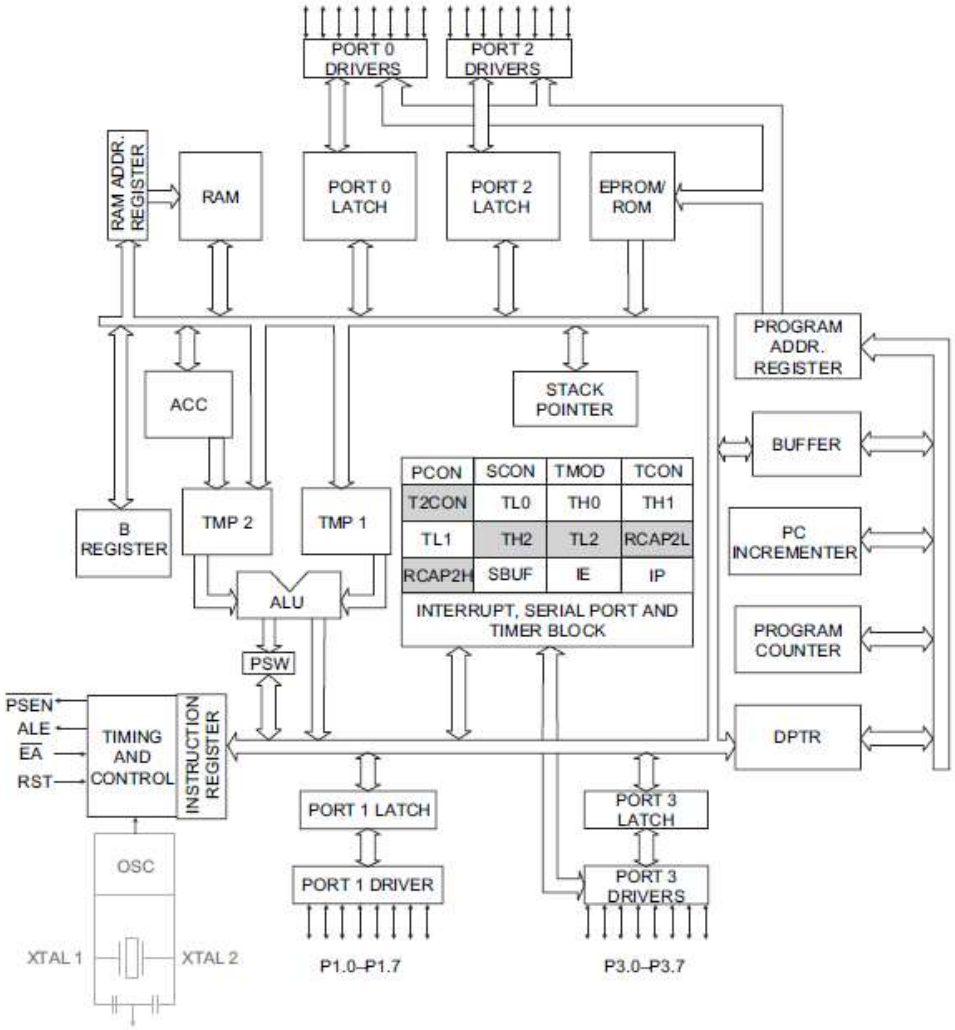


END 4 mar 2026

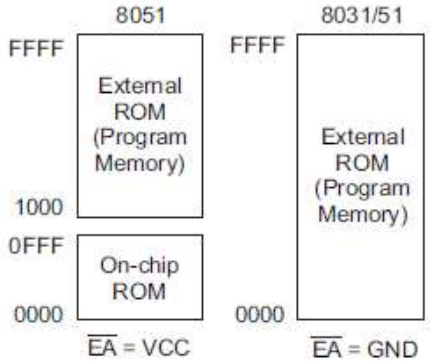
EFECTE RESET LA MCU 8051

REGISTER(S)	CONTENTS
Program counter	0000H
Accumulator	00H
B register	00H
PSW	00H
SP	07H
DPTR	0000H
Ports 0-3	FFH
IP (8031/8051)	XXX00000B
IP (8032/8052)	XX000000B
IE (8031/8051)	0XX00000B
IE (8032/8052)	0X000000B
Timer registers	00H
SCON	00H
SBUF	00H
PCON (HMOS)	0XXXXXXXB
PCON (CMOS)	0XXX0000B

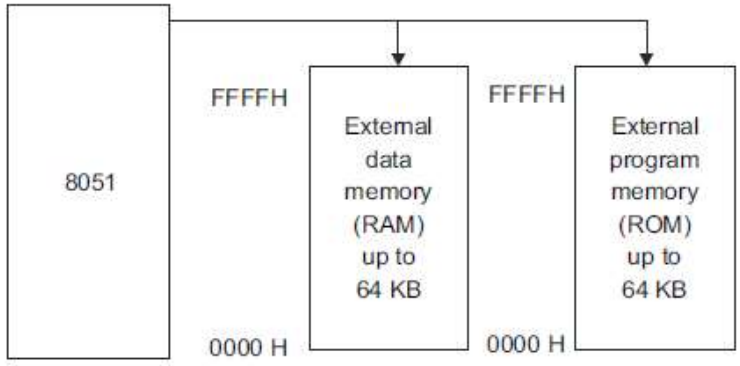
STRUCTURA INTERNA MCU 8051



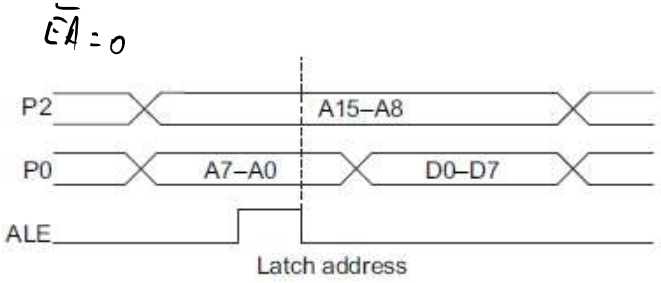
MEMORIA DE PROGRAME LA MCU 8051

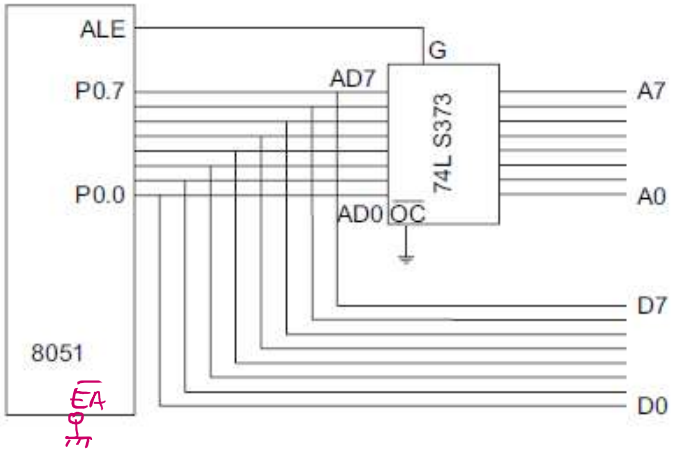


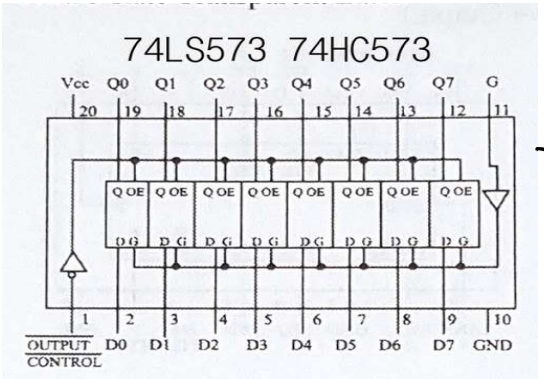
MEMORIA EXTERNA LA MCU 8051 (EA = 0)

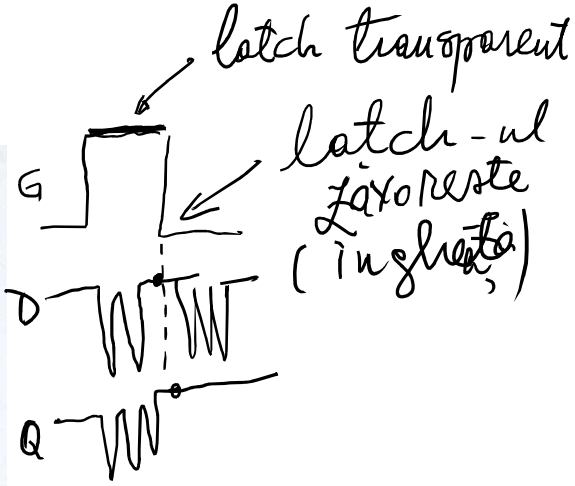


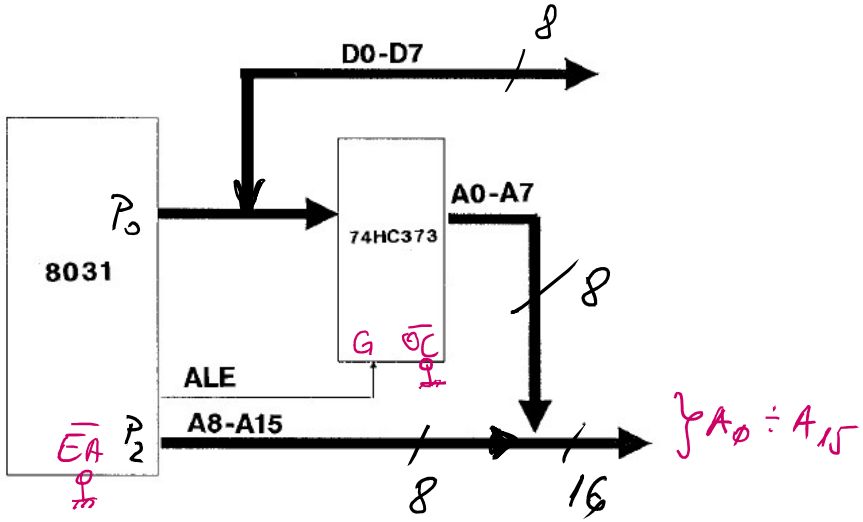
FORMAREA MAGISTRALELOR EXTERNE MCU 8051 (EA = 0)







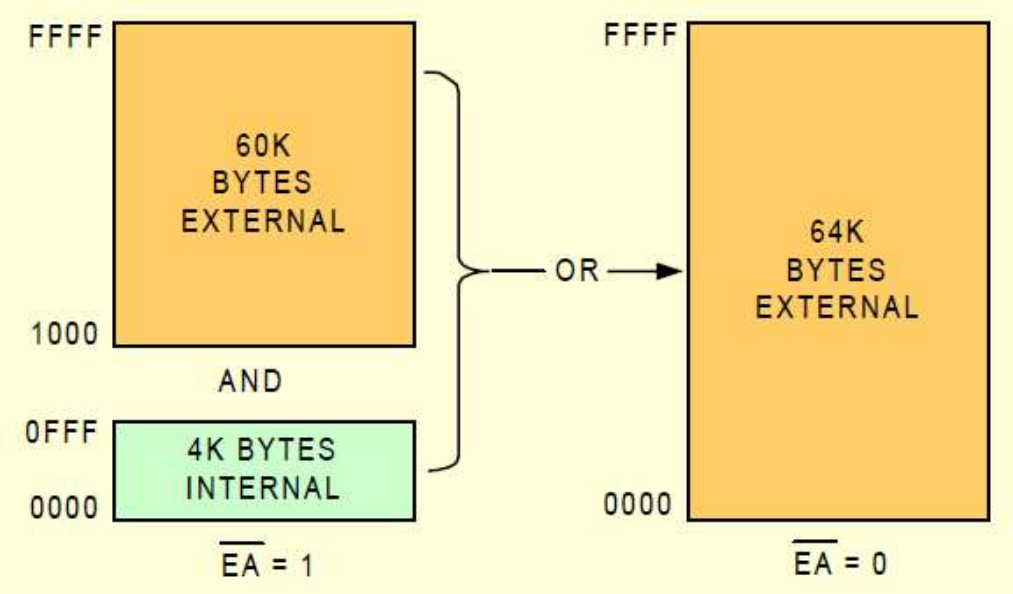




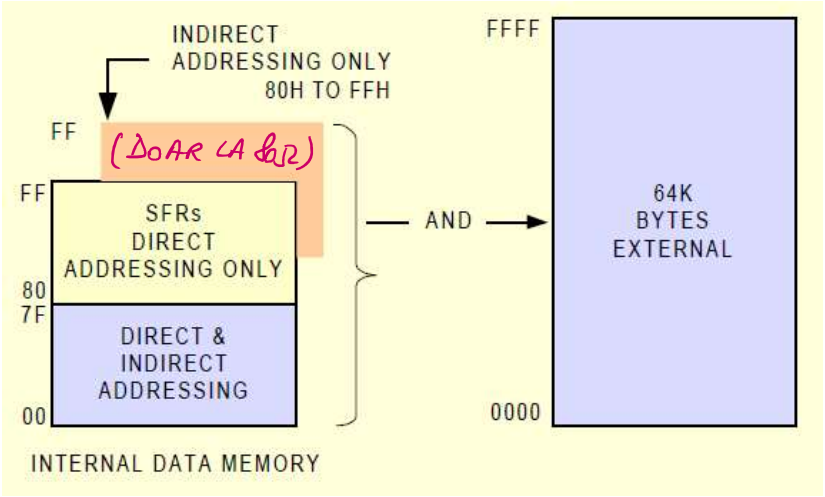
FUNCTIONALITATI PINI CONTROL LA MCU 8051

	*PSEN	*RD	*WR
Program Instruction Fetch	0	1	1
External Data Read	1	0	1
External Data Write	1	1	0

MEMORIA DE PROGRAME LA MCU 8051

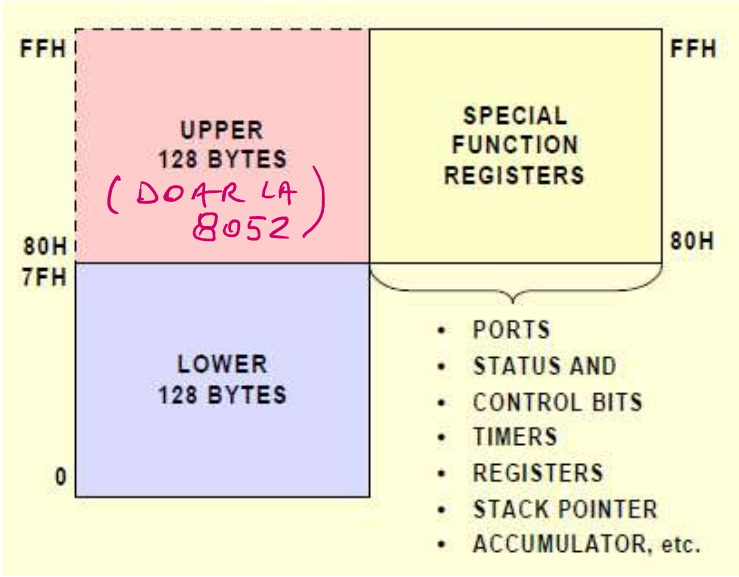


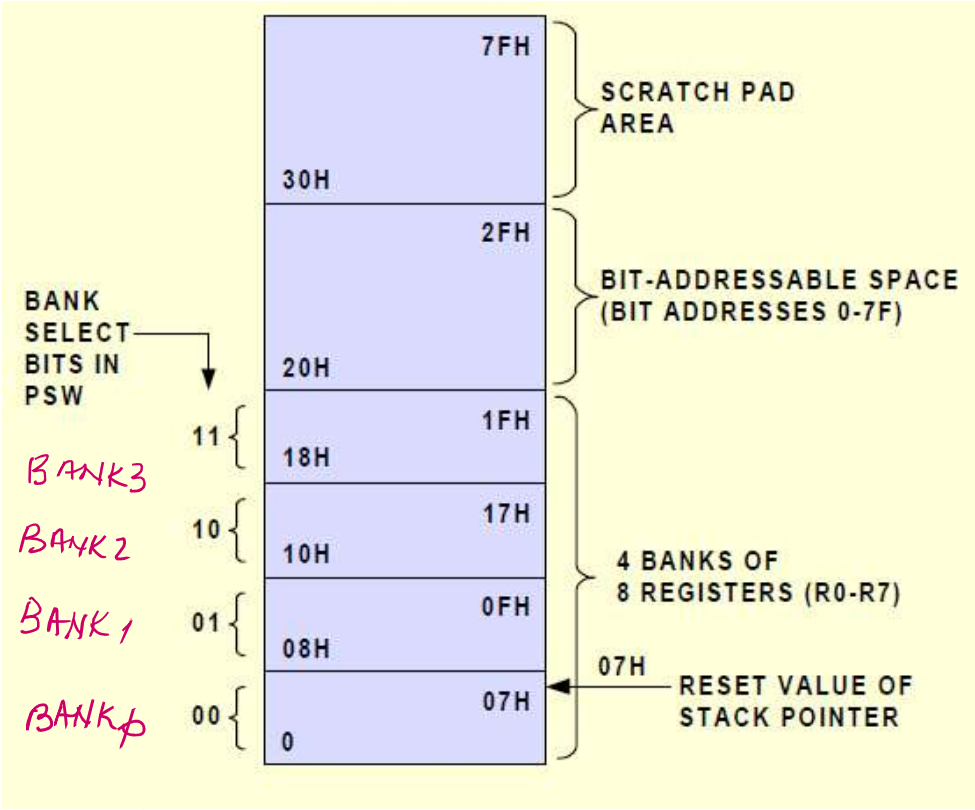
MEMORIA DE DATE LA MCU 8051



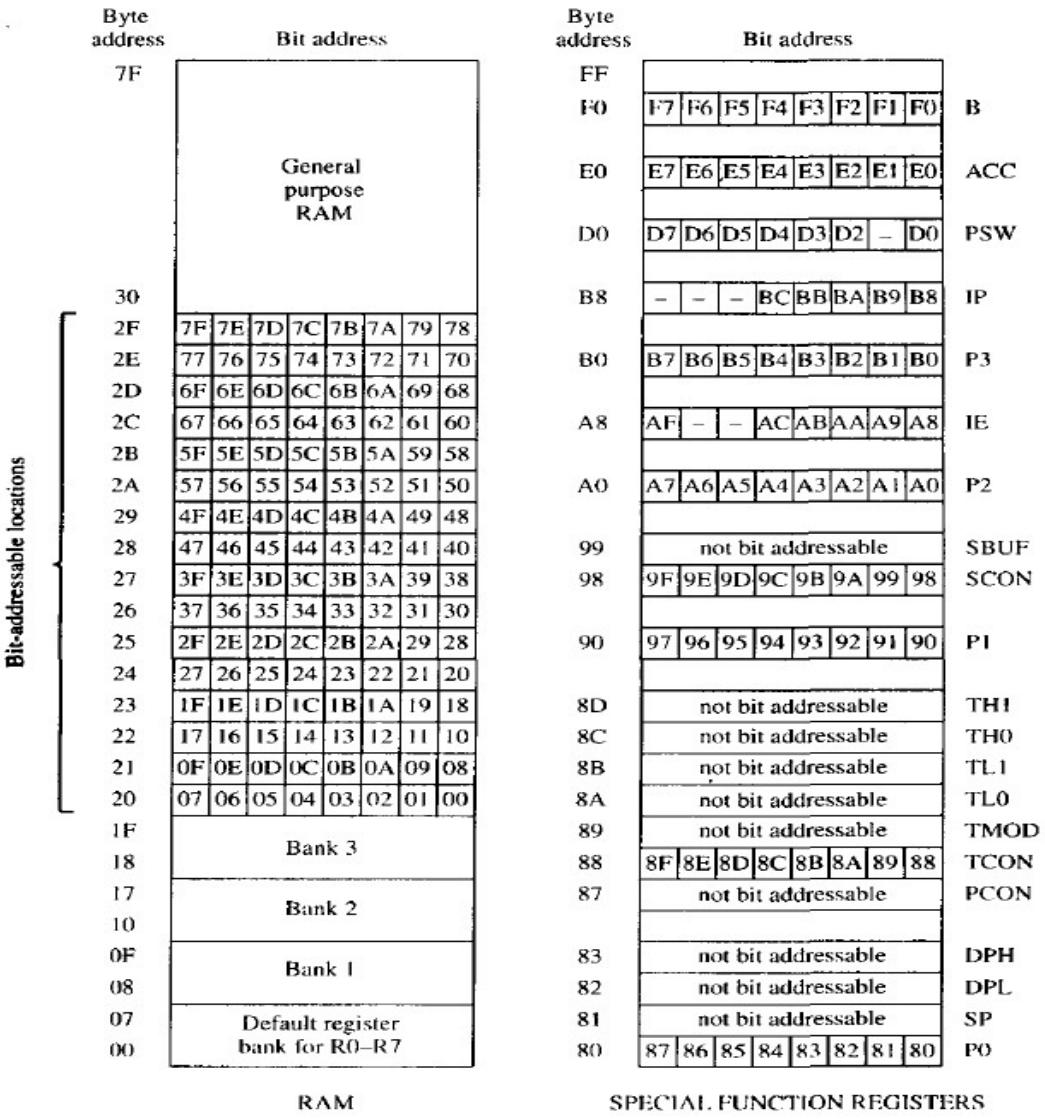
SFR - Special Function Registers

MEMORIA RAM INTERNA (DATE) LA MCU 8051

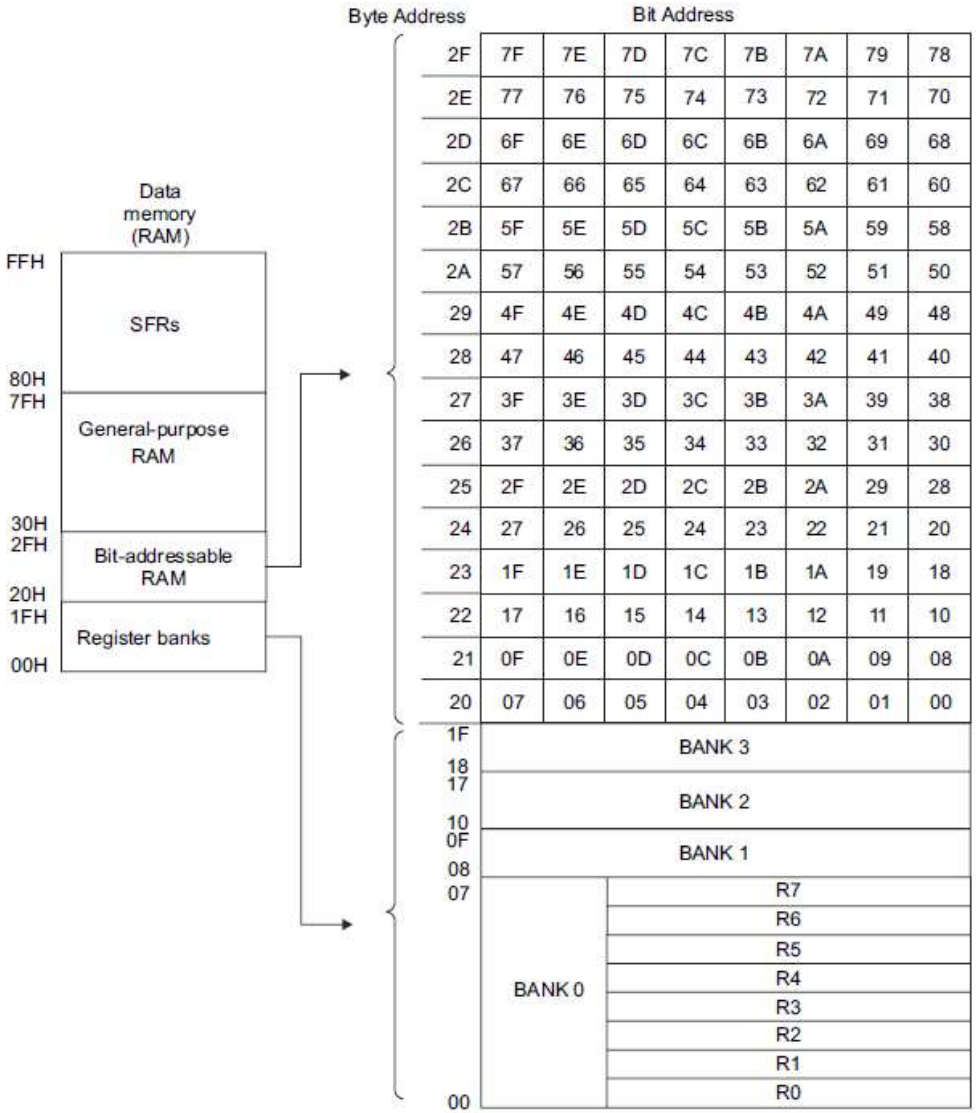


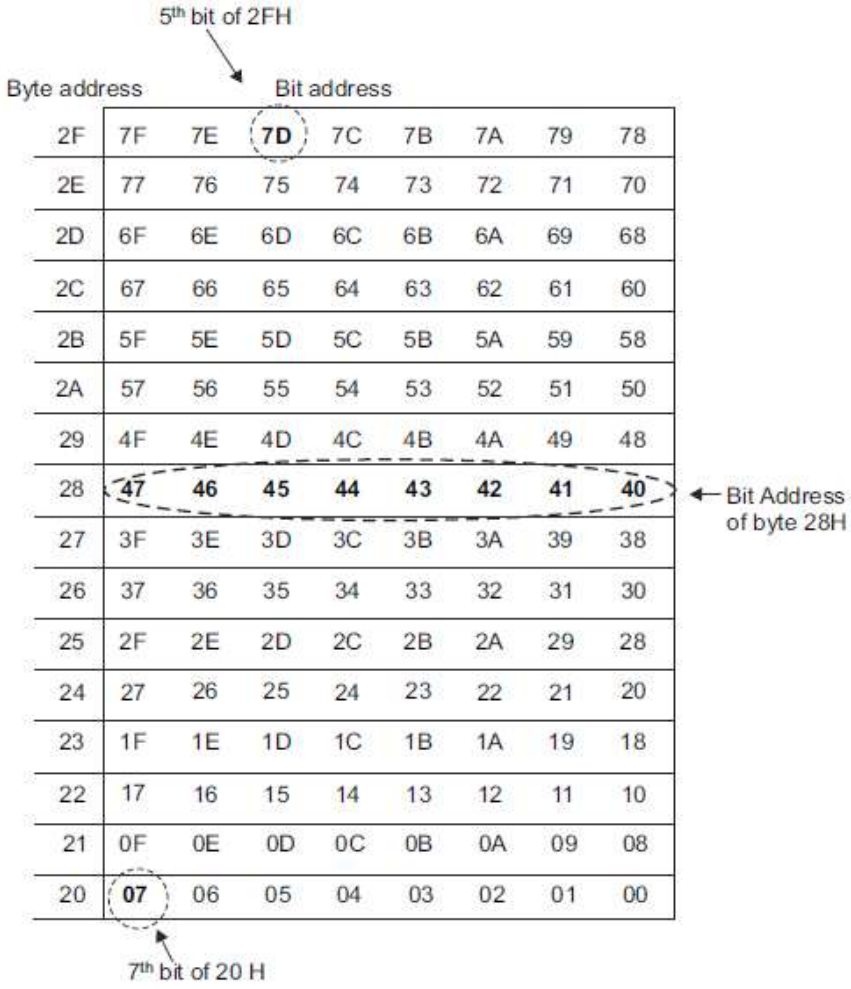


MEMORIA RAM INTERNA (DATE) LA MCU 8051

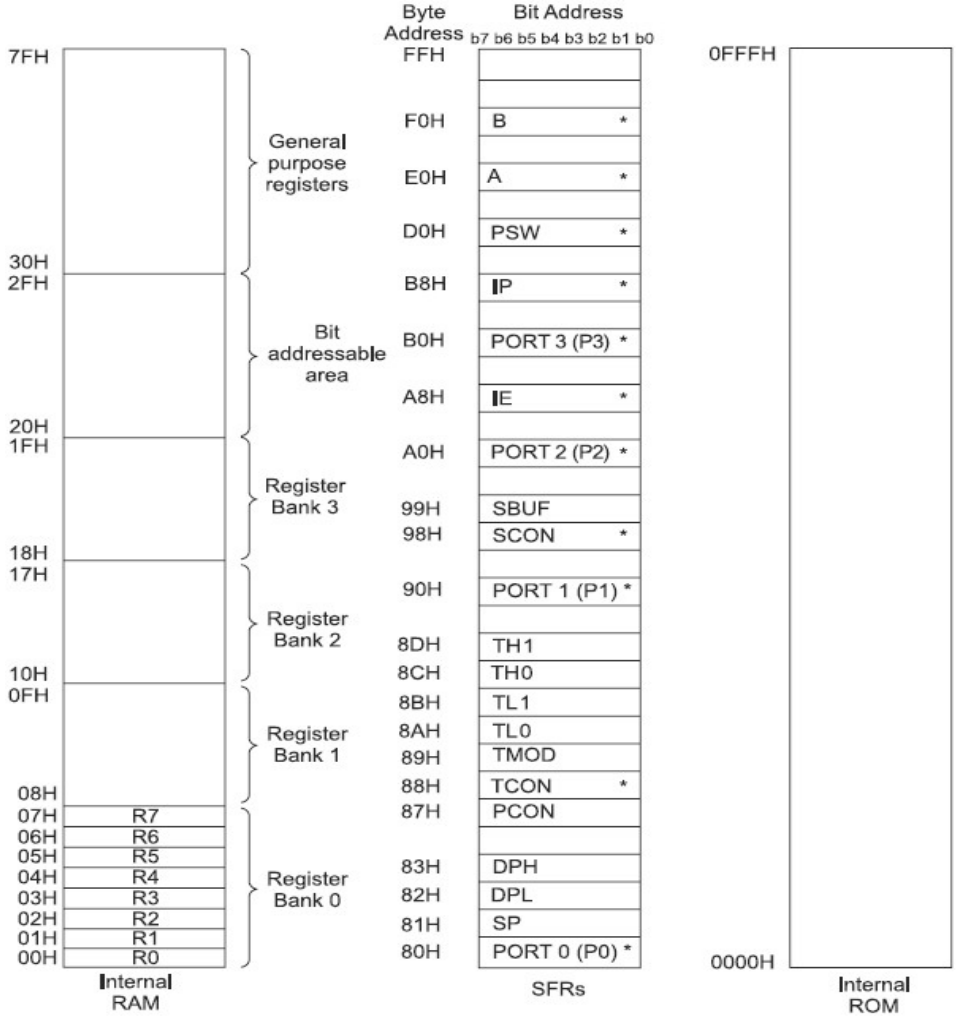


ZONA ADRESABILA PE BIT RAM INTERN MCU 8051





MCU INTEL 8051 – Programming Model

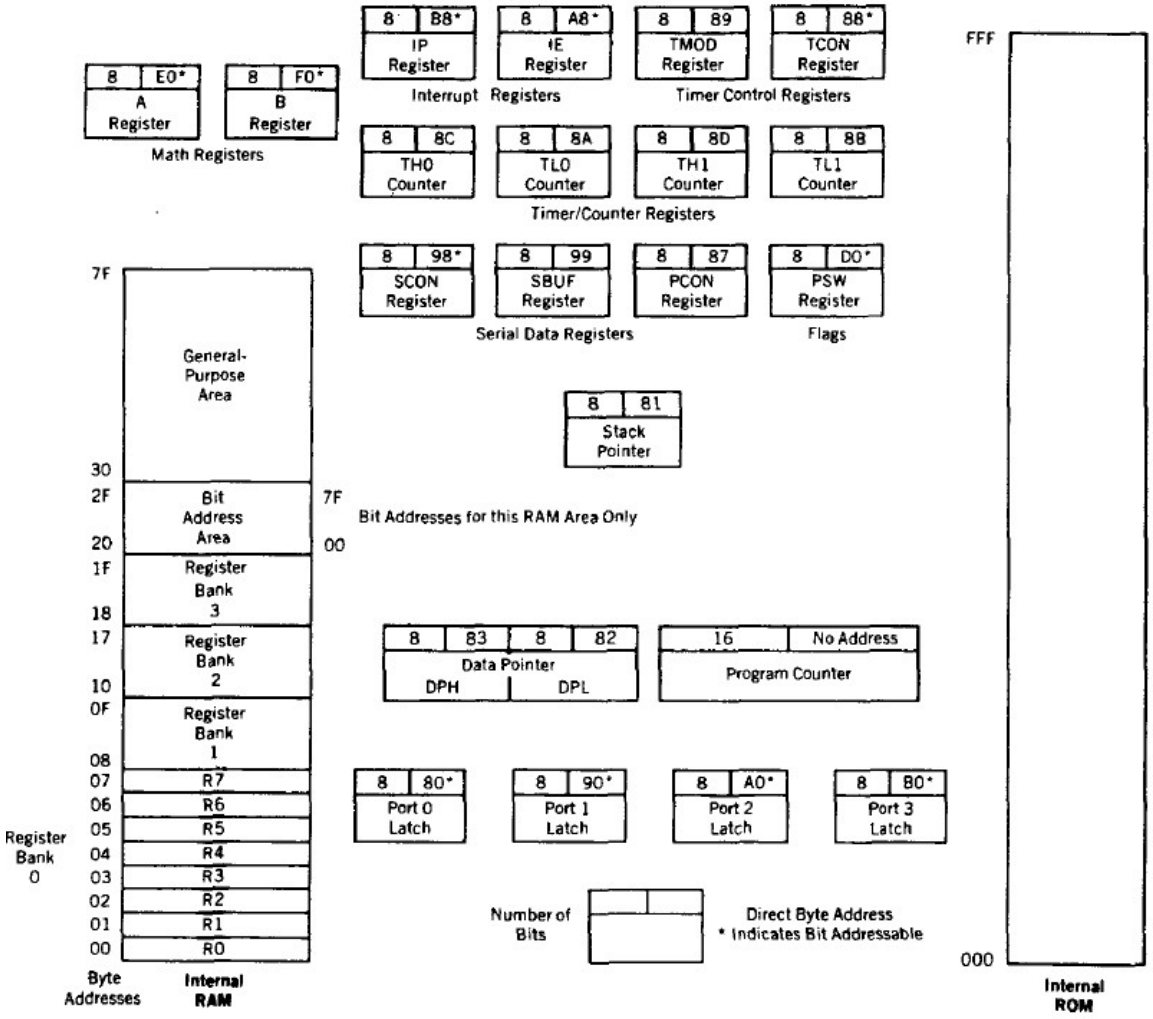


*Indicates the SFRs which are also bit addressable

DPTR

NAME	FUNCTION	INTERNAL RAM ADDRESS (HEX)
A	Accumulator	0E0
B	Arithmetic	0F0
DPH	Addressing external memory	83
DPL	Addressing external memory	82
IE	Interrupt enable control	0A8
IP	Interrupt priority	0B8
P0	Input/output port latch	80
P1	Input/output port latch	90
P2	Input/output port latch	A0
P3	Input/output port latch	0B0
PCON	Power control	87
PSW	Program status word	0D0
SCON	Serial port control	98
SBUF	Serial port data buffer	99
SP	Stack pointer	81
TMOD	Timer/counter mode control	89
TCON	Timer/counter control	88
TL0	Timer 0 low byte	8A
TH0	Timer 0 high byte	8C
TL1	Timer 1 low byte	8B
TH1	Timer 1 high byte	8D

MCU INTEL 8051 – Programming Model



MCU INTEL 8051 – Programming Model

MCU 8051 – Special Function Registers - SFR

+	0	1	2	3	4	5	6	7
F8								
F0	B							
E8								
E0	ACC							
D8								
D0	PSW							
C8	~~T2CON~~		~~RCAP2L~~	~~RCAP2H~~	~~TL2~~	~~TH2~~		
C0								
B8	IP							
B0	P3							
A8	IE							
A0	P2							
98	SCON	SBUF						
90	P1							
88	TCON	TMOD	TL0	TL1	TH0	TH1		
80	P0	SP	DPL	DPH				PCON

- Denotes bit addressable Special Function Registers in this table and all following diagrams

Registri CPU:
ACC (A), B, DPH, DPL (DPTR), SP, PSW, PCON

Registri Porturi Paralele:
P0, P1, P2, P3

Registri Timere:
TCON, TMOD, TH0, TL0, TH1, TL1

Registri UART:
SCON, SBUF

Registri logica intreruperi:
IE, IP

REGISTRI CPU DIN SFR

ACC (A)

B (MUL AB ; DIV AB)
multiply divide

MUL AB
A, B ← A × B
ocms ocmps

A B ← A / B
cat rest
2 3
11/4

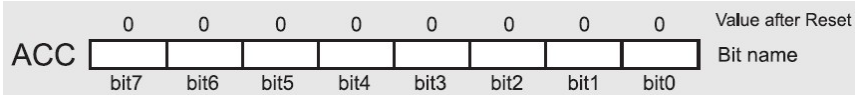
DPH, DPL (DPTR) – Data Pointer

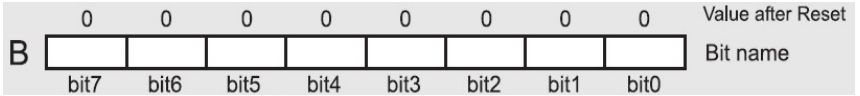
SP - Stack Pointer (initializat la Reset cu 07h)

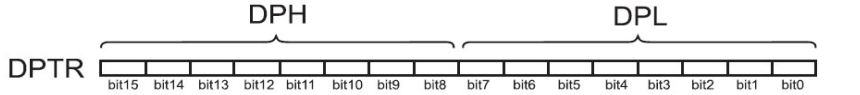
PSW – (Program Status Word)

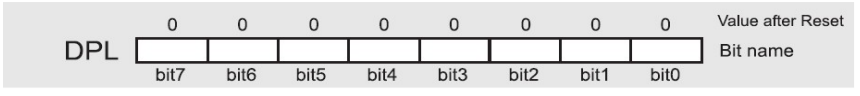
PCON – (Power Control)

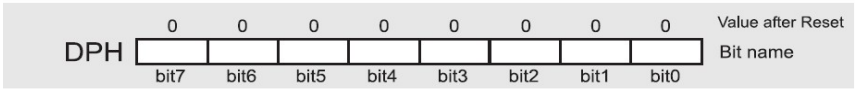
(IDLE, POWER DOWN)

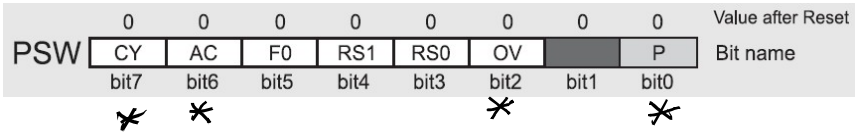












IDL - idle - 1mA – 5mW
PD - power down - 10μA – 50μW
GF1,0 - general flag.

DPH, DPL (DPTR) – Data Pointer

DPH și DPL – prin reuniunea lor formează singurul registru pe 16 biți accesibil utilizatorului, denumit DPTR (data pointer). Prin intermediul acestui registru se poate adresa memoria externă de programe și date, dar și memoria internă de programe. Nu poate folosit pentru adresarea RAM-ului intern sub forma DPTR.

move

Astfel, instrucțiunile de transfer de date cu memoria RAM externă sunt:

- MOVX A,@DPTR – realizează o citire din memoria RAM externă; A ← @DPTR (X reprezentand eXternal). A ← (DPTR)
- MOVX @DPTR, A – realizează o scriere în memoria RAM externă; @DPTR ← A sau (DPTR) ← A.

Transferul din memoria de programe se face cu MOVC A, @A + DPTR (C reprezinta Code). În acumulator este adus conținutul locației memoriei de programe a cărei adresă se obține prin însumarea conținutului DPTR cu cel al acumulatorului anterior:

A ← (A+DPTR).

MOVC permite citirea memoriei de programe atât internă ($\overline{EA}$ = 1) cât și externă ($\overline{EA}$ = 0). Această instrucțiune trebuie pregătită prin încărcarea DPTR și a acumulatorului.

MOV DPTR,#adresă memorie

MOV A, #deplasament

MOVC A,@A+DPTR

END 11 mar 2026

MOV A, #31h

A ← #31h

MOV A, 31h

A ← (31h)

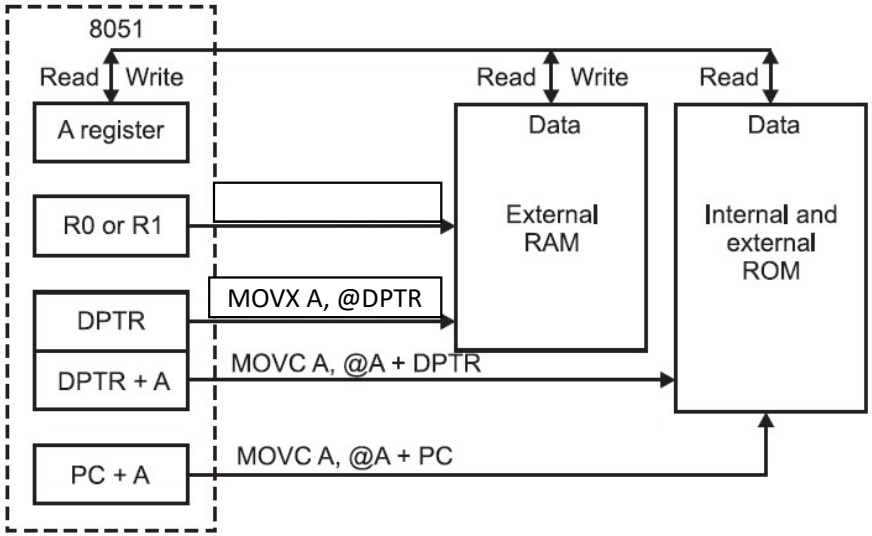
Salvarea în stiva a DPTR se poate face doar individual pe jumătățile din care este format.

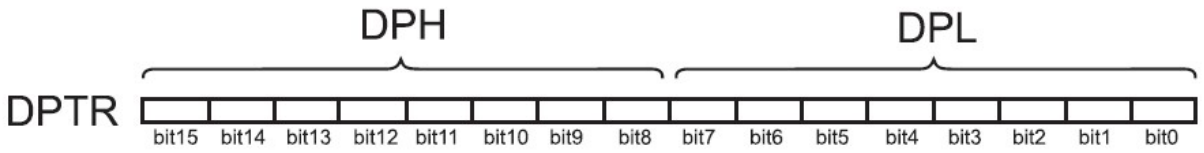
Notatii folosite in instructiuni:

\# - reprezinta operand imediat

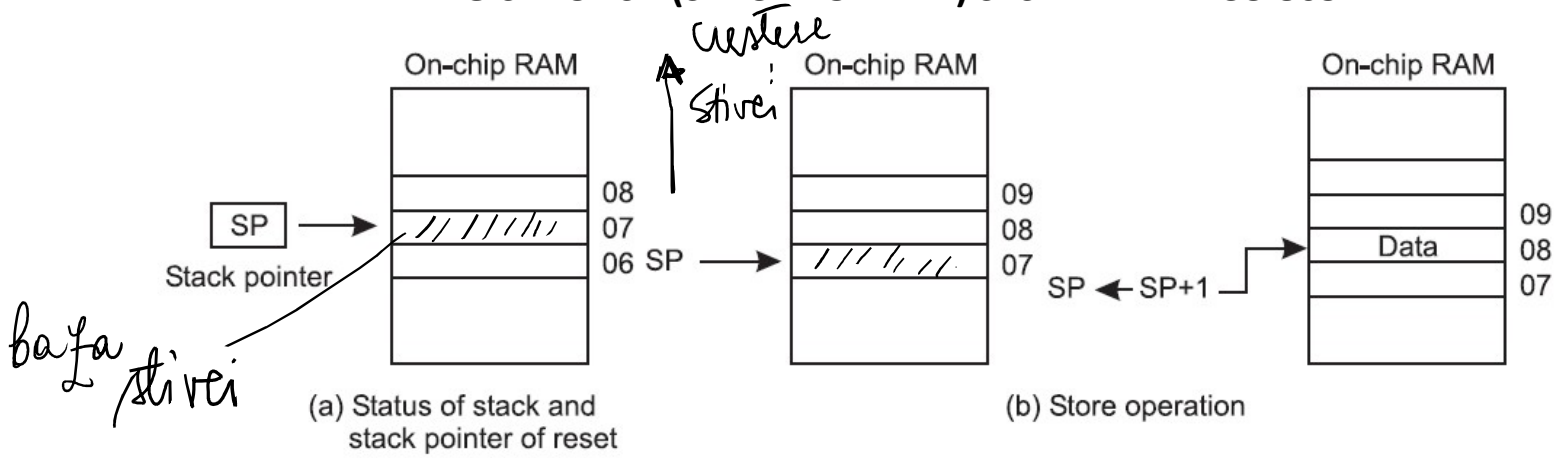
@ - continut al unei locatii de memorie a carei adresa urmeaza dupa simbol

DPH, DPL (DPTR) – Data Pointer





REGISTRUL SP (STACK POINTER) SI STIVA LA MCU 8051



Stiva se formează în RAM-ul intern și este adresată în tehnică LIFO de un registru specializat, denumit SP, pe 8 biți. La inițializarea controller-ului, SP se încarcă cu adresa 07_H (baza stivei), prima locație în care se poate depune în stivă fiind 08_H. Stiva crește către sfârșitul memoriei (la depuneri în stivă) și descrește la extrageri, adică SP de incrementează la depuneri și se decrementează la extrageri. Registrul SP este localizat în zona de memorie RAM internă, denumită SFR.

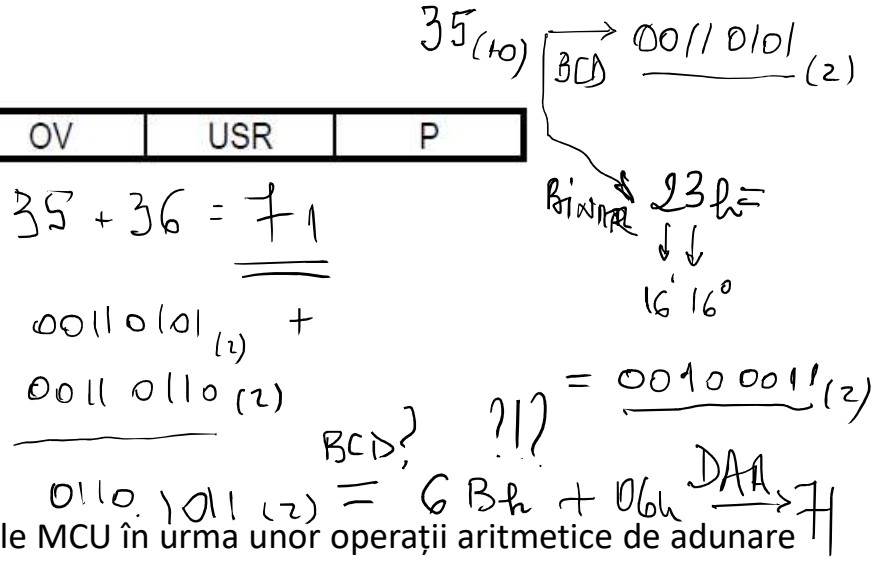
Din cele de mai sus rezultă faptul că proiectanții propun formarea stivei în zona în care se formează bank-urile de regiștri 1,2,3. Dacă aplicația în care este folosit controller-ul permite acest lucru opțiunea este posibilă. Dacă nu (se folosesc bank-urile de regiștri), atunci este posibilă schimbarea zonei de memorie RAM în care se formează stiva. În acest sens registrul SP se încarcă cu o nouă adresă de bază a stivei printr-o instrucțiune de tipul *MOV SP, #adresă_bază_stivă*, unde noua adresă de bază a stivei poate fi mai mare de 30_H (sau într-o altă zonă liberă de sarcini).

REGISTRUL PSW MCU 8051

Processor Status Word (PSW) - Bit Addressable

CY	AC	F0	RS1	RS0	OV	USR	P

CY	Carry Flag
AC	Auxiliary Carry Flag
F0	General Purpose Flag
RS1	Register Bank Selector 1. MSB of selector.
RS0	Register Bank Selector 0. LSB of selector.
OV	Overflow Flag
USR	User Definable Flag
P	Accumulator Parity Flag



Bitul CY – (bitul de transport/împrumut) care este setat în mod automat de mecanismele MCU în urma unor operații aritmetice de adunare respectiv scădere. Bitul poate fi acționat și manual de către utilizator prin instrucțiuni specifice.

Bitul AC – (auxiliar carry) este bitul de transport între cele două jumătăți ale acumulatorului în urma operațiilor aritmetice de adunare și scădere.

Bitul F0 – (flag zero) este bit accesibil utilizatorului și la dispoziția acestuia care poate fi setat – resetat în funcție de necesitățile programului în care este folosit.

Bitii RS_1 și RS_0 – (Registers select) selectează bank-ul de regiștri de uz general R_0 până la R_7 activ la un moment dat. La inițializarea MCU RS_1= RS_0=0.

Bitul OV – (overflow) este un bit acționat de mecanismele MCU prin care se indică depășirea gamei de reprezentare a rezultatelor obținute ca urmare a efectuării a operației aritmetice (adunare sau scădere) cu numere reprezentate în CC2 pe 8 biti.

Bitul P – (parity) este bit prin care se indică paritatea rezultatului obținut în urma operațiilor aritmetico-logice. Bitul se calculează cu metoda parității pare. Spre deosebire de procesorul Z-80, bitul P este acționat atât în urma operațiilor logice cât și a celor aritmetice.

REGISTRUL PCON MCU 8051

Power Control Register (PCON) - Not Bit Addressable

SMOD	-	-	-	GF1	GF0	PDWN	IDLE

SMOD	Serial baud rate generator mode. If SMOD=1 the baud rate of the UART is doubled.
-	Reserved.
-	Reserved.
-	Reserved.
GF1	General Purpose flag.
GF0	General Purpose flag.
PDWN	Power Down flag. Setting this bit causes activation of power down mode.
IDLE	Idle flag. Setting this bit causes activation of idle mode.

Bitul IDLE=1 conduce controller-ul la o stare de funcționare în care procesorul este oprit, dar toate circuitele I/O sunt funcționale iar memoria RAM internă este alimentată. Consumul controller-ului se reduce la sub 1mA. Revenirea în starea de funcționare se poate face fie printr-o întrerupere generată de circuitele I/O, fie printr-un reset aplicat procesorului.

Bitul PDWN=1 conduce la intarea procesorului într-o stare în care doar memoria RAM internă rămâne alimentată și sunt oprite procesorul, respectiv toate circuitele de I/O. Consumul în această stare se reduce la pâna 10 uA.

Biții GF_0 și GF_1 – sunt biți ce pot fi scriși de utilizator, prin care se poate realiza o codare a 4 situații posibile în urma cărora ar fi rezultat stările PD, respectiv IDL. Codarea va putea fi folosită de către procesor la revenirea în starea de funcționare pentru a detecta situația care a generat intrarea în stările PD, respectiv IDL.

Bitul SMOD – când este 1 logic dublează viteza de comunicație serială proiectată uzual pentru UART.

Porturile paralele din MCU 8051

Sunt notate P_0, P_1, P_2, P_3, iar pinii gestionați au denumirile: P_{00}-P_{07}, P_{10}-P_{17}, P_{20}-P_{27}, P_{30}-P_{37}.

Dintre cele 4 porturi doar P_1 nu are ale funcționalități suplimentare. Cele 4 porturi sunt gestionate prin intermediul a 4 regiștri aflați în zona SFR care au aceleași denumiri: P_0, P_1,P_2,P_3. Cei 4 regiștri menționați sunt regiștri de date pentru porturile asociate, neexistând nici un registru pentru controlul software al acestuia. Utilizarea regiștrilor se face în modul următor:

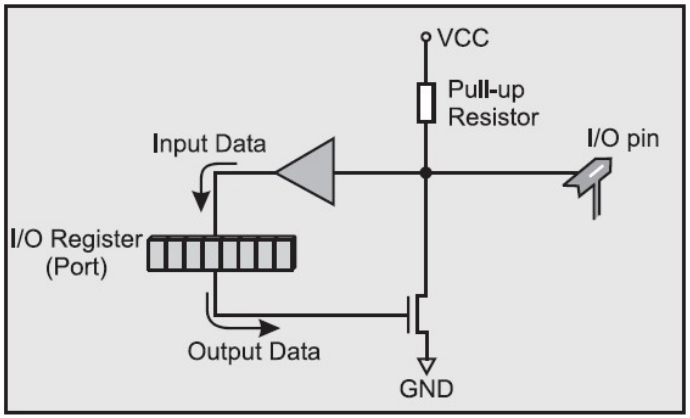
- o scriere efectuată în unul din regiștri descriși conduce la generarea semnalelor digitale corespunzătoare pe liniile portului corespondent;

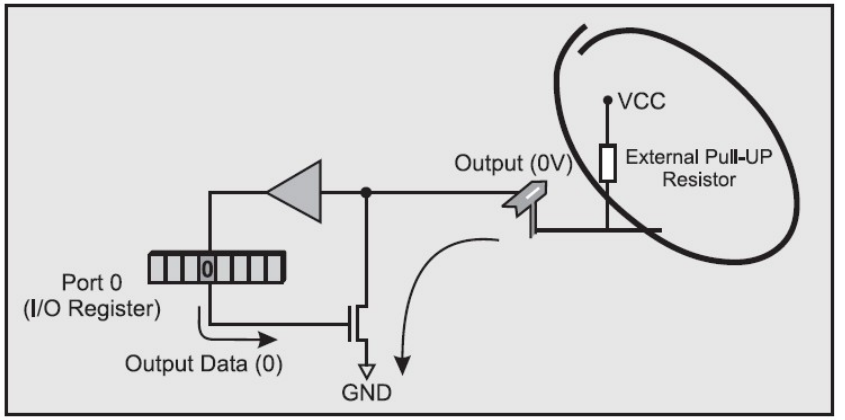
- o citire efectuată din unul din regiștri menționați provoacă preluarea informației corespunzătoare semnalelor digitale existente pe liniile portului asociat;

Obs. Nu există mecanism software de stabilire a direcționalitatii liniilor unui port. Liniile unui port pot avea direcționalități diverse. Reamintim că regiștrii P_0,P_1,P_2,P_3 sunt în prima coloană (stânga) din tabelul SFR și au proprietatea de a fi acționati la nivel de bit prin instrucțiuni specifice.

Modificări pe liniile portului (intrări) nu pot fi semnalizate prin întreruperi procesorului din controller, tehnica de idenficare a unei modificări fiind de tip polling (interogare software permanentă).

Porturile paralele din MCU 8051





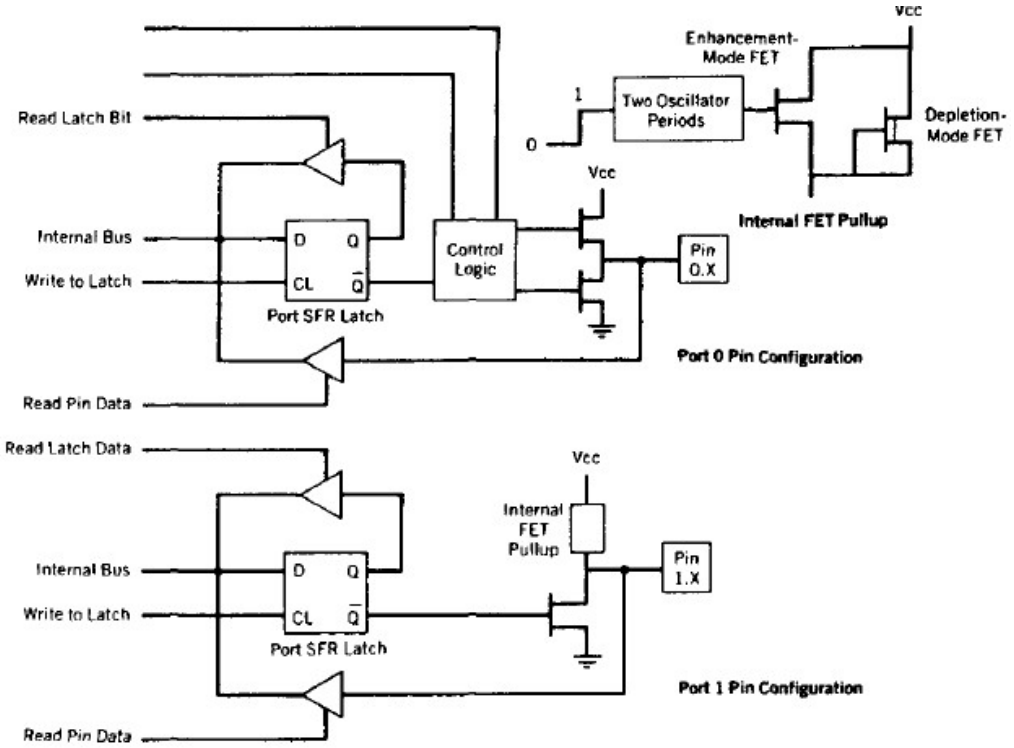
Când sunt configurati ca ieșiri (logic 0), pinii dintr-un singur port pot „primi” curent de pana la 10mA. Dacă toți cei 8 biți ai unui port sunt activi, curentul total trebuie să fie limitat la 15mA (la portul P0: 26mA). Dacă toate porturile (32 biți) sunt active, curentul maxim total trebuie limitat la 71mA.

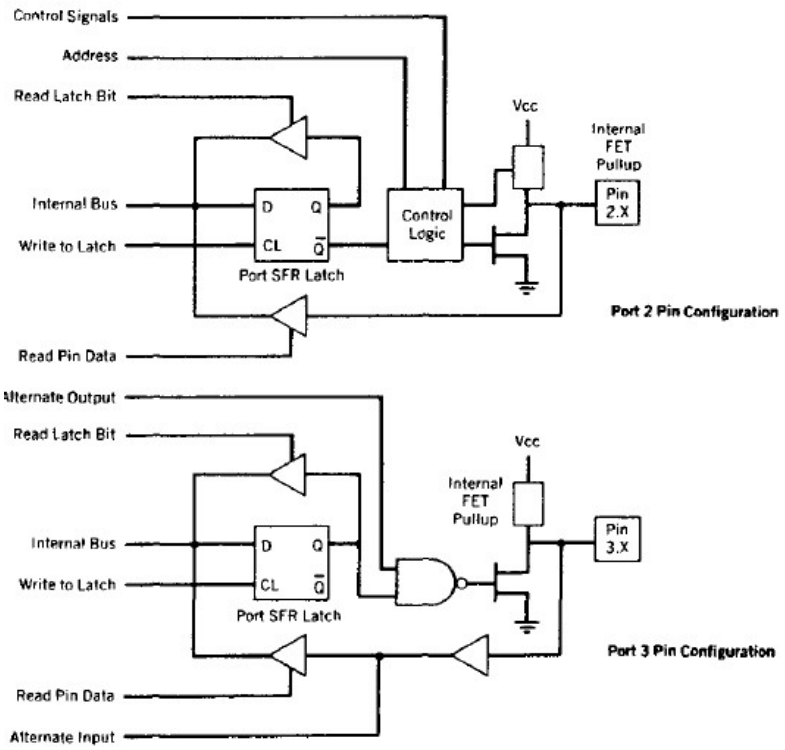
Când sunt configurati ca intrari (logic 1), rezistența de pull-up încorporată asigură un curent foarte slab, dar suficient de puternic pentru a conecta până la 4 intrări TTL din seria LS.

Chiar dacă toți pinii porturilor au mai mult sau mai puțin structură internă similară este necesar să se acorde atenție la cum si ce vor fi folositi.

De exemplu: Dacă sunt utilizate ca ieșiri cu nivel de tensiune ridicat (5V), atunci portul P0 ar trebui să fie evitat deoarece pinii săi nu au rezistor adăugat pentru conectarea la + 5V; prin urmare, numai un nivel logic scăzut poate fi obținut; dacă un alt port este utilizat în același scop, ar trebui să avem în vedere că rezistențele de pull-up au valori relativ mari. În consecință, se poate conta numai pe câteva sute de microamperi de curent care ies dintr-un pin.

Porturile paralele din MCU 8051





Canalele Timer din MCU 8051

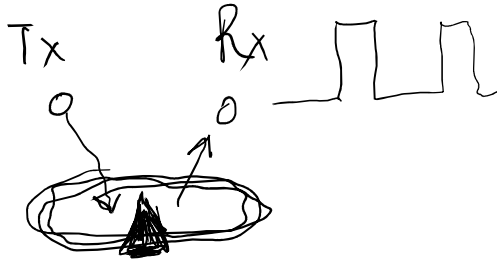
La nivelul circuitului 8051 există două canele timer care pot opera independent capacitatea de numarare este de 16 biti. Sunt posibile 4 moduri de operare și două regimuri de funcționare programabile software.

Regimul de funcționare permite alegerea sursei semnalului de ceas în ritmul căruia canalul timer operează. Se poate opta pentru ceas extern controller-ului aplicat pe una din intrările T_0 sau T_1 , în funcție de canal sau ceas intern provenit de la oscilatorul local (divizat prin 12).

Cele două canale timer sunt gestionate de regiștri TMOD și TCON, respectiv regiștrii de date, TH_0 și TL_0 la canalul timer 0, respectiv TH_1 și TL_1 la canalul timer 1.

Indiferent de sursa semnalului de ceas si indiferent de modul de operare, canalele timer din 8051 numără înainte până la atingerea stării finale.

Starea finală poate fi 2^8=256 când numără pe 8 biți sau 2^{16}=65536 când se numără pe 16 biți. Numărul de strări prin care se trece în procesul de numărare la aplicarea unui impuls de ceas este dat de diferența dintre starea finală, amintită mai sus, și starea inițială care se înscrie la începutul procesului de numărare în perechea de regiștri TH_i și TL_i.



Registrul TMOD - Timer Mode

	7	6	5	4	3	2	1	0
	GATE	C/$\overline{T}$	M_1	M_0	GATE	C/$\overline{T}$	M_1	M_0
	Timer 1				Timer 0			

Bitii M0 si M1 – aleg modul de operare la canal

Bitii C/$\overline{T}$ – denumirea provine din counter/timer; permite alegerea regimului de funcționare (sursa semnalului de ceas) la respectivul canal timer. Astfel, C/$\overline{T}$=1 înseamnă regim counter (numerator), sursa semnalului de ceas este externă aplicată pe intrarea T_i a canalului, iar C/$\overline{T}$=0, înseamnă regim timer (temporizator), sursa semnalului de ceas este internă.

Bitii GATE – dacă GATE=1 declanșarea numărării la respectivul canal timer este validată hardware de un semnal extern printr-un semnal de tip GATE=1 aplicat pe un pin din P_3 notat $\overline{INT_i}$. Acești pini apar că au triplă funcționaliate: pin de tip port paralel, pin pentru primirea întreruperilor externe și pin pentru declanșarea hardware a procesului de numărare.

Registrul TCON – Timer Control

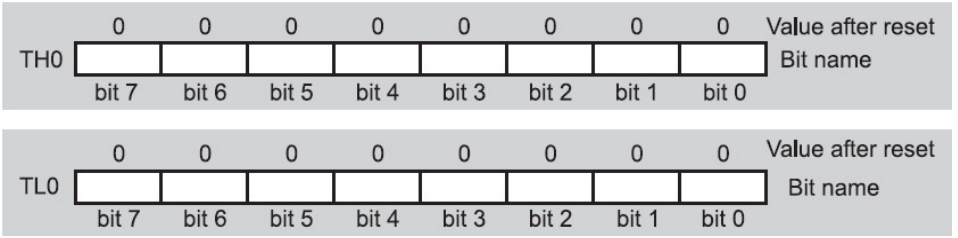
	7	6	5	4	3	2	1	0
	TF_1	TR_1	TF_0	TR_0	Folosiți la logica de întreruperi			

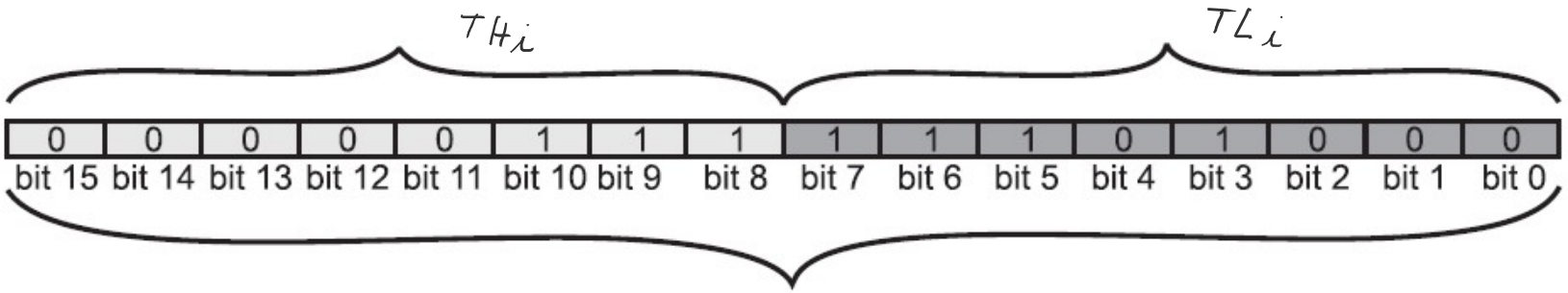
Bitii TR_i (timer run) – bit ce poate fi acționat de utilizator. TR_i=1 validează din punct de vedere software numărarea la canal. Dacă TR_i=0 numărarea este blocată la canalul respectiv.

Bitii TF_i (timer flag) – este de tip read only. Mecanismele canalului timer setează acest bit (1 logic) la finalul numărării, odată cu atingerea stării finale. Acești biți reprezintă sursă de întreruperi pentru procesor. De obicei operarea cu canale timer la 8051 se face cu întreruperi. Începutul tratării unei întrerupere provenite de la canalul timer resetează automat bitul TF_i care a generat întreruperea.

Obs. Numararea pe canalele timer din 8051 se face catre inainte (prin incrementare).

Registrii pentru constanta de divizare (presetare) THi si TLi





Pasi programare canal timer 8051

- Programare TMOD;
- Programare TH, TL la canal;
- Programare TCON (bitul TR <- 1).

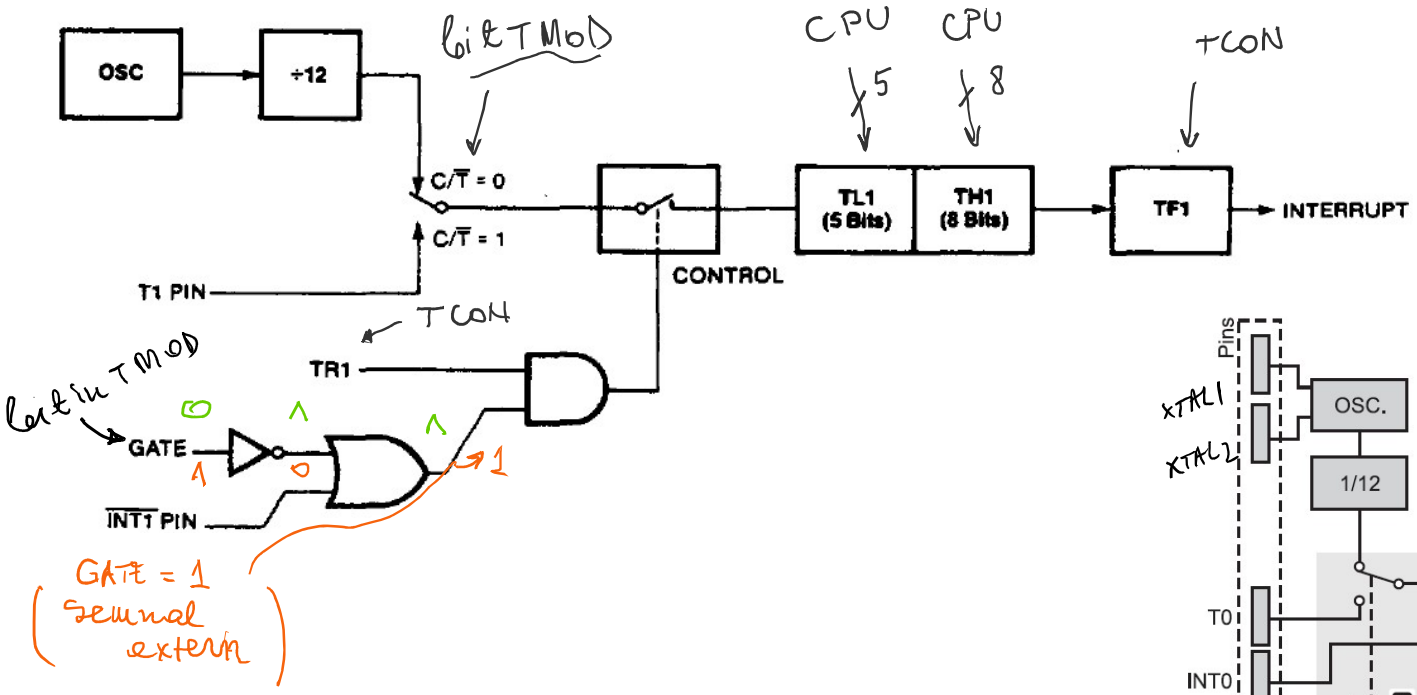
END 18 mar 2026

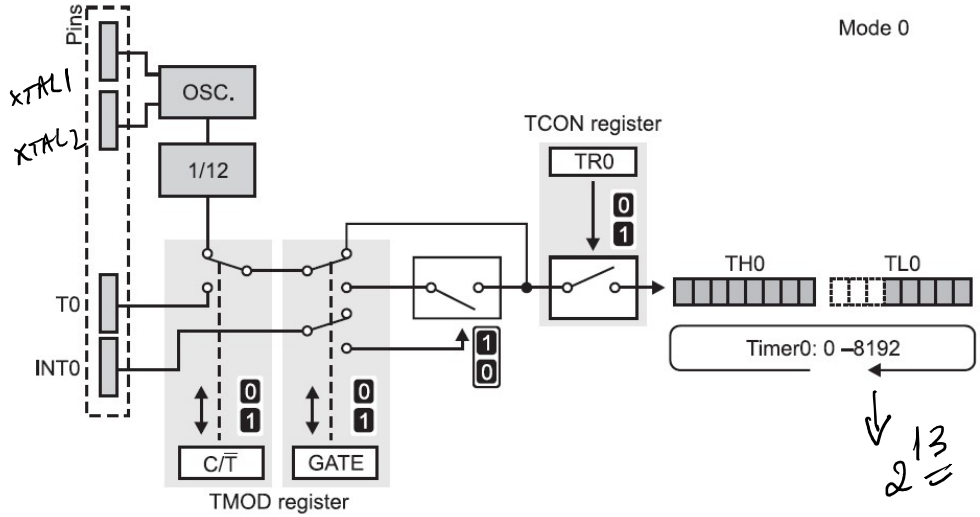
Obs. Numararea pe canalele timer din 8051 se face catre inainte (prin incrementare).

Modurile de operare la canalele Timer din MCU 8051

Modul 0 – cu numarare pe 13 biti si blocare la sfarsitul numararii

Modul 0 a fost proiectat pentru a asigura compatibilitatea cu un circuit anterior (MCU 8048). Pentru a relua procesul de numărare este necesară reîncărcarea lui TH_i și TL_i. Este selectat prin bitii M1 = 0 si M0 = 0 in TMOD.



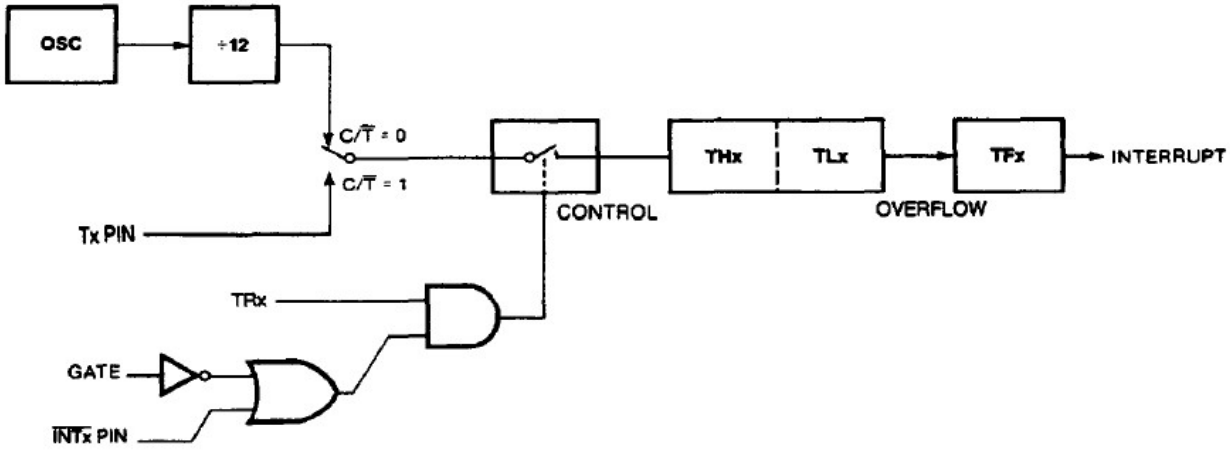


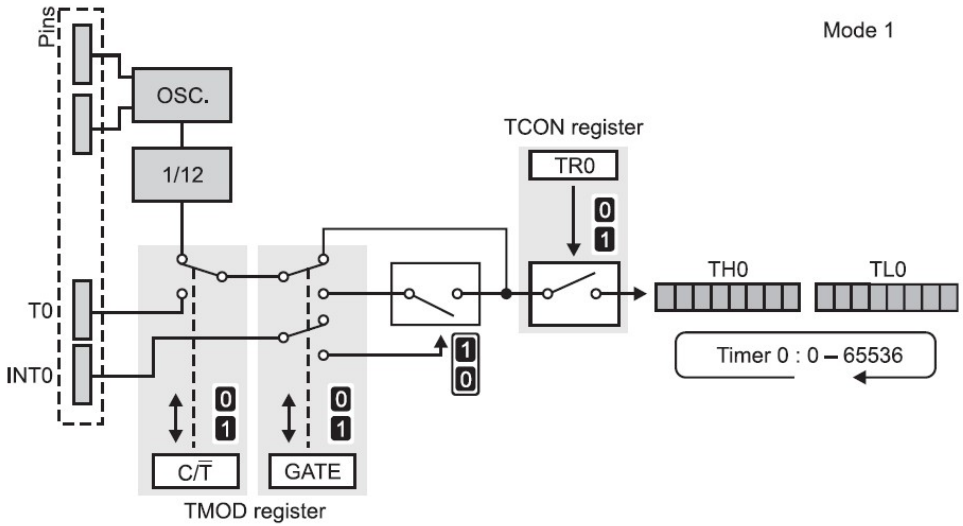
Modurile de operare la canalele Timer din MCU 8051

Modul 1 – cu numarare pe 16 biti si blocare la sfarsitul numararii; modul este selectat prin M1 = 0 si M0 = 1 in TMOD.

Reluarea procesului de numarare se face prin:

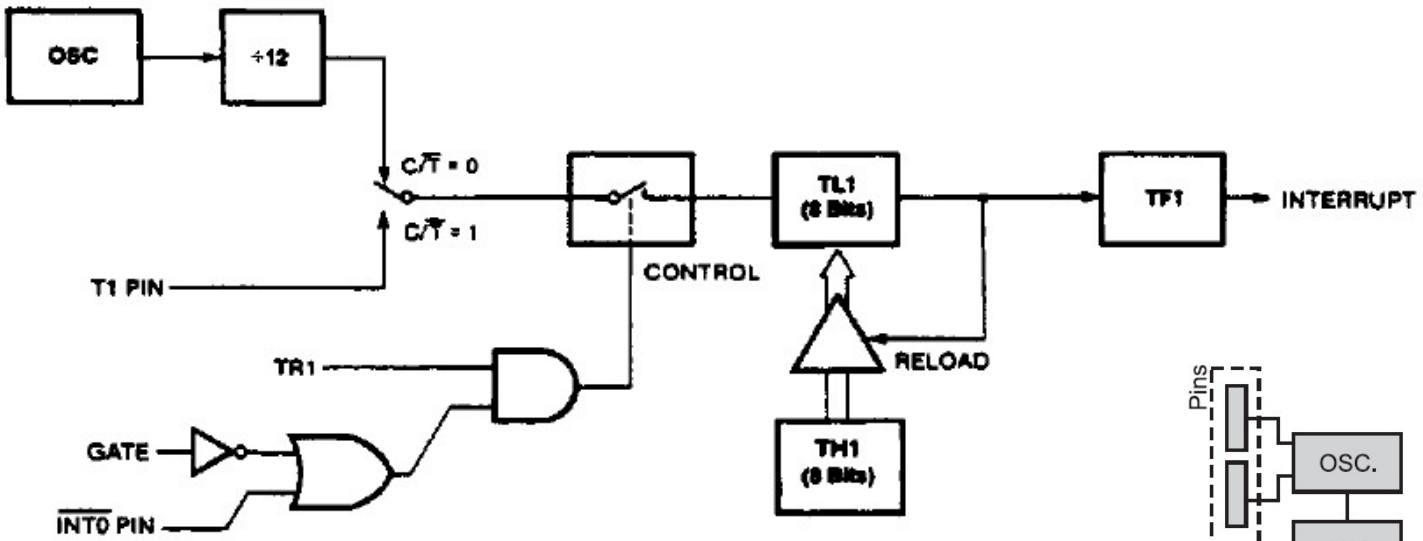
- Programare TH, TL la canal;

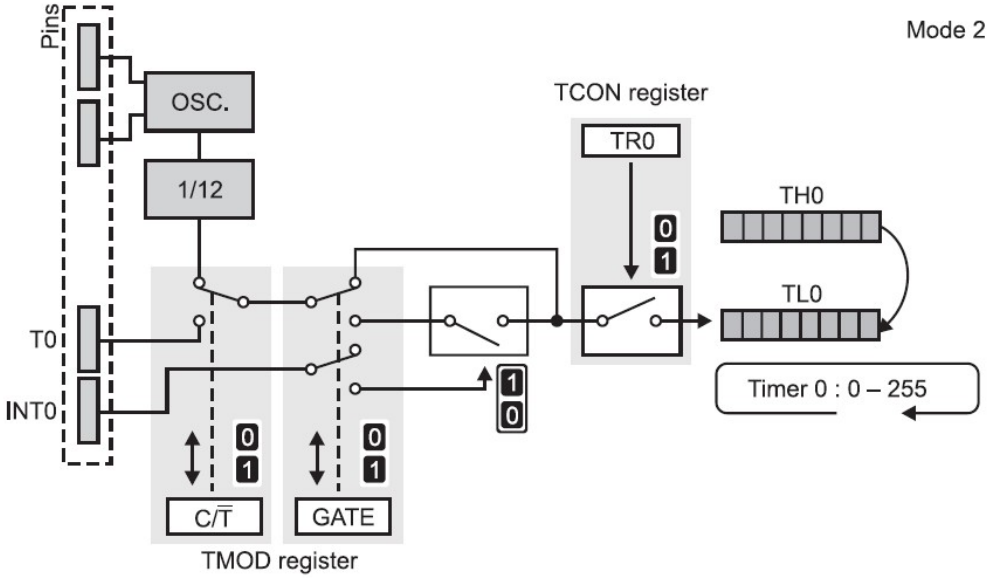




Modurile de operare la canalele Timer din MCU 8051

Modul 2 – cu numarare pe 8 biti si reluare automata la sfarsitul numararii; modul este selectat prin M1 = 1 si M0 = 0 in TMOD.

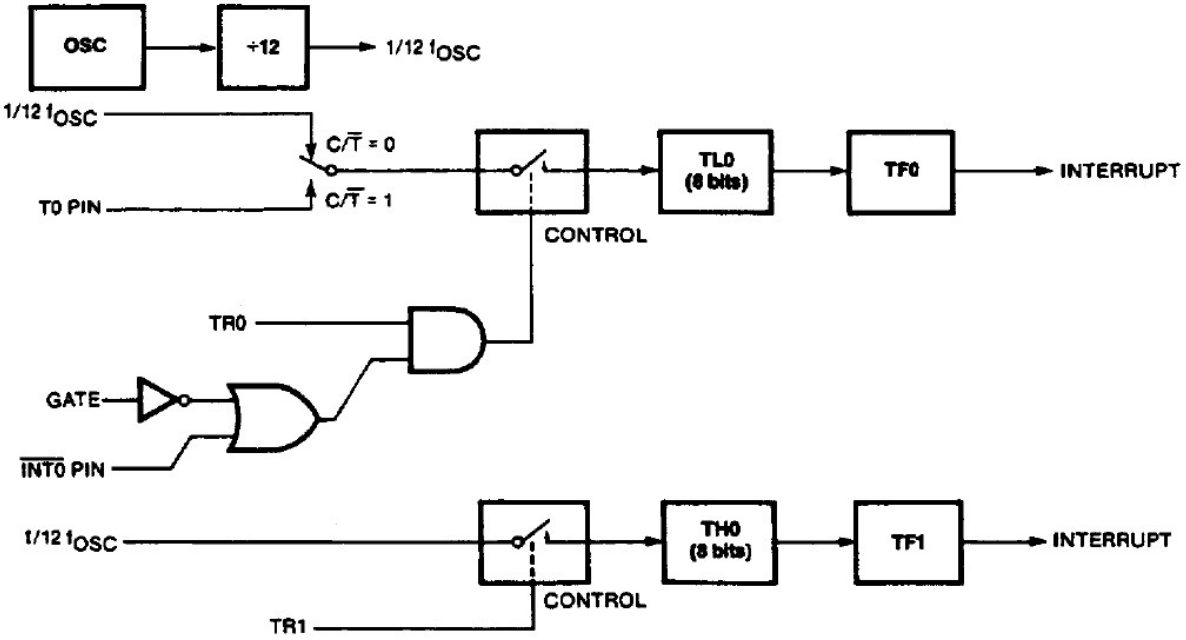


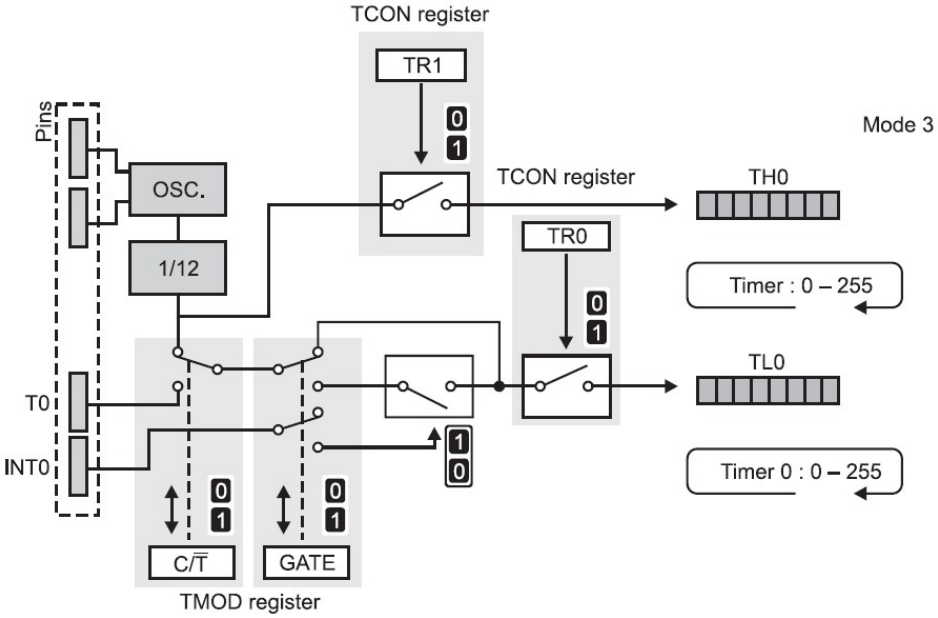


Obs. Timer-ul 1 operând în modul 2 se constituie ca sursă a semnalului de ceas pentru canalul de comunicație serială UART stabilind viteza de comunicație serială (baud-rate).

Modurile de operare la canalele Timer din MCU 8051

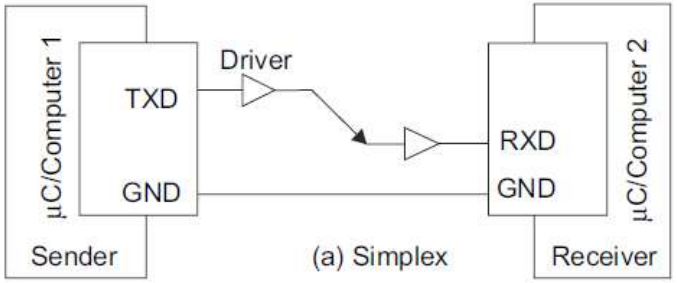
Modul 3 (M1=M0=1 in TMOD; doar la Timer 0)

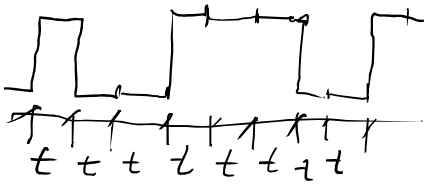




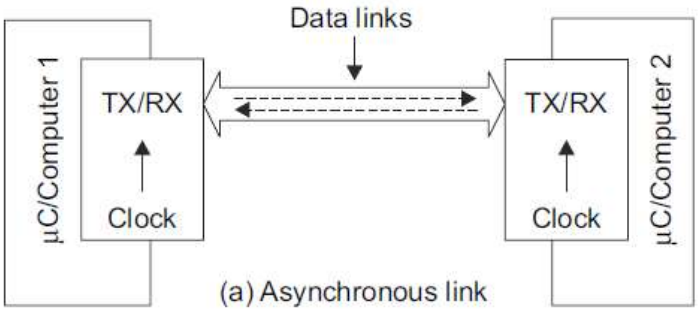
Obs. Modul de operare 3 este posibil doar pentru canalul timer 0 și în condițiile în care timerul 1 este folosit în modul 2 ca sursă a semnalului de ceas pentru UART. În acest mod mai multe resurse ale timerului 1 devin libere de sarcini și pot fi folosite în modul 3 de către timerul 0. În modul 3 apar două canale timer suplimentare la nivelul timerului 0 împreună cu resursele devenite libere din timerul 1.

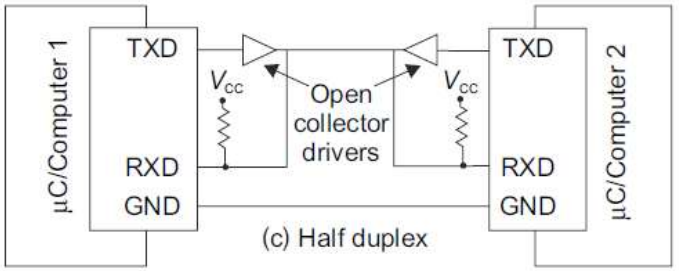
Comunicatii seriale in microsisteme de calcul

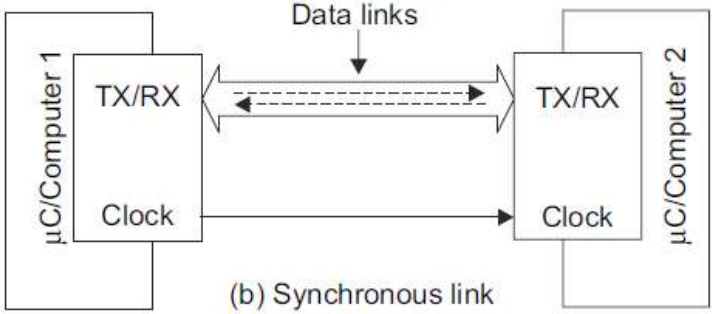


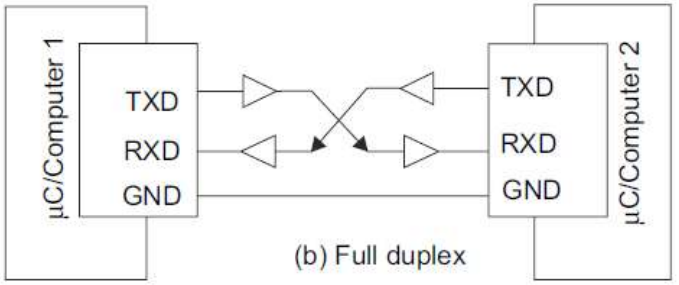


t – timpul celui bit

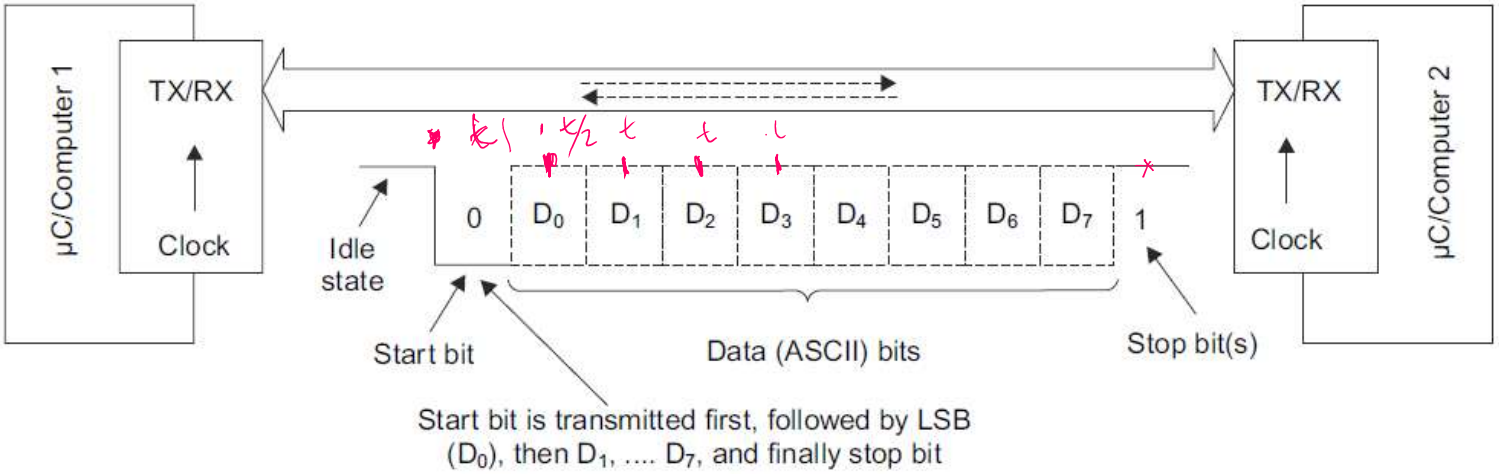






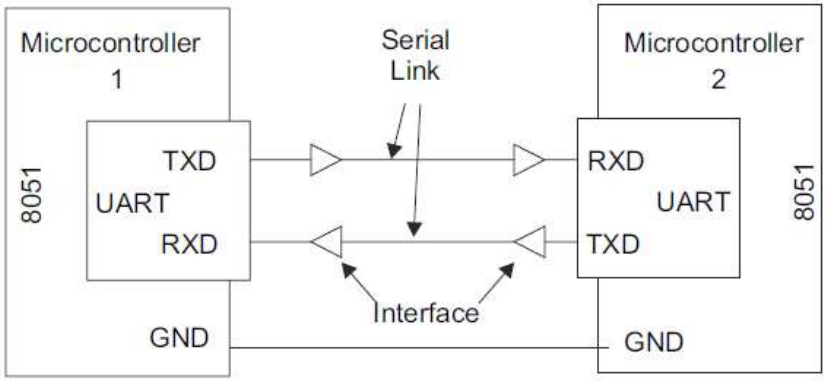


Pachet (frame) de date in format serial asincron



baud vs b/s

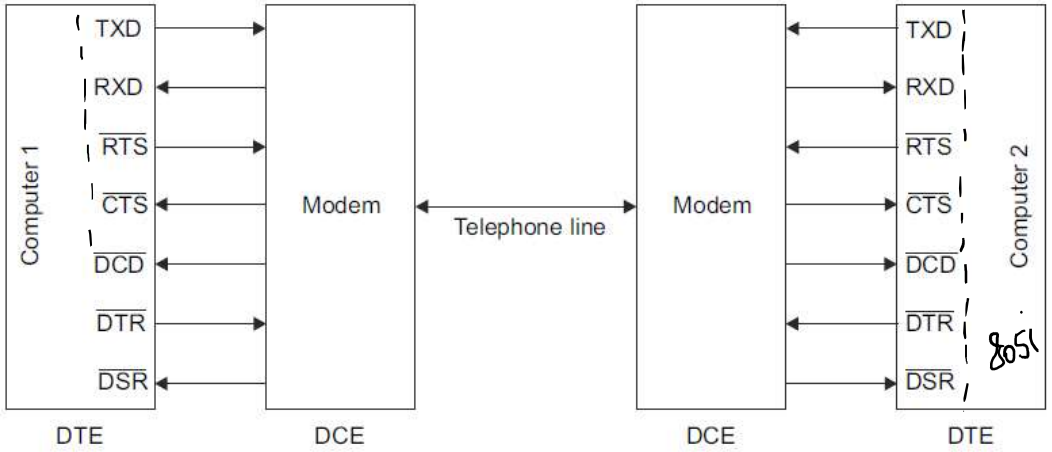
UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER



Viteza de comunicatie seriala se masoara in b/s sau bps.
Multipli folositi: kbps, Mbps, Gbps.

Folosind tehnici de modulatie informatia se mai poate transmite prin simboluri: schimbare de faza, frecvente diferite sau amplitudini diferite (folosind semnale sinusoidale) – baud(s)

RS 232 - SERIAL DATA TRANSMISSION STANDARD



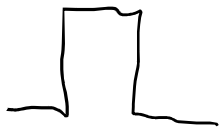
DCE – Data communication equipment
DTE – Data terminal equipment

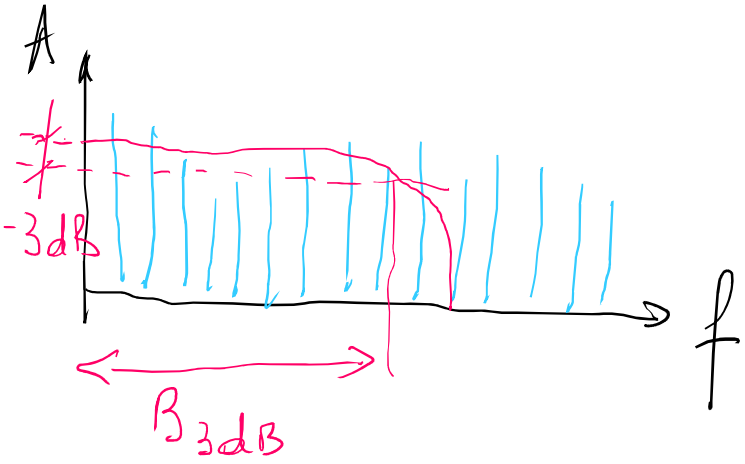
LL –Leased Line

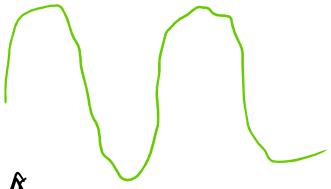
Data Carrier Detect (DCD)
Received Data (RXD)
Transmitted Data (TXD)
Data Terminal Ready (DTR)
Signal Ground (GND)
Data Set Ready (DSR)
Request To Send (RTS)
Clear To Send (CTS)
Ring Indicator (RI)

Protocolul Hand-shaking intre DTE si DCE

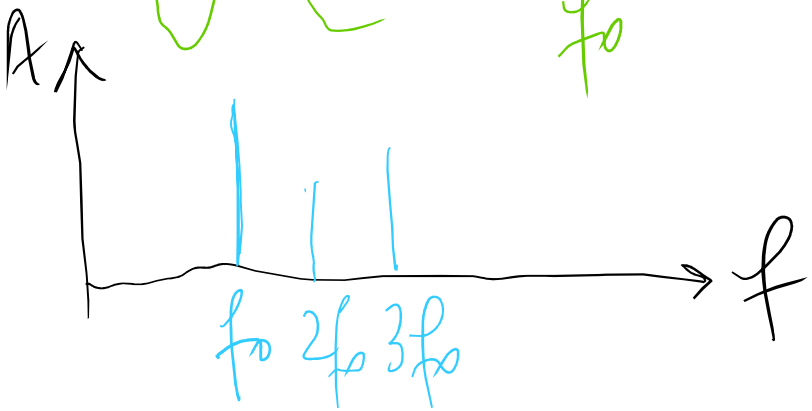
După aplicarea tensiunii de alimentare, DTE activeaza semnalul Data Terminal Ready (DTR) pentru a informa DCE (modemul) că este pregatit pentru comunicatie. Similar, cand modemul este gata, acesta activează semnalul Data Send Ready (DSR) către DTE. Odată ce semnalul DSR a fost primit, DTE face o cerere de utilizare a canalului de date prin activarea semnalului RTS pentru a începe transmisia. Apoi, se apeleaza modemul de la celălalt capăt (capătul receptorului). Acest modem (de obicei în modul answer - răspuns) răspunde prin trimiterea unui semnal cu o frecvență purtătoare de 2255 Hz. Când modemul de la terminalul expeditor primește acest semnal, acesta trimite Data Carrier Detect (DCD) către DTE, ulterior modemul trimite Clear To Send (CTS) care arată că linia este gata pentru transmisie.
Imediat după primirea semnalului CTS, DTE de pe partea transmițătorului trimite date seriale pe iesirea sa TXD.
Similar, are loc un proces similar de tip hand-shaking si pentru sensul invers de comunicatie.

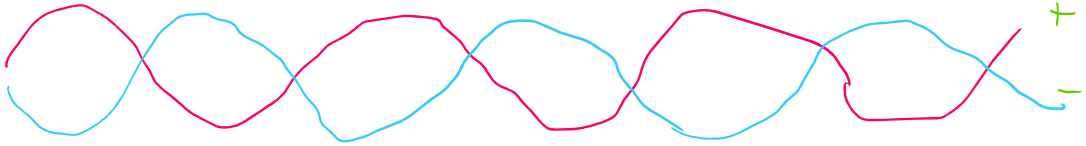


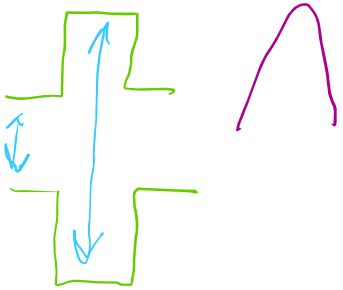




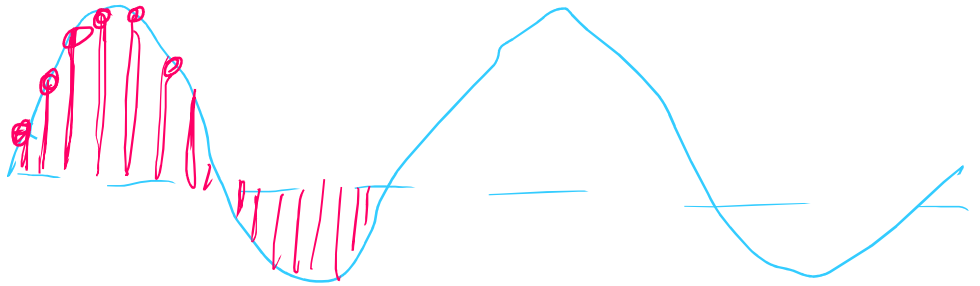
Sinusoidal
f_0







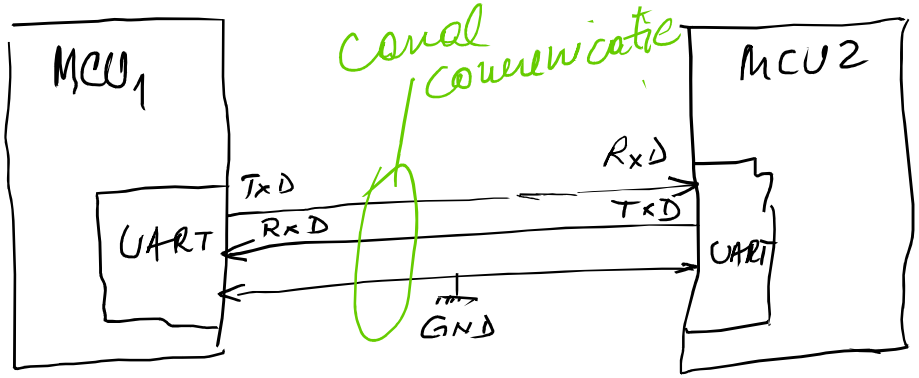
Vot majoritar

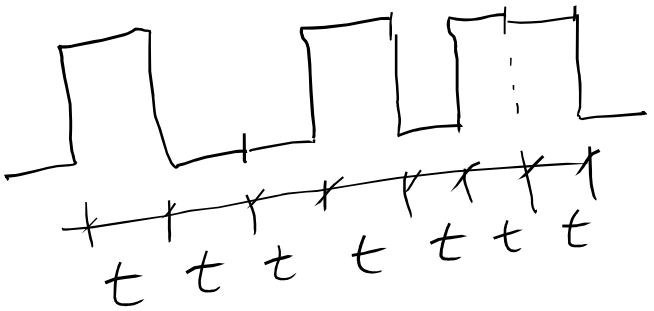


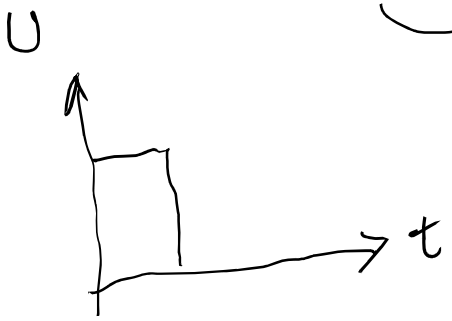
Esantionare
digitală

DAC

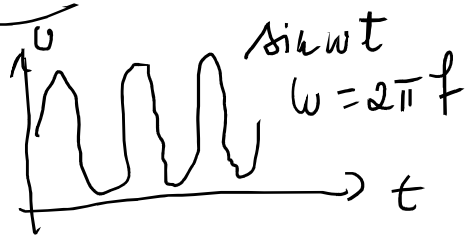
FILTRU
T. J.





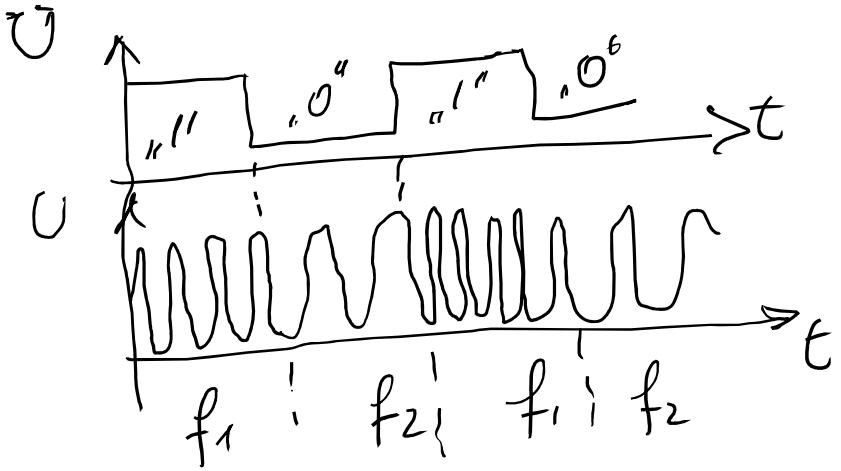


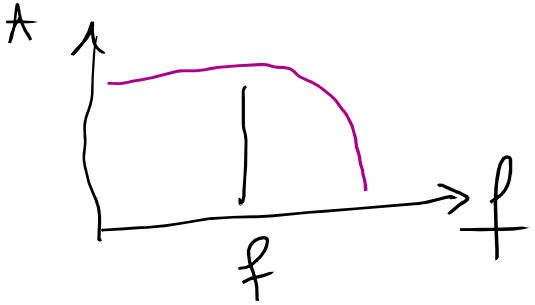
metri

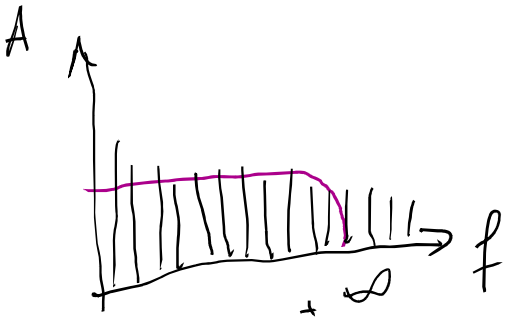


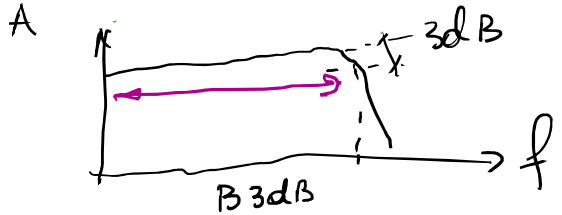
MODEM - modulator demodulator

MF - amplitudine si faza constanta

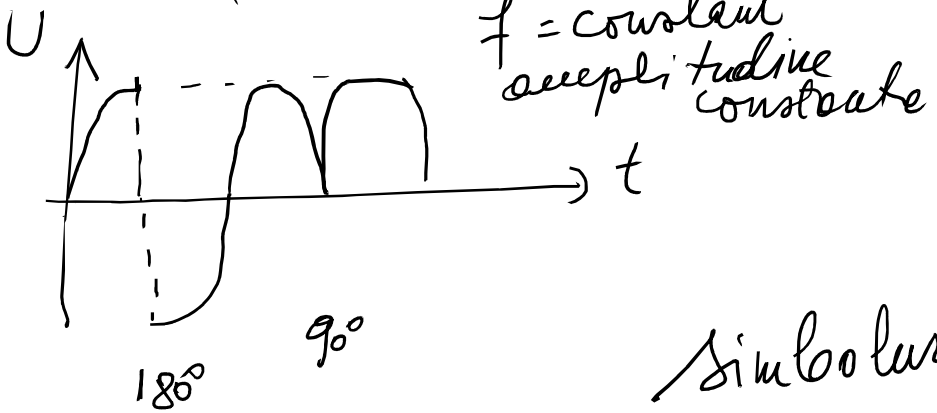






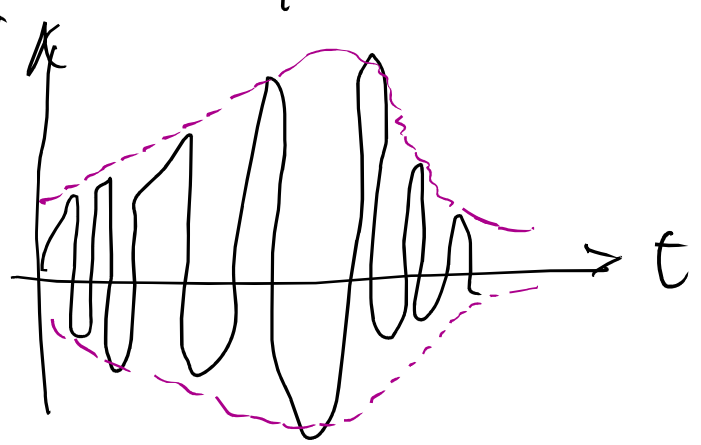


PM - phase modulation

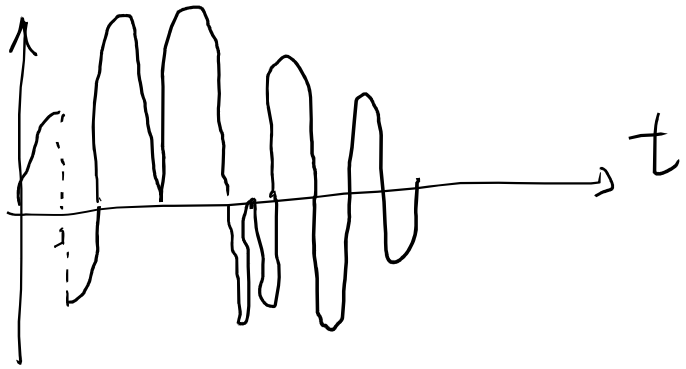


f = constant
amplitudine constantă

AM - amplitude modulation



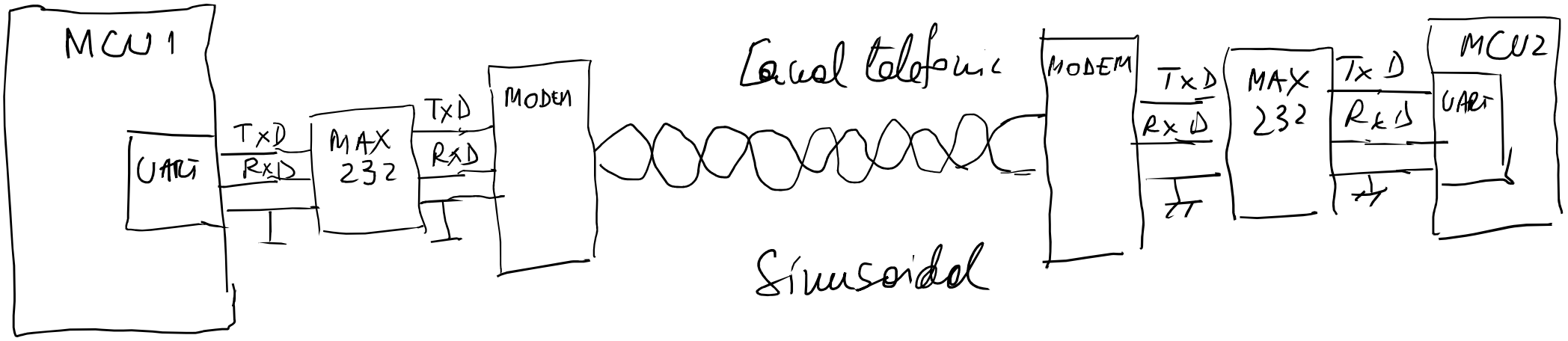
Simboluri



baude

0°	000
45°	001
90°	010
135°	011
180°	100
225°	101
270°	110
315°	111
360°	

0°	00
90°	01
180°	10
270°	11



END 25 03 2026

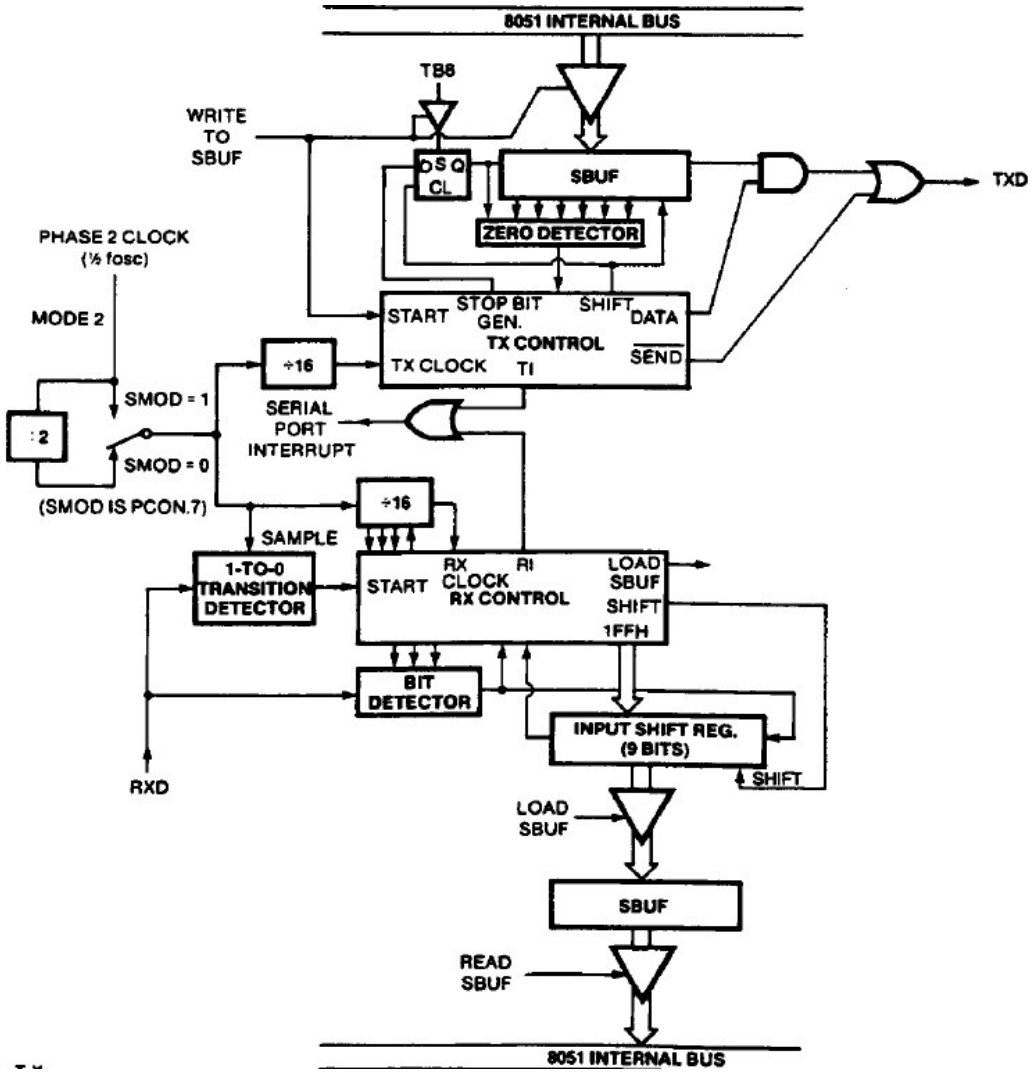
Canalul UART din MCU 8051

SBUF – Serial Buffer
TI – Transmission Interrupt
TB8 - Transmission Bit 8
RB8 – Receive Bit 8
RI – Receive Interrupt
SCON – Serial Control

Viteze de comunicatie seriala RS232:
150bps, 300bps, 600bps, 1200bps, 2400bps, 4800bps, 9600bps, 19200bps, 38,4kbps, …
14,4kbps, 28,8kbps, 57,6kbps

Care este timpul de transmisie al unui pachet serial cu 8 biti date, 1bit start,1bit stop (10 biti) daca viteza de comunicatie este 9,6kbps?

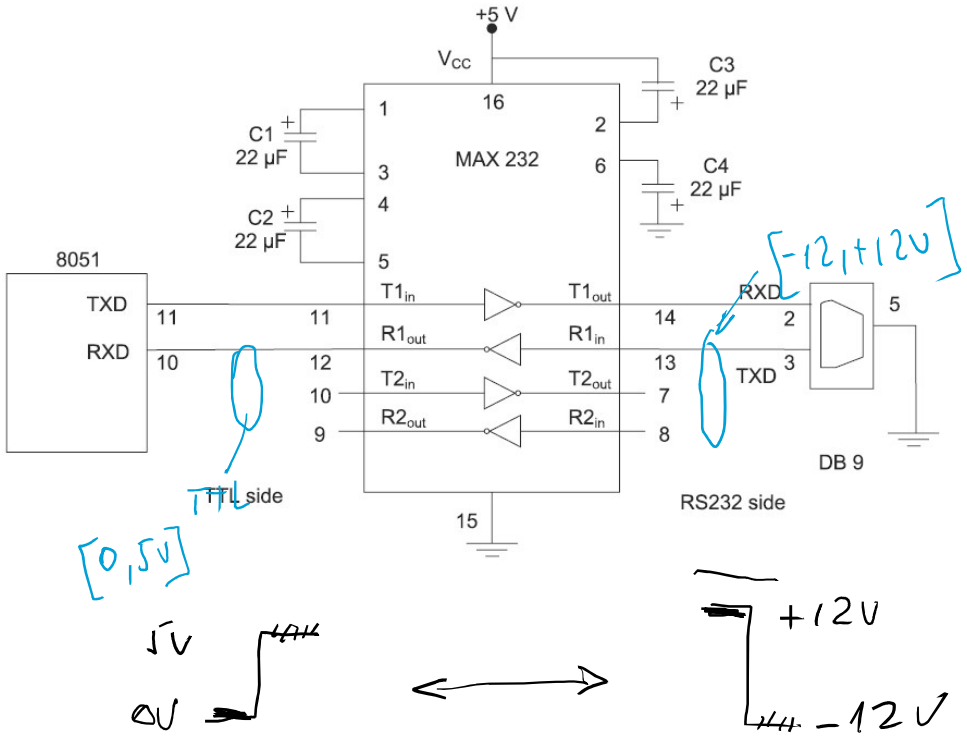
t=1/9600 [s]
Ttransm = t*n = 1/9600*10 = 1/960 [s] = 0,00104[s] = 1,04ms



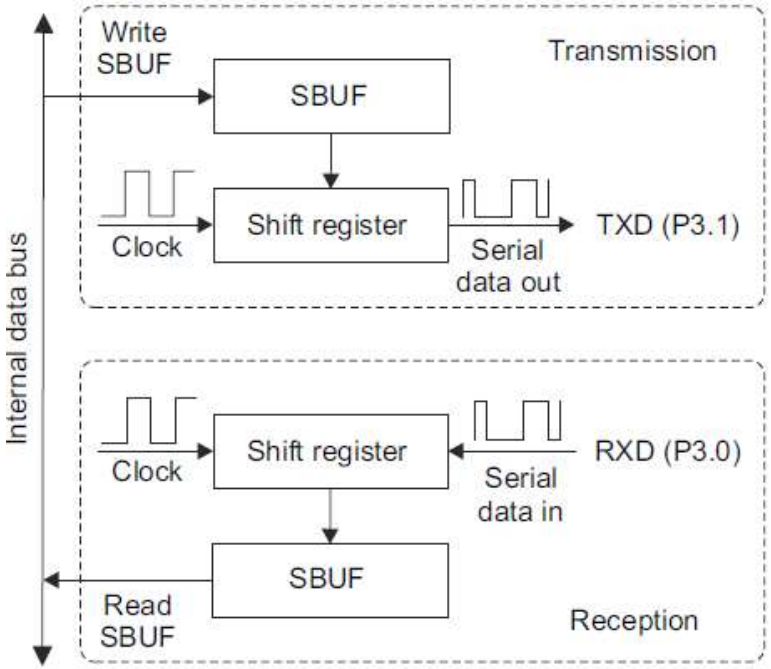
Canalul UART din MCU 8051

UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER

Interfatare RS232 pentru MCU 8051

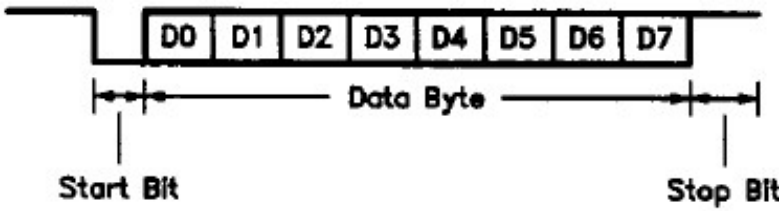


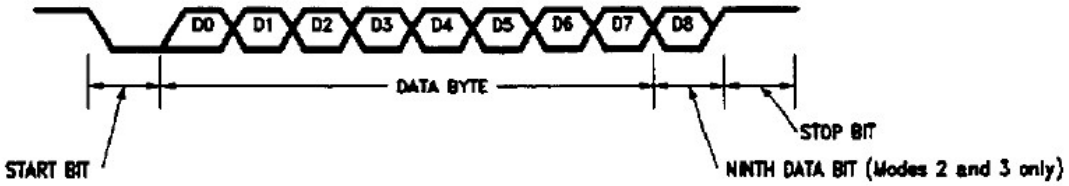
Registrul SBUF din MCU 8051



Canalul UART din MCU 8051

UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER





Registrul SCON din MCU 8051

SCON Register (Serial Port Control Register)

	0	0	0	0	0	0	0	0
SCON	SM0	SM1	SM2	REN	TB8	RB8	TI	RI
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0

Value after reset

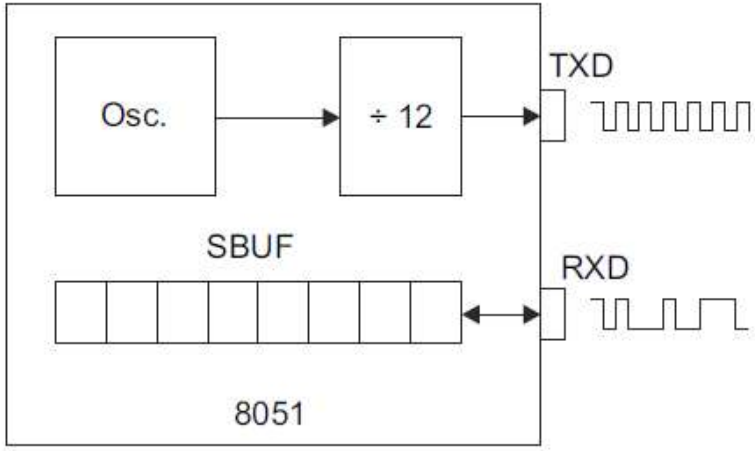
Bit name

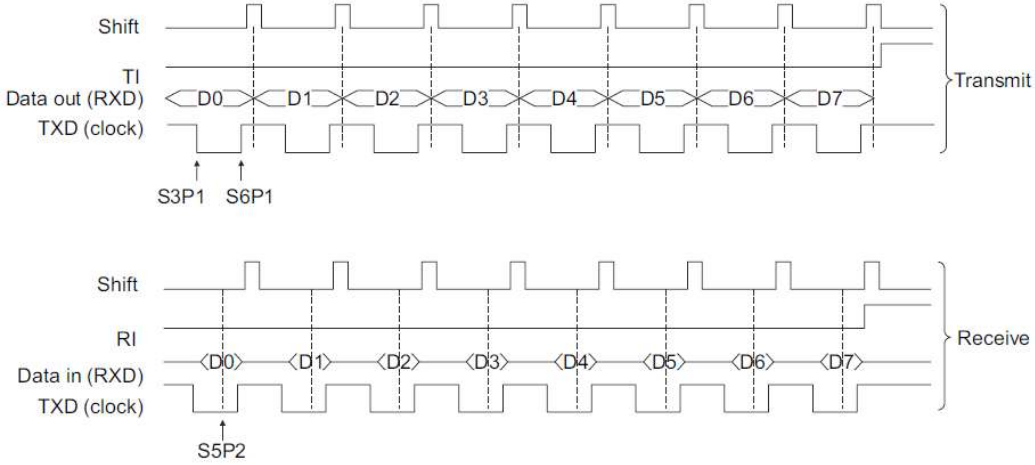
SM0	SM1	Mode	Description	Baud Rate
0	0	0	8-bit Shift Register	1/12 the quartz frequency
0	1	1	8-bit UART	Determined by the timer 1
1	0	2	9-bit UART	1/32 the quartz frequency (1/64 the quartz frequency)
1	1	3	9-bit UART	Determined by the timer 1

SM2 – activeaza modul de lucru multiprocesor in modurile de operare 2 si 3;
REN – receive enable (validare receptie);
TB8 – bitul 8 la transmisie in modurile de operare 2 si 3;
RB8 – bitul 8 la receptie in modurile de operare 2 si 3;
TI – transmit interrupt flag;
RI – receive interrupt flag

Canalul UART din MCU 8051

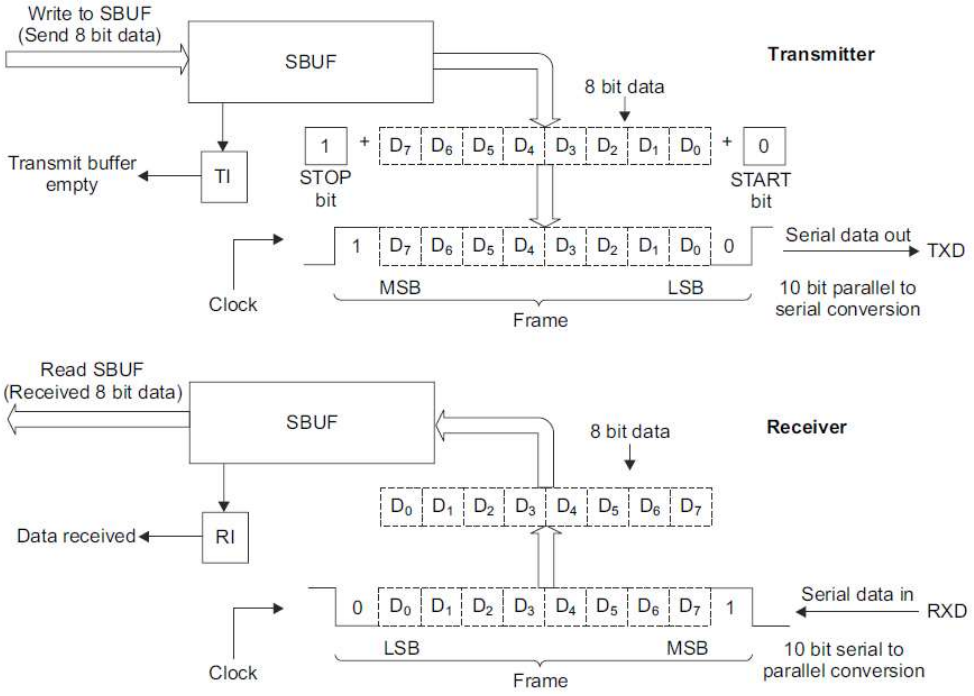
Modul 0 de operare registru serial la UART din MCU 8051



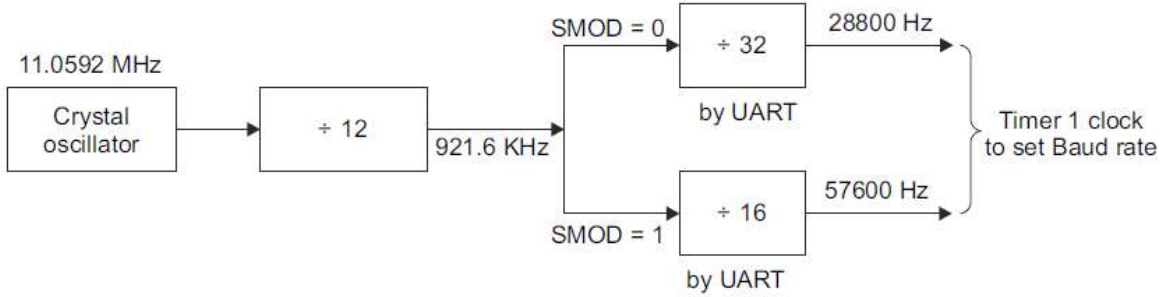


Canalul UART din MCU 8051

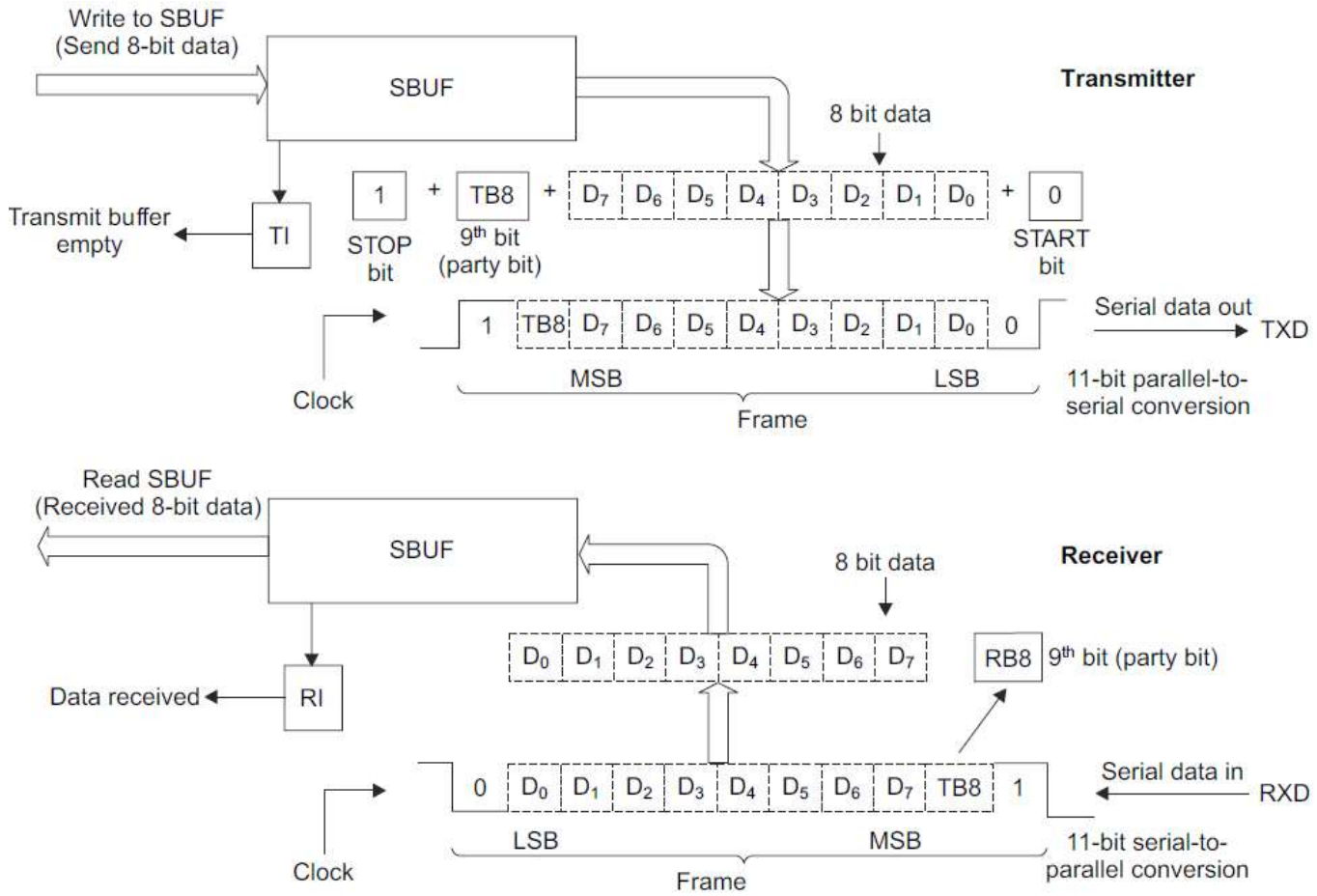
Modul 1 de operare UART din MCU 8051



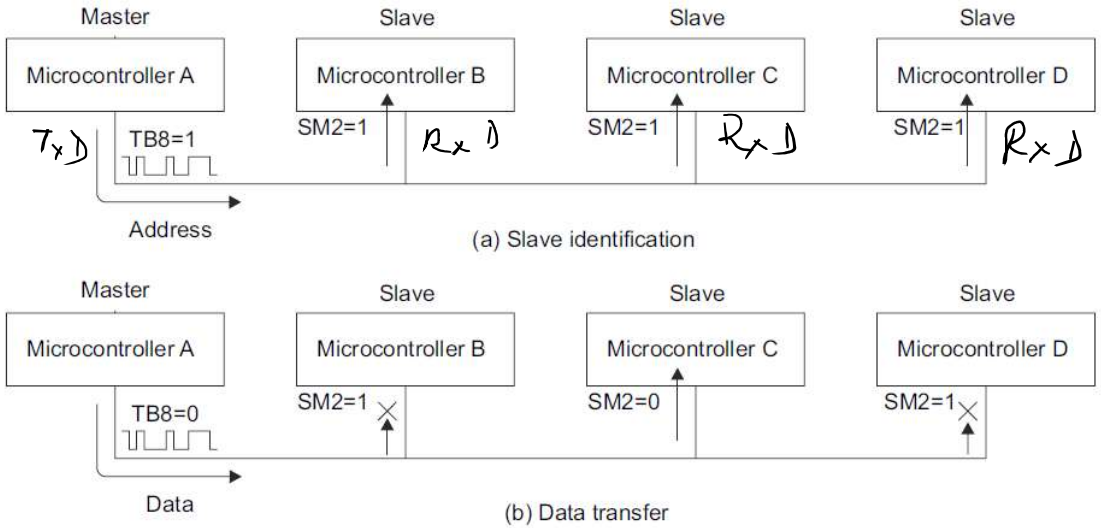
Timerul 1 ca sursa de ceas pentru UART din MCU 8051

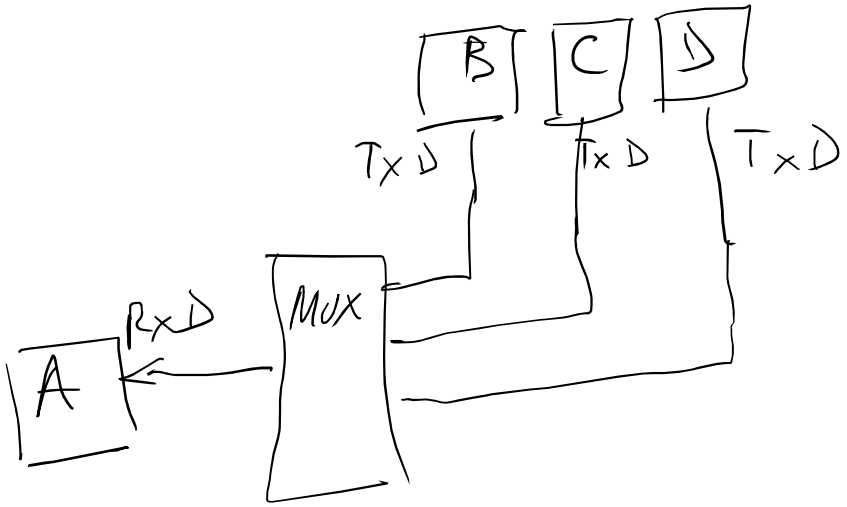


Modul 2 de operare UART din MCU 8051; Modul 3 de operare UART din MCU 8051



Modul Comunicatie multiprocesor





Fiecarui MCU i se atribuie o adresă unică. Un octet de adresă este diferențiat de octetul de date folosind cel de-al 9 bit. Pentru adresă, al nouălea bit = 1, iar pentru octetul de date, acest bit este 0. Când microcontrolerul A (master) dorește să transmită date către unul dintre mai mulți sclavi (să zicem, microcontrolerului C), acesta trimite mai întâi octetul de adresă. (Toți sclavii vor avea inițial SM2 = 1 așa cum se arată în figura). Toți sclavii vor primi acest octet de adresă, generandu-se astfel întrerupere în toți sclavii. Rutina de tratare a întreruperii va verifica dacă se potrivește cu adresa proprie. Deoarece toți sclavii au adrese unice, numai unui sclav (microcontrolerul C) i se va potrivi cu adresa. Astfel sclavul dorit este identificat, iar acest sclav isi va șterge bitul SM2 al registrului SCON și va fi gata să primeasca octeții de date trimiși de master așa cum se arată în figura. Ceilalti sclavi (neadresați) ignoră octeții de date primiți, deoarece bitul lor SM2 este încă 1.

Canalul UART din MCU 8051

Obtinerea frecventelor de comunicatie seriala la canalul UART din MCU 8051

$$\text{Mode 0 Baud Rate} = \frac{\text{Oscillator Frequency}}{12}$$

(quartz)

$$\text{Modes 1, 3 Baud Rate} = \frac{2^{SMOD}}{32} \times \frac{\text{Oscillator Frequency}}{12x\,[256 - (TH1)]}$$

(standardizata) (TIMER 1 in modul 2)

$$\text{Mode 2 Baud Rate} = \frac{2^{SMOD}}{64} \times (\text{Oscillator Frequency})$$

$$\text{Modes 1, 3 Baud Rate} = \frac{2^{SMOD}}{32} \times (\text{Timer 1 Overflow Rate})$$

(TIMER 1 in mod 1)

PCON

TH1

Baud Rate	f_{OSC}	SMOD	Timer 1		
			C/$\overline{T}$	Mode	Reload Value
Mode 0 Max: 1 MHz	12 MHz	X	X	X	X
Mode 2 Max: 375K	12 MHz	1	X	X	X
Modes 1, 3: 62.5K	12 MHz	1	0	2	FFH
19.2K	11.059 MHz	1	0	2	FDH
9.6K	11.059 MHz	0	0	2	FDH
4.8K	11.059 MHz	0	0	2	FAH
2.4K	11.059 MHz	0	0	2	F4H
1.2K	11.059 MHz	0	0	2	E8H
137.5	11.986 MHz	0	0	2	1DH
110	6 MHz	0	0	2	72H
110	12 MHz	0	0	1	FEEBH

Logica de control a intreruperilor in MCU 8051

Gestioneaza 5 surse de intreruperi la nivelul MCU 8051. Trei surse sunt interne: cele 2 canale timer si unitatea UART, iar 2 surse sunt externe (se primesc pe pinii INTi).

Registrul IE
Interrupt Enable

EA	--	[redacted]	ES	ET1	EX1	ET0	EX0
MSB							LSB

Valideaza individual fiecare sursa de intreruperi si global operarea in intreruperi.

Registrul IP
Interrupt Priority

--	--	[redacted]	PS	PT1	PX1	PT0	PX0
MSB							LSB

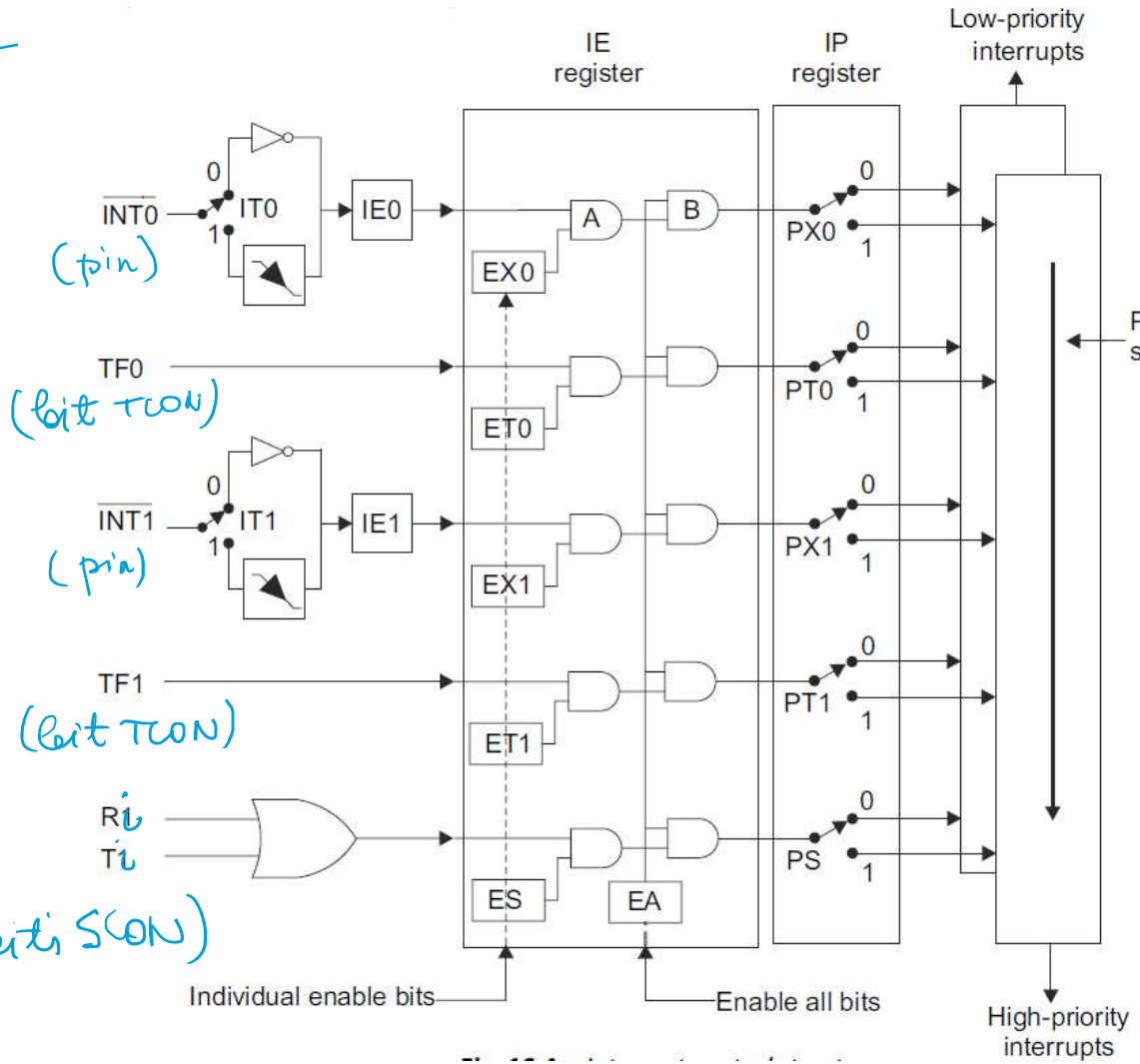
Plaseaza sursele de intreruperi pe 2 nivele de prioritati.

IT_i - interrupt type
IE_i - -11- edge

$IT\phi = 0$
(TCON

$IT\phi = 1$
(TCON)

Logica de control a intreruperilor in MCU 8051



TCON (Timer control)

7 6 5 4 3 2 1 0

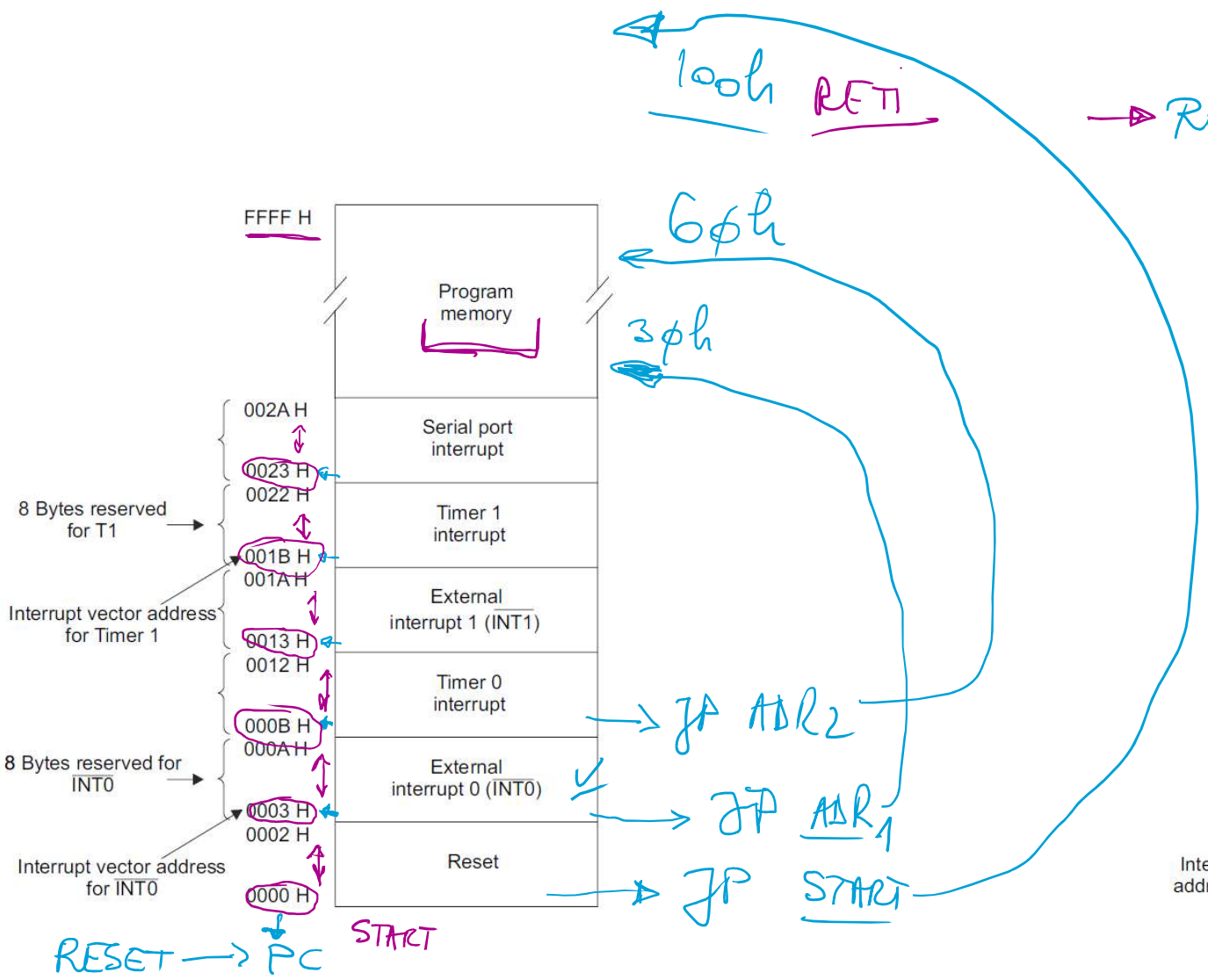
TR_1 TF_1 TR_0 TF_0 = TIMER

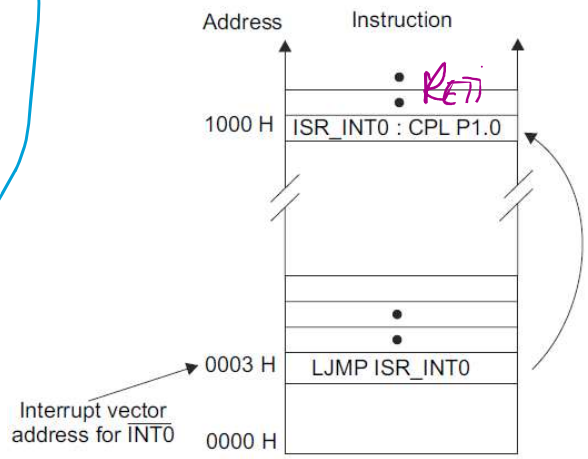
IT_1 IE_1 IT_0 IE_0 = INT. EXT

? 1. SERIALA (H)
2. TF_1 (L)
3. INT_1 (?)

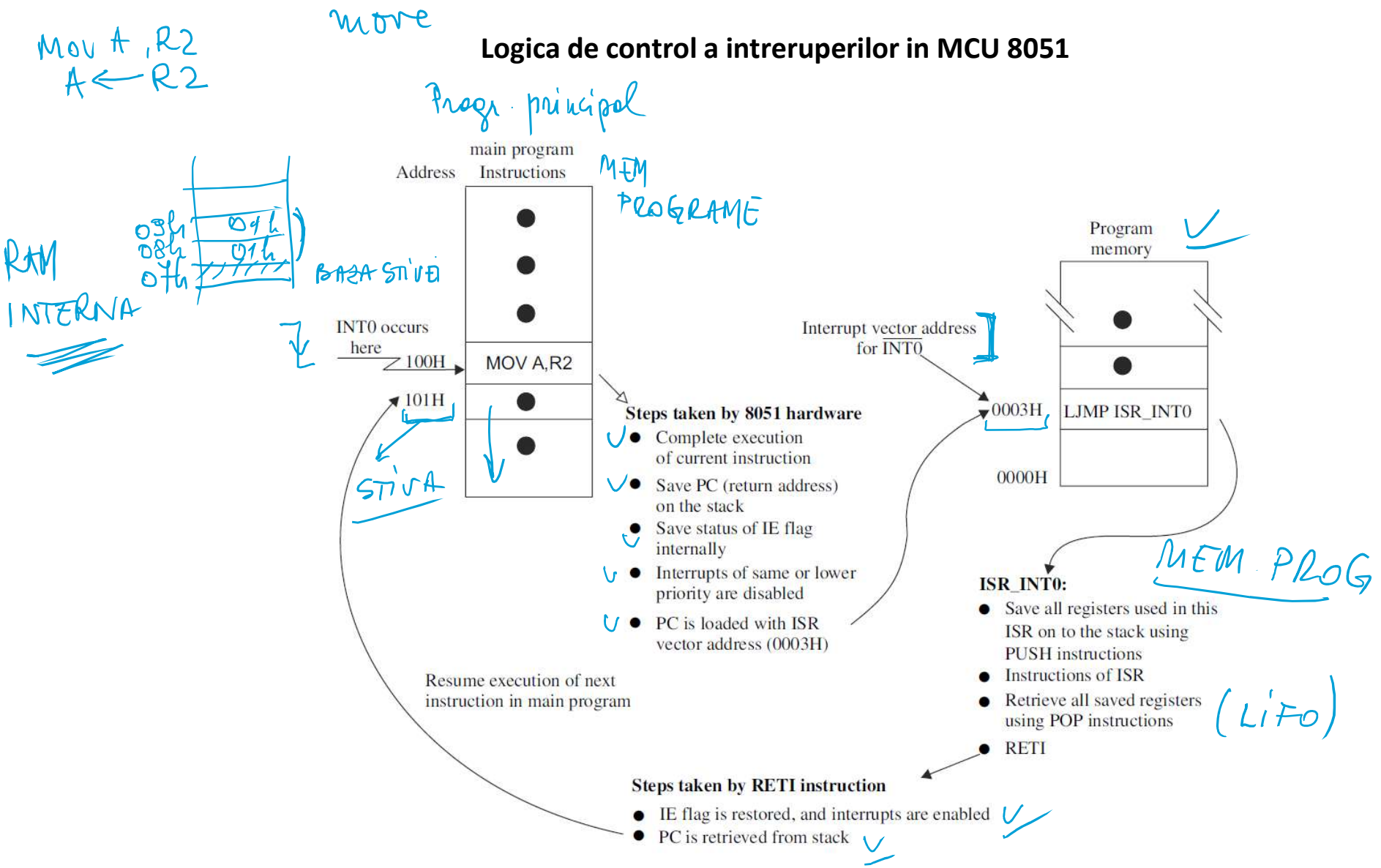
END 1 apr 2026

Logica de control a intreruperilor in MCU 8051





Logica de control a intreruperilor in MCU 8051



END 8 apr 2026

Setul de instructiuni al MCU 8051

Instructiuni aritmetice

Instructiuni logice

Instructiuni de transfer cu memoria de date interna

Instructiuni de transfer cu memoria de date externa

Instructiuni de transfer cu memoria de programe

Instructiuni de manipulare la nivel de bit

Instructiuni de salt, lucru cu subrutine

Instructiuni de salt conditionat

Adresare directa

Adresare indirecta

Adresare cu registru

Adresare imediata

Notatii folosite

Rn	— Register R7–R0 of the currently selected Register Bank.
direct	— 8-bit internal data location's address. This could be an Internal Data RAM location (0–127) or a SFR [i.e., I/O port, control register, status register, etc. (128–255)].
@Ri	— 8-bit internal data RAM location (0–255) addressed indirectly through register R1 or R0.
#data	— 8-bit constant included in instruction.
#data 16	— 16-bit constant included in instruction.
addr 16	— 16-bit destination address. Used by LCALL & LJMP. A branch can be anywhere within the 64K-byte Program Memory address space.
addr 11	— 11-bit destination address. Used by ACALL & AJMP. The branch will be within the same 2K-byte page of program memory as the first byte of the following instruction.
rel	— Signed (two's complement) 8-bit offset byte. Used by SJMP and all conditional jumps. Range is −128 to +127 bytes relative to first byte of the following instruction.
bit	— Direct Addressed bit in Internal Data RAM or Special Function Register.

@Ri

$Ri \in \{R_0, R_1\}$

@Ri ↔ (Ri)

#d8 ; #d16

Instructiuni care afecteaza indicatorii de conditii

Instruction	Flag C	OV	AC	Instruction	Flag C	OV	AC
ADD	X	X	X	CLR C	0		
ADDC	X	X	X	CPL C	X		
SUBB	X	X	X	ANL C,bit	X		
MUL	0	X		ANL C,/bit	X		
DIV	0	X		ORL C,bit	X		
DA	X			ORL C,bit	X		
RRC	X			MOV C,bit	X		
RLC	X			CJNE	X		
SETB C	1						

ADD A, 31h A ← A + (31h)

ADD A, #31h A ← A + #31h

MOV R1, 31h
ADD A, @R1 R1 ← (31h)

MOV R1, #31h
ADD A, @R1 A ← A + (?) ((R1))

ADDC - addition with carry
SUBB - Substract with borrow

Instructiuni aritmetice

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
ADD A,<byte>	A = A + <byte>	X	X	X	X	1
ADDC A,<byte>	A = A + <byte> + C	X	X	X	X	1
SUBB A,<byte>	A = A − <byte> − C	X	X	X	X	1
INC A	A = A + 1	Accumulator only				1
INC <byte>	<byte> = <byte> + 1	X	X	X		1
INC DPTR	DPTR = DPTR + 1	Data Pointer only				2
DEC A	A = A − 1	Accumulator only				1
DEC <byte>	<byte> = <byte> − 1	X	X	X		1
MUL AB	B:A = B x A	ACC and B only				4
DIV AB	A = Int [A/B] B = Mod [A/B]	ACC and B only				4
DA A	Decimal Adjust	Accumulator only				1

Cy ← 0
SUBB ≡ SUB

SUBB - Substract with Borrow

C = Carry

MUL - multiply
DIV - divide

24 0010 0100 (BCD)
↓ (10)
18(h) 0001 1000 (BCD)

CLK = 12MHz

ADD - addition
ADD A, FFh ≡ { MOV R0, #FFh / ADD A, @R0 } ⟺
ADD A, #FFh ≡ { MOV R7, #FFh / ADD A, R7 }

ADD	A,7FH	(direct addressing)
ADD	A,@R0	(indirect addressing)
ADD	A,R7	(register addressing)
ADD	A,#127	(immediate constant)

A ← A + (FFh)
A ← A + (R0)
A ← A + R7
A ← A + #FFh

A = 7
B = 3
A:B → Cat 2, Rest 1

79h — 7·16 + 9 = 112 + 9 = 121 (10)
79(10) BCD → 0111 1001 (2)

=(10)
ADD A, #FFh ≡ FFh

DIV: 7 / 3 → 2 (A), 1 (B)

Mnemonic		Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS				
ADD	A,Rn	Add register to Accumulator	1	12
ADD	A,direct	Add direct byte to Accumulator	2	12
ADD	A,@Ri	Add indirect RAM to Accumulator	1	12
ADD	A,#data	Add immediate data to Accumulator	2	12
ADDC	A,Rn	Add register to Accumulator with Carry	1	12
ADDC	A,direct	Add direct byte to Accumulator with Carry	2	12
ADDC	A,@Ri	Add indirect RAM to Accumulator with Carry	1	12
ADDC	A,#data	Add immediate data to Acc with Carry	2	12
SUBB	A,Rn	Subtract Register from Acc with borrow	1	12
SUBB	A,direct	Subtract direct byte from Acc with borrow	2	12
SUBB	A,@Ri	Subtract indirect RAM from ACC with borrow	1	12
SUBB	A,#data	Subtract immediate data from Acc with borrow	2	12
INC	A	Increment Accumulator	1	12
INC	Rn	Increment register	1	12
INC	direct	Increment direct byte	2	12
INC	@Ri	Increment direct RAM	1	12
DEC	A	Decrement Accumulator	1	12
DEC	Rn	Decrement Register	1	12
DEC	direct	Decrement direct byte	2	12
DEC	@Ri	Decrement indirect RAM	1	12

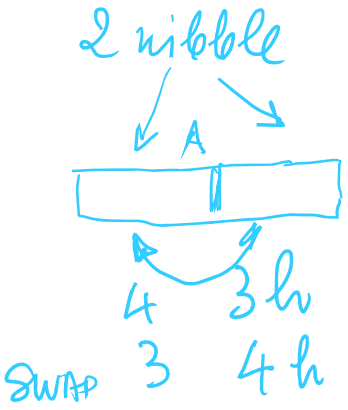
Mnemonic		Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS (Continued)				
INC	DPTR	Increment Data Pointer	1	24
MUL	AB	Multiply A & B	1	48
DIV	AB	Divide A by B	1	48
DA	A	Decimal Adjust Accumulator	1	12

Instructiuni logice

$f_{CLK} = 12MHz$

25h ← 25h AND A
15h ← 25h AND #25h

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
ANL A,<byte>	A = A .AND. <byte>	X	X	X	X	1
ANL <byte>,A	<byte> = <byte> .AND. A	X				1
ANL <byte>,#data	<byte> = <byte> .AND. #data	X				2
ORL A,<byte>	A = A .OR. <byte>	X	X	X	X	1
ORL <byte>,A	<byte> = <byte> .OR. A	X				1
ORL <byte>,#data	<byte> = <byte> .OR. #data	X				2
XRL A,<byte>	A = A .XOR. <byte>	X	X	X	X	1
XRL <byte>,A	<byte> = <byte> .XOR. A	X				1
XRL <byte>,#data	<byte> = <byte> .XOR. #data	X				2
~~CRL~~ A ← CLR A	A = 00H	Accumulator only				1
CPL A	A = .NOT. A	Accumulator only				1
RL A	Rotate ACC Left 1 bit	Accumulator only				1
RLC A	Rotate Left through Carry	Accumulator only				1
RR A	Rotate ACC Right 1 bit	Accumulator only				1
RRC A	Rotate Right through Carry	Accumulator only				1
SWAP A	Swap Nibbles in A	Accumulator only				1

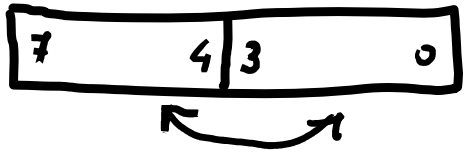



```
ANL   A,7FH     (direct addressing)
ANL   A,@R1     (indirect addressing)
ANL   A,R6      (register addressing)
ANL   A,#53H    (immediate constant)
```

Mnemonic		Description	Byte	Oscillator Period
LOGICAL OPERATIONS				
ANL	A,Rn	AND Register to Accumulator	1	12
ANL	A,direct	AND direct byte to Accumulator	2	12
ANL	A,@Ri	AND indirect RAM to Accumulator	1	12
ANL	A,#data	AND immediate data to Accumulator	2	12
ANL	direct,A	AND Accumulator to direct byte	2	12
ANL	direct,#data	AND immediate data to direct byte	3	24
ORL	A,Rn	OR register to Accumulator	1	12
ORL	A,direct	OR direct byte to Accumulator	2	12
ORL	A,@Ri	OR indirect RAM to Accumulator	1	12
ORL	A,#data	OR immediate data to Accumulator	2	12
ORL	direct,A	OR Accumulator to direct byte	2	12
ORL	direct,#data	OR immediate data to direct byte	3	24
XRL	A,Rn	Exclusive-OR register to Accumulator	1	12
XRL	A,direct	Exclusive-OR direct byte to Accumulator	2	12
XRL	A,@Ri	Exclusive-OR indirect RAM to Accumulator	1	12
XRL	A,#data	Exclusive-OR immediate data to Accumulator	2	12
XRL	direct,A	Exclusive-OR Accumulator to direct byte	2	12
XRL	direct,#data	Exclusive-OR immediate data to direct byte	3	24
CLR	A	Clear Accumulator	1	12
CPL	A	Complement Accumulator	1	12

Mnemonic		Description	Byte	Oscillator Period
LOGICAL OPERATIONS (Continued)				
RL	A	Rotate Accumulator Left	1	12
RLC	A	Rotate Accumulator Left through the Carry	1	12
RR	A	Rotate Accumulator Right	1	12
RRC	A	Rotate Accumulator Right through the Carry	1	12
SWAP	A	Swap nibbles within the Accumulator	1	12

A



END 8 apr 2026

dest - adr ind R_1
src - adr reg R_4

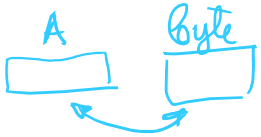
MOV - move

Instructiuni de transfer cu memoria de date interna

$f_{CLK} = 12MHz$

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
MOV A,<src>	A = <src>	X	X	X	X	1
MOV <dest>,A	<dest> = A	X	X	X		1
MOV <dest>, <src>	<dest> = <src>	X	X	X	X	2
MOV DPTR,#data16	DPTR = 16-bit immediate constant.				X	2
PUSH <src>	INC SP : MOV "@SP",<src>	X				2
POP <dest>	MOV <dest>, "@SP" : DEC SP	X				2
XCH A,<byte>	ACC and <byte> exchange data	X	X	X		1
XCHD A,@Ri	ACC and @Ri exchange low nibbles		X			1

MOV @R_1, R_4 ←



XCHD

Mnemonic		Description	Byte	Oscillator Period
DATA TRANSFER				
MOV	A,Rn	Move register to Accumulator	1	12
MOV	A,direct	Move direct byte to Accumulator	2	12
MOV	A,@Ri	Move indirect RAM to Accumulator	1	12
MOV	A,#data	Move immediate data to Accumulator	2	12
MOV	Rn,A	Move Accumulator to register	1	12
MOV	Rn,direct	Move direct byte to register	2	24
MOV	Rn,#data	Move immediate data to register	2	12
MOV	direct,A	Move Accumulator to direct byte	2	12
MOV	direct,Rn	Move register to direct byte	2	24
MOV	direct,direct	Move direct byte to direct	3	24
MOV	direct,@Ri	Move indirect RAM to direct byte	2	24
MOV	direct,#data	Move immediate data to direct byte	3	24
MOV	@Ri,A	Move Accumulator to indirect RAM	1	12

$f_{clk} = 12\,MHz$

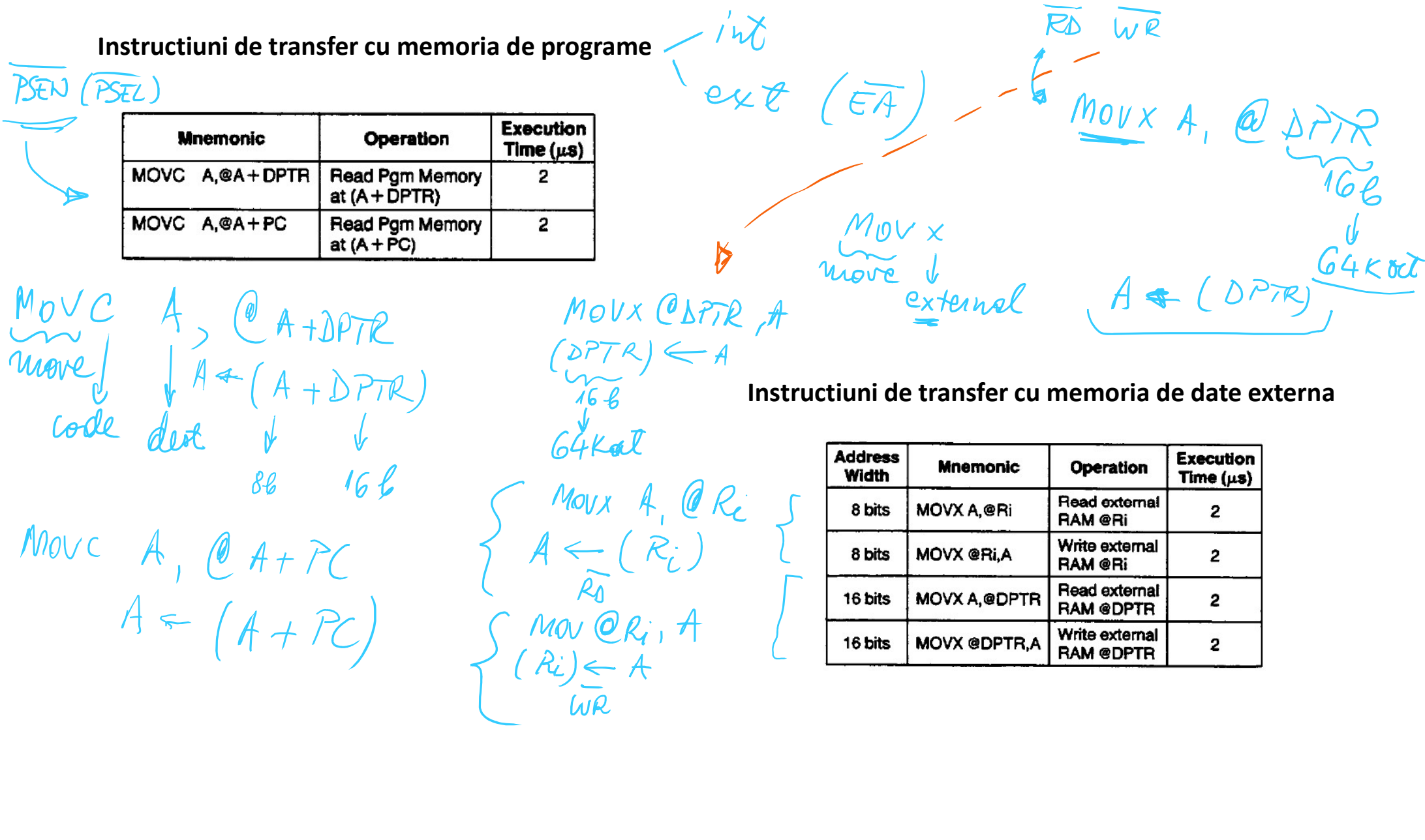
$12\,T = 1\,\mu s$

$1\,T = \frac{1}{12}\,\mu s$

Mnemonic		Description	Byte	Oscillator Period
DATA TRANSFER (Continued)				
MOV	@Ri,direct	Move direct byte to indirect RAM	2	24
MOV	@Ri,#data	Move immediate data to indirect RAM	2	12
MOV	DPTR,#data16	Load Data Pointer with a 16-bit constant	3	24
MOVC	A,@A+DPTR	Move Code byte relative to DPTR to Acc	1	24
MOVC	A,@A+PC	Move Code byte relative to PC to Acc	1	24
MOVX	A,@Ri	Move External RAM (8-bit addr) to Acc	1	24
MOVX	A,@DPTR	Move External RAM (16-bit addr) to Acc	1	24
MOVX	@Ri,A	Move Acc to External RAM (8-bit addr)	1	24
MOVX	@DPTR,A	Move Acc to External RAM (16-bit addr)	1	24
PUSH	direct	Push direct byte onto stack	2	24
POP	direct	Pop direct byte from stack	2	24
XCH	A,Rn	Exchange register with Accumulator	1	12
XCH	A,direct	Exchange direct byte with Accumulator	2	12
XCH	A,@Ri	Exchange indirect RAM with Accumulator	1	12
XCHD	A,@Ri	Exchange low-order Digit indirect RAM with Acc	1	12

Instructiuni de transfer cu memoria de programe

Mnemonic	Operation	Execution Time (μs)
MOVC A,@A+DPTR	Read Pgm Memory at (A+DPTR)	2
MOVC A,@A+PC	Read Pgm Memory at (A+PC)	2

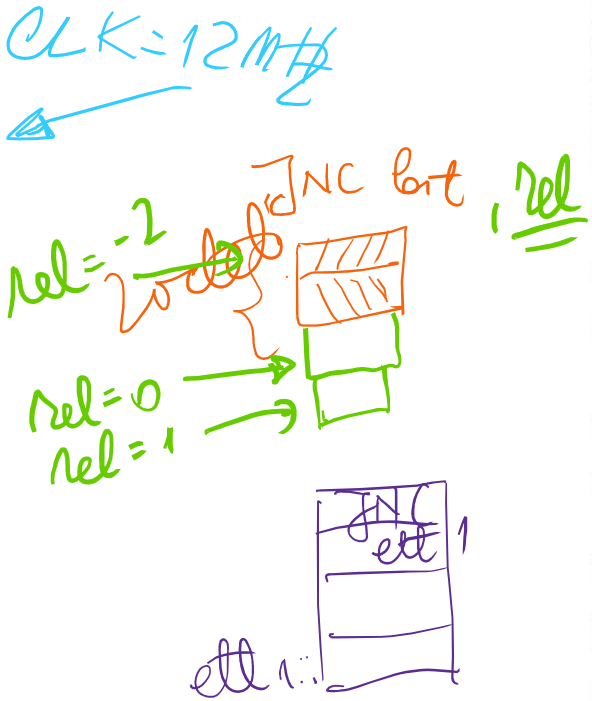


Instructiuni de transfer cu memoria de date externa

Address Width	Mnemonic	Operation	Execution Time (μs)
8 bits	MOVX A,@Ri	Read external RAM @Ri	2
8 bits	MOVX @Ri,A	Write external RAM @Ri	2
16 bits	MOVX A,@DPTR	Read external RAM @DPTR	2
16 bits	MOVX @DPTR,A	Write external RAM @DPTR	2

Instructiuni de manipulare la nivel de bit

Mnemonic		Operation	Execution Time (μs)
ANL	C,bit	C = C .AND. bit	2
ANL	C,/bit	C = C .AND. .NOT. bit	2
ORL	C,bit	C = C .OR. bit	2
ORL	C,/bit	C = C .OR. .NOT. bit	2
MOV	C,bit	C = bit	1
MOV	bit,C	bit = C	2
CLR	C	C = 0	1
CLR	bit	bit = 0	1
SETB	C	C = 1	1
SETB	bit	bit = 1	1
CPL	C	C = .NOT. C	1
CPL	bit	bit = .NOT. bit	1
JC	rel	Jump if C = 1	2
JNC	rel	Jump if C = 0	2
JB	bit,rel	Jump if bit = 1	2
JNB	bit,rel	Jump if bit = 0	2
JBC	bit,rel	Jump if bit = 1; CLR bit	2



JBC – jump if bit (bit=1) and clear bit (bit=0)

Mnemonic		Description	Byte	Oscillator Period
BOOLEAN VARIABLE MANIPULATION				
CLR	C	Clear Carry	1	12
CLR	bit	Clear direct bit	2	12
SETB	C	Set Carry	1	12
SETB	bit	Set direct bit	2	12
CPL	C	Complement Carry	1	12
CPL	bit	Complement direct bit	2	12
ANL	C,bit	AND direct bit to CARRY	2	24
ANL	C,/bit	AND complement of direct bit to Carry	2	24
ORL	C,bit	OR direct bit to Carry	2	24
ORL	C,/bit	OR complement of direct bit to Carry	2	24
MOV	C,bit	Move direct bit to Carry	2	12
MOV	bit,C	Move Carry to direct bit	2	24
JC	rel	Jump if Carry is set	2	24
JNC	rel	Jump if Carry not set	2	24
JB	bit,rel	Jump if direct Bit is set	3	24
JNB	bit,rel	Jump if direct Bit is Not set	3	24
JBC	bit,rel	Jump if direct Bit is set & clear bit	3	24

Instructiuni de salt, lucru cu subrutine

Mnemonic	Operation	Execution Time (μs)
JMP addr	Jump to addr	2
JMP @A+DPTR	Jump to A+DPTR	2
CALL addr	Call subroutine at addr	2
RET	Return from subroutine	2
RETI	Return from interrupt	2
NOP	No operation	1

Mnemonic		Description	Byte	Oscillator Period
PROGRAM BRANCHING				
ACALL	addr11	Absolute Subroutine Call	2	24
LCALL	addr16	Long Subroutine Call	3	24
RET		Return from Subroutine	1	24
RETI		Return from interrupt	1	24
AJMP	addr11	Absolute Jump	2	24
LJMP	addr16	Long Jump	3	24
SJMP	rel	Short Jump (relative addr)	2	24

The Table lists a single "JMP addr" instruction, but in fact there are three—SJMP, LJMP and AJMP—which differ in the format of the destination address. JMP is a generic mnemonic which can be used if the programmer does not care which way the jump is encoded.

The SJMP instruction encodes the destination address as a relative offset, as described above. The instruction is 2 bytes long, consisting of the opcode and the relative offset byte. The jump distance is limited to a range of −128 to +127 bytes relative to the instruction following the SJMP.

The LJMP instruction encodes the destination address as a 16-bit constant. The instruction is 3 bytes long, consisting of the opcode and two address bytes. The destination address can be anywhere in the 64K Program Memory space.

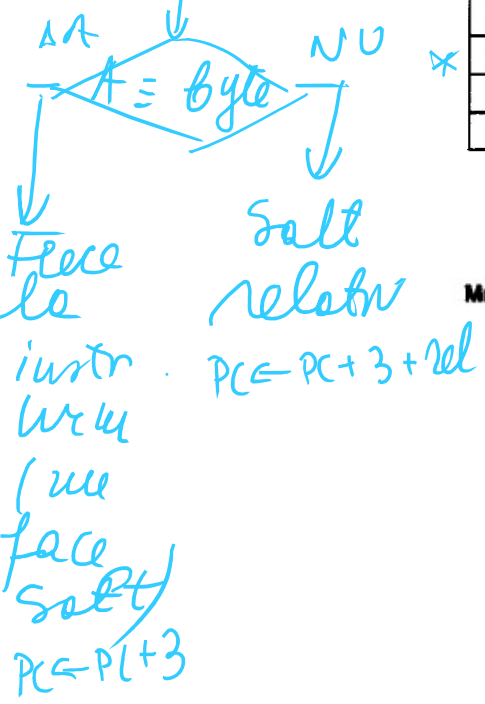
The AJMP instruction encodes the destination address as an 11-bit constant. The instruction is 2 bytes long, consisting of the opcode, which itself contains 3 of the 11 address bits, followed by another byte containing the low 8 bits of the destination address. When the instruction is executed, these 11 bits are simply substituted for the low 11 bits in the PC. The high 5 bits stay the same. Hence the destination has to be within the same 2K block as the instruction following the AJMP.

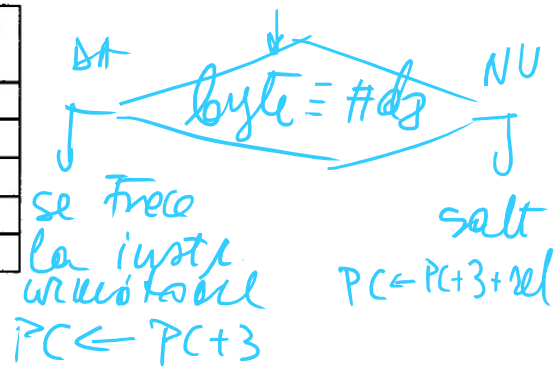
CJNE - compare and jump if not equal

Instructiuni de salt conditionat

CJNE byte, #data, rel

CJNE A, byte, rel





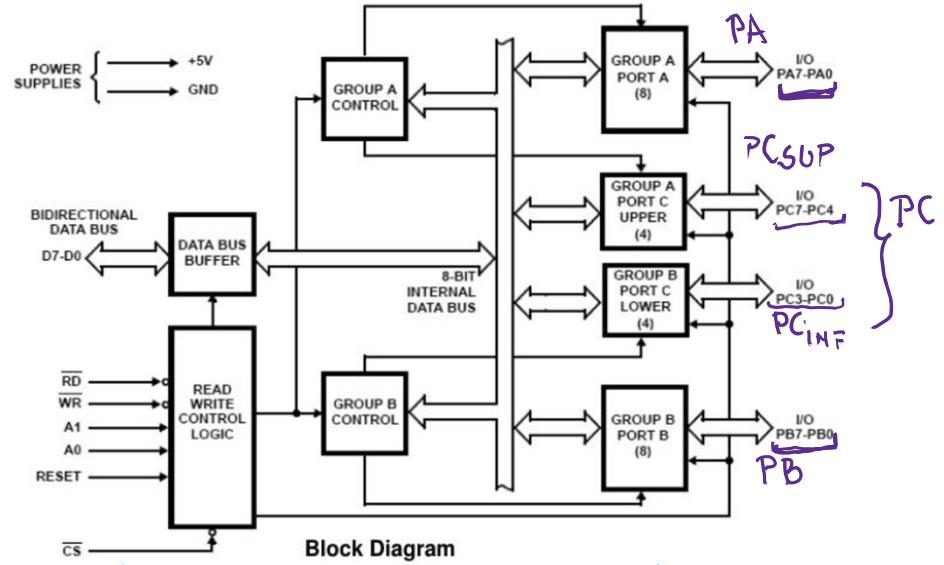
Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
JZ rel	Jump if A = 0	Accumulator only				2
JNZ rel	Jump if A ≠ 0	Accumulator only				2
DJNZ <byte>,rel	Decrement and jump if not zero	X		X		2
CJNE A,<byte>,rel	Jump if A ≠ <byte>	X			X	2
CJNE <byte>,#data,rel	Jump if <byte> ≠ #data		X	X		2

Mnemonic		Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)				
JMP	@A+DPTR	Jump indirect relative to the DPTR	1	24
JZ	rel	Jump if Accumulator is Zero	2	24
JNZ	rel	Jump if Accumulator is Not Zero	2	24
CJNE	A,direct,rel	Compare direct byte to Acc and Jump if Not Equal	3	24
CJNE	A,#data,rel	Compare immediate to Acc and Jump if Not Equal	3	24

Mnemonic		Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)				
CJNE	Rn,#data,rel	Compare immediate to register and Jump if Not Equal	3	24
CJNE	@Ri,#data,rel	Compare immediate to indirect and Jump if Not Equal	3	24
DJNZ	Rn,rel	Decrement register and Jump if Not Zero	2	24
DJNZ	direct,rel	Decrement direct byte and Jump if Not Zero	3	24
NOP		No Operation	1	12

END 15 apr 2026

8255:Programmable Peripheral Interface



Block Diagram

interfata µS — MEMORY MAPPING

interfata user (24 pini I/O)

- 40 pini; NMOS (CMOS) +5V
- 3 porturi paralele x 8 biti = 24 pini I/O (PA, PB, PC)
- Porturile sunt grupate în grupe { grup A / grup B

gr. A = {PA, PC sup} 7÷4

gr. B = {PB, PC inf} 3÷0

$PA_0 \div PA_7$
$PB_0 \div PB_7$
$PC_0 \div PC_7$

INTEL: 8255 (PPI), 8253 (TIMER), 8251 (USART), 8259 (PIC), 8257 (DMA)

PC: ×86 ×2 ×1 8250 (2 × 8251) 8259 8257

SOUTHBRIDGE (ASIC)

ZILOG: Z80, PIO, CTC, SIO

MOTOROLA 6800, 68000 ---- 8b 16b

arc I/O programabil —

* memory mapping

- 3 moduri operare (gr. A)

(gr B — 2 moduri de operare)

moduri { 0 - mod operare port paralel
1 - comunicatii paralele unidirect.
2 - —"— —"— bidirectionale

modul 1, 2 - pentru conectare cu printere

Interfata CPU

$D_0 \div D_7$ ($BD_0 \div BD_7$)

A_0, A_1 - 2^2=4 locatii (CPU are 4 adrese la care răspunde)

$\overline{RD}$; $\overline{WR}$ - (mag. control) RESET (activ 1)

$\overline{CS}$ - chip select; validare comunic CPU - 8255

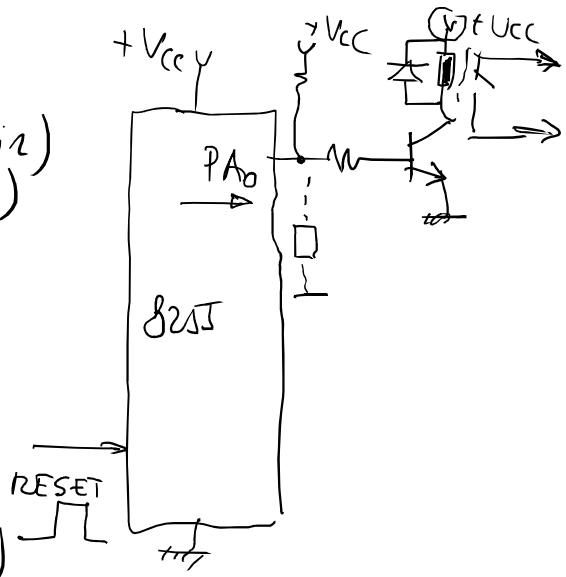
$\overline{CS} = 0$

A_1	A_0	$\overline{RD}$	$\overline{WR}$	Operație CPU cu 82$\overline{55}$
0	0	1	0	scrie PA
0	1	1	0	scrie PB
1	0	1	0	scrie PC
1	1	1	0	scrie RCC (reg cuv com) (*)
0	0	0	1	citire PA
0	1	0	1	citire PB
1	0	0	1	citire PC
1	1	0	1	- invalide
x	x	1	1	- invalida (neop.)

b_6 b_5	Operare Gr. A
0 0	→ mod 0 oper (port par.)
0 1	→ mod 1 (com par unidir)
1 0	→ mod 2 (com par. bidir)
1 1	→ invalida

Direcționalitate
0 - output
1 - input

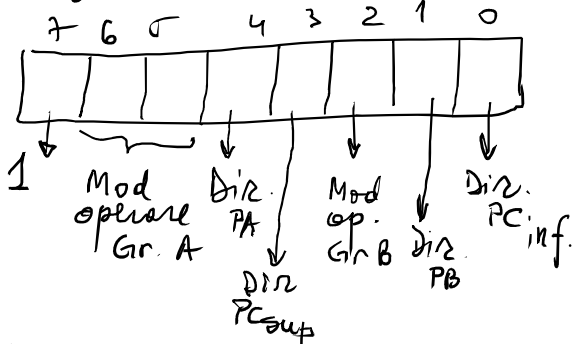
b_2	Operare Gr. B
0	mod 0 oper (port paralel)
1	mod 1 oper



Programare circ. 82$\overline{55}$

1) Programare cuv. com. în RCC → a) - Cuv. de com se transmite pe adresa RCC (*)
- RESET - se pierde programarea 82$\overline{55}$

→ b) - se operează cu liniile de port folosind adresele PA, PB sau PC

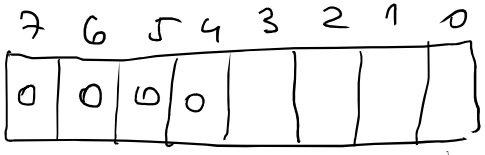


(primul format al cuv. de com)

OBS
- Cuv. de com se transmite pe adresa RCC (*)
- RESET - se pierde programarea 82$\overline{55}$
- se operează cu liniile de port folosind adresele PA, PB sau PC

Al 2-lea format al cuv. de comandă

- operează doar asupra PC-ului, deci anterior acesta a fost programat cu primul format al CW. de com. se operează în mod 0 (PC_{sup} și PC_{inf}) iar direcționalitatea PC să fie output
- avantaj - operare individuală cu liniile PC



Specifică primul din PC care va fi actionat

indică modul de actionare

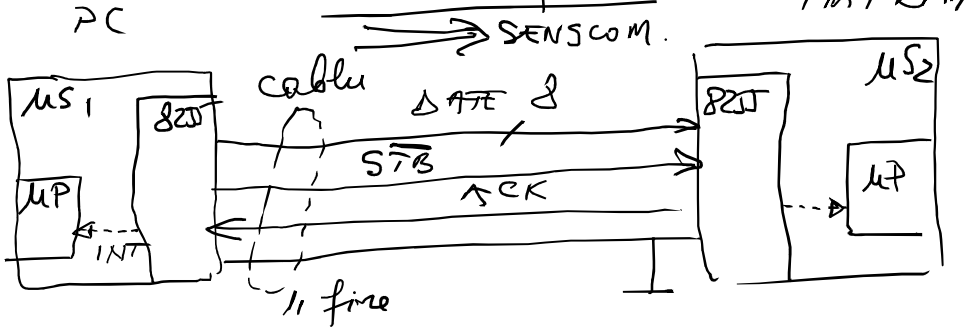
- 0 (bitul/primul specificat trece în 0)
- 1 (bitul/primul specificat trece în 1)

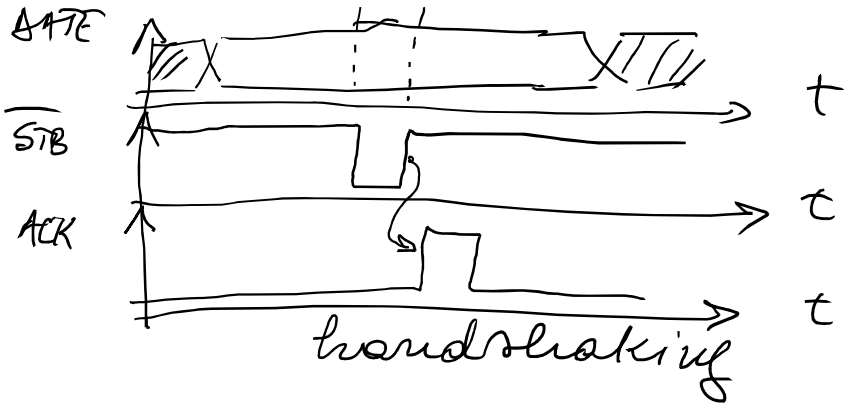
b_3 b_2 b_1	Pin specificat
0 0 0	PC_0
0 0 1	PC_1
0 1 0	PC_2
0 1 1	PC_3
1 0 0	PC_4
1 0 1	PC_5
1 1 0	PC_6
1 1 1	PC_7

$00001010_{(2)} = 0Ah \Rightarrow PC_5 \leftarrow 0$

$00001011_{(2)} = 0Bh \Rightarrow PC_5 \leftarrow 1$

MOD 1 operare





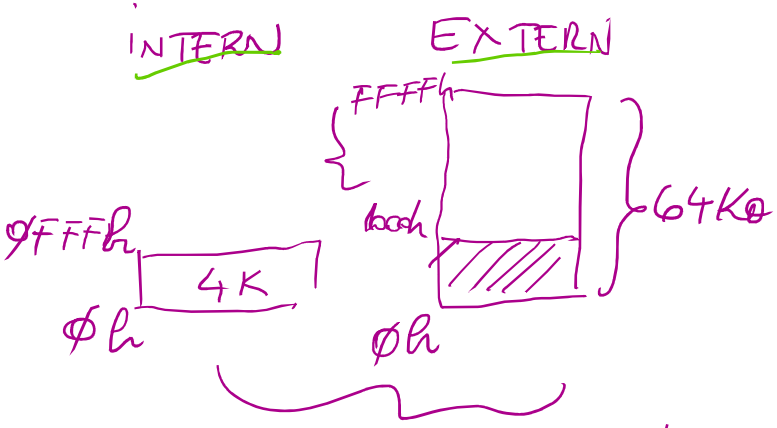
(T) Conectarea circuitelor de memorie la magistralele SAU

* - memorie de program (externă) - 64 Ko - $\overline{PSEN}$ ($\overline{PSEL}$)
* - memorie de date (externă) - 64 Ko - $\overline{RD}$; $\overline{WR}$
  - $\overline{EA}$ = 0 (configurare lucru cu mem. externă)

LUCRU CU MEM EXT (PROG, DATE)

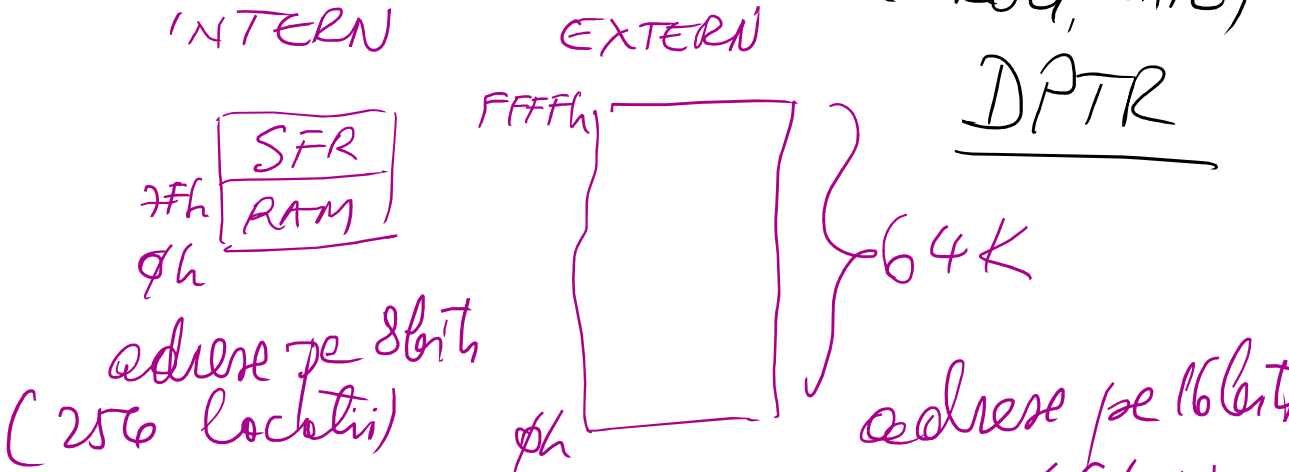
DPTR

MEM PROG



Primii 4K din mem. program interne!

MEM DATE



adrese pe 8 biți (256 locații)

adrese pe 16 biți (64K)

NU SE SUPRAPUN!

1) Conectarea mem. de prog. externă.

Problema

Să se conecteze la unul MCU 8051 – 8Ko ROM în spațiul mem. de prog. externă începând cu adresa 1000h (adresarea va fi completă), folosind soluția circuitul ROM 2764 cu organizare 8K×8.

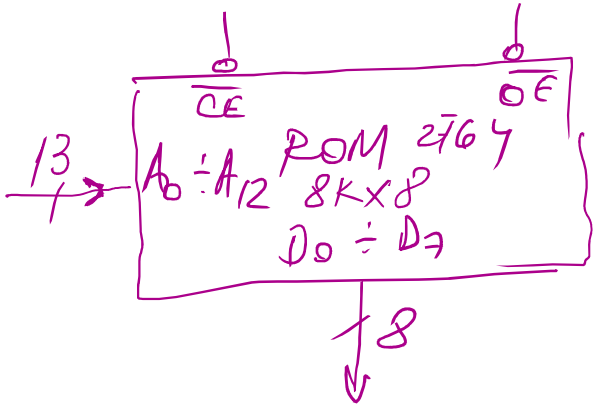
Soluție

1. Prezentare circuit cu care realizăm implementarea

ROM 8K×8 (2764)

$8K = 2^3 \cdot 2^{10} = 2^{13}$

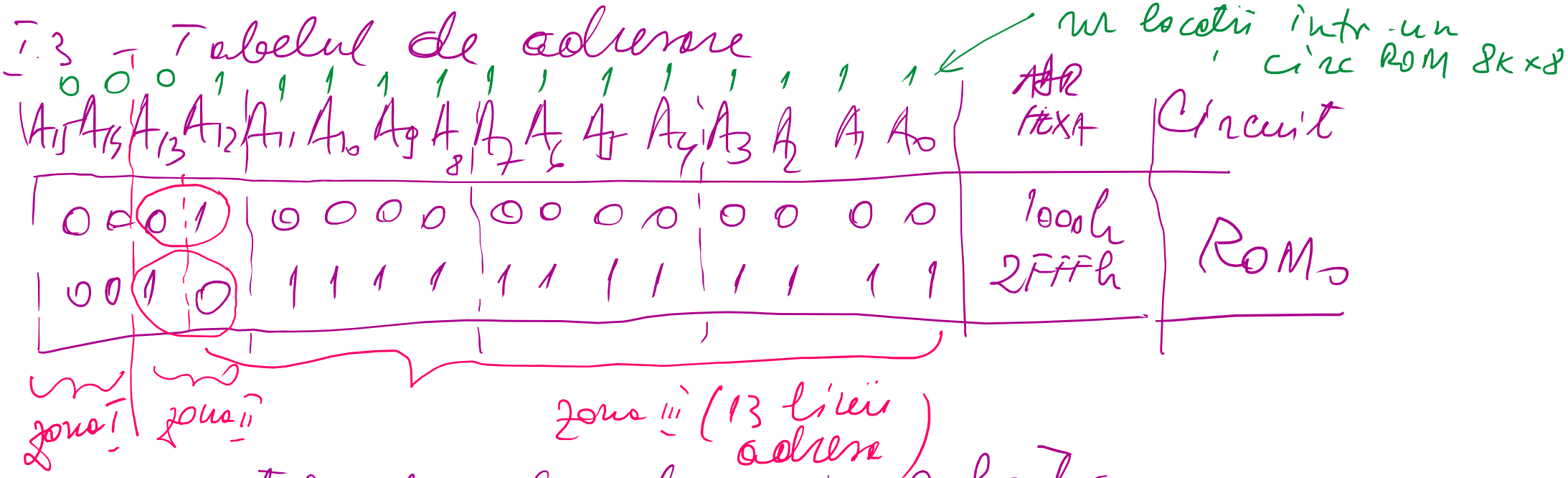
⇒ 13 linii adrese



1.2 Calculul nr. circuite necesare

$\frac{8K \times 8}{8K \times 8} = 1 \times 1 = 1$ circ (ROM)

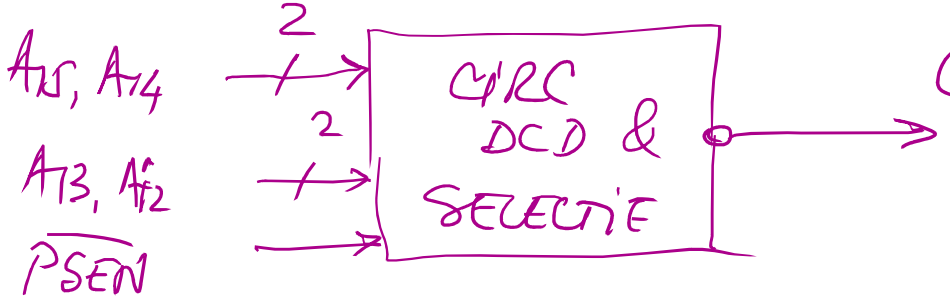
1.3 – Tabelul de adresare



un locatii intr-un circ ROM 8k x 8

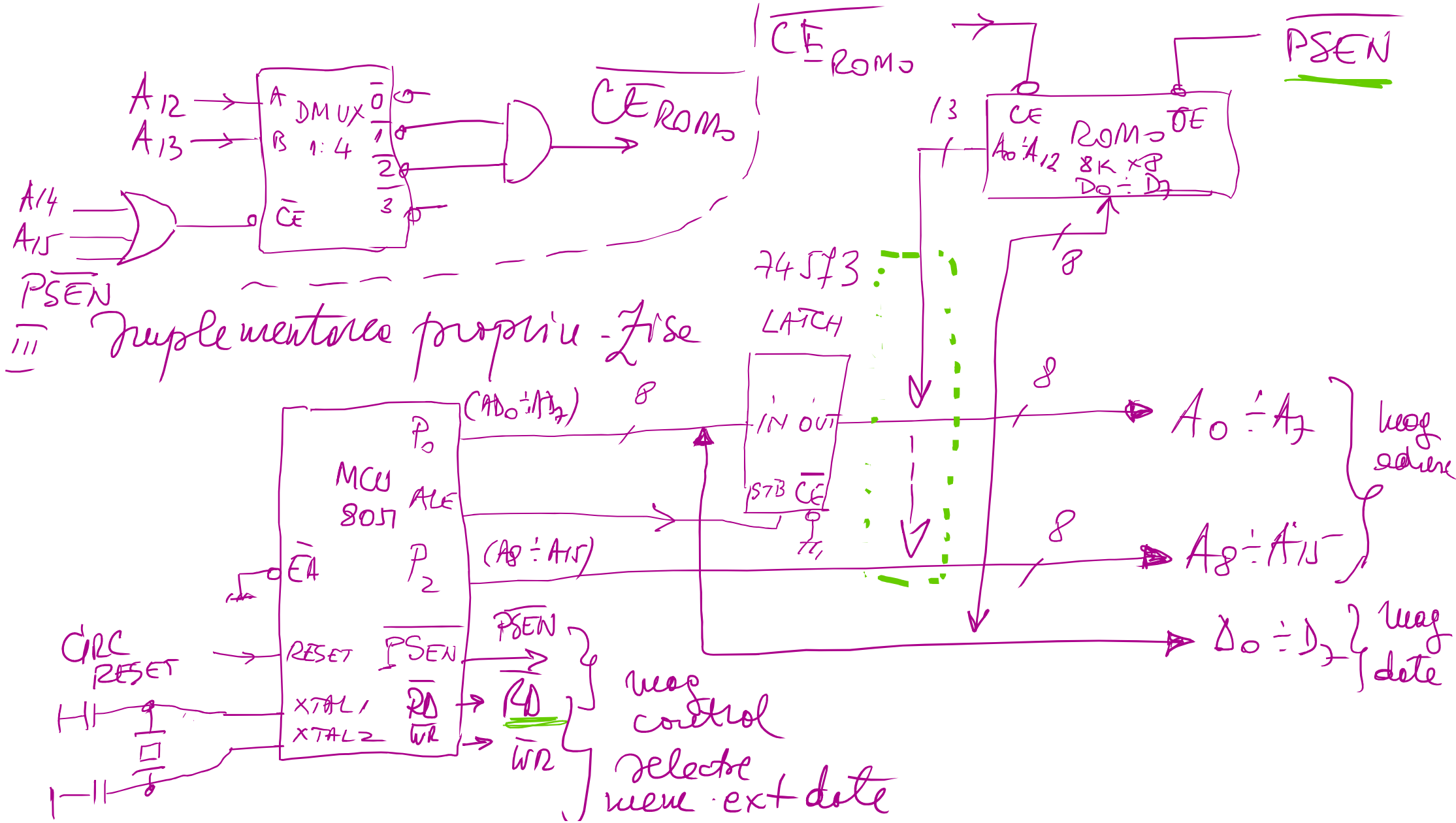
II Circuitul de decodare si selectie

Sch. bloc



- circ trebuie sa decodifice 2 comb. pentru A_{13} A_{12}
⇒ AMUX-ul folosit trebuie sa aiba 2 intrari selectie (4 iesiri) ⇒

⇒ AMUX 1:4



III Implementarea propriu-zisă

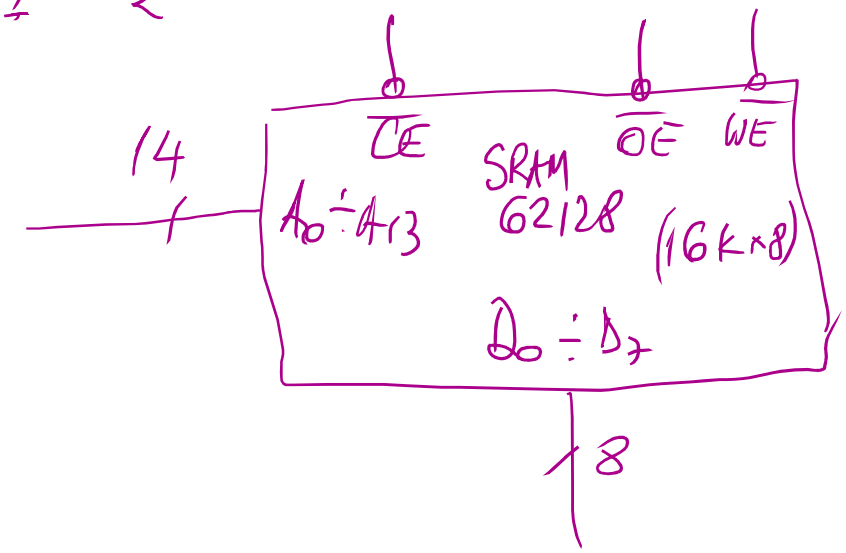
2) Conectarea memoriei de date externe —

Problemă

Să se conecteze 16 Ko memorie SRAM la un MCU 8051 folosind circuitul 62128 (16K × 8). Adresa de început a zonei de memorie 5000h (adresare nu finalizată)

Soluție

1.1 Prezentarea circuitului 62128 (SRAM)



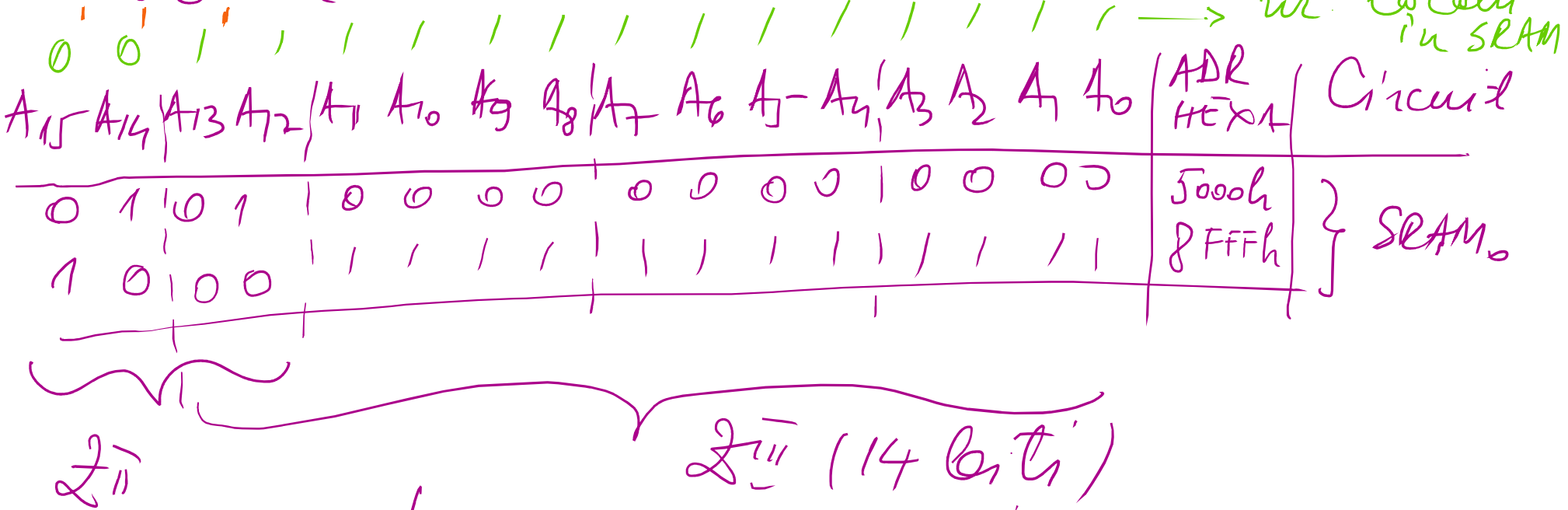
16K × 8

$2^4 \cdot 2^{10} = 2^{14}$ - linii de adrese

I.2. Calculul nr. circuite necesare

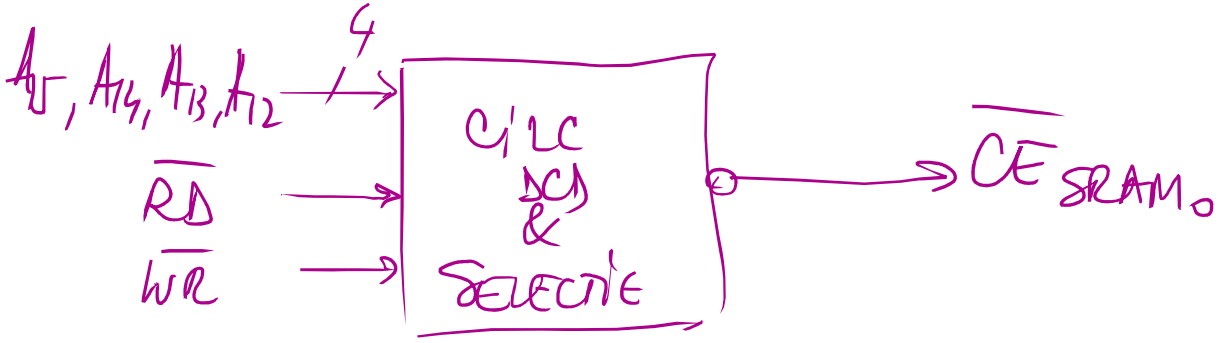
$\frac{16K \times 8}{16K \times 8} = 1 \times 1 = 1$ circuit SRAM (SRAM$_0$)

I.3 Tabelul de adresare (mem. de date ext – 64 ko)

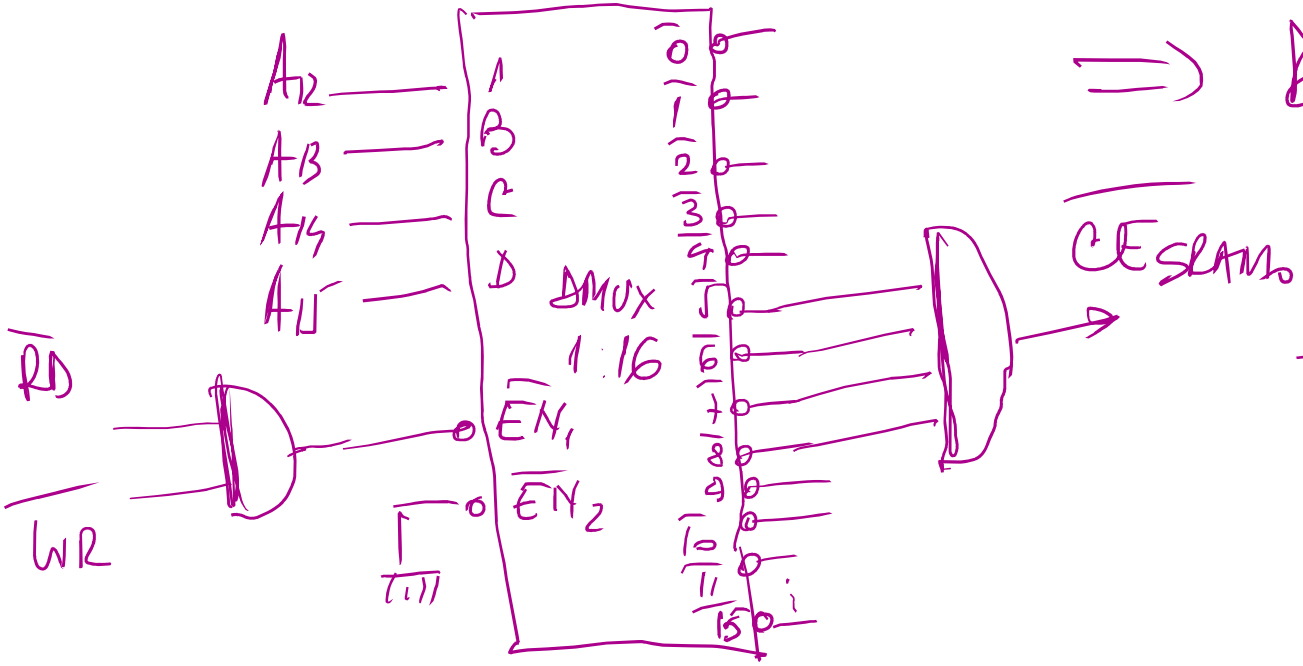


Nu există 2^{11}!

II Circuitul DCS & selecție



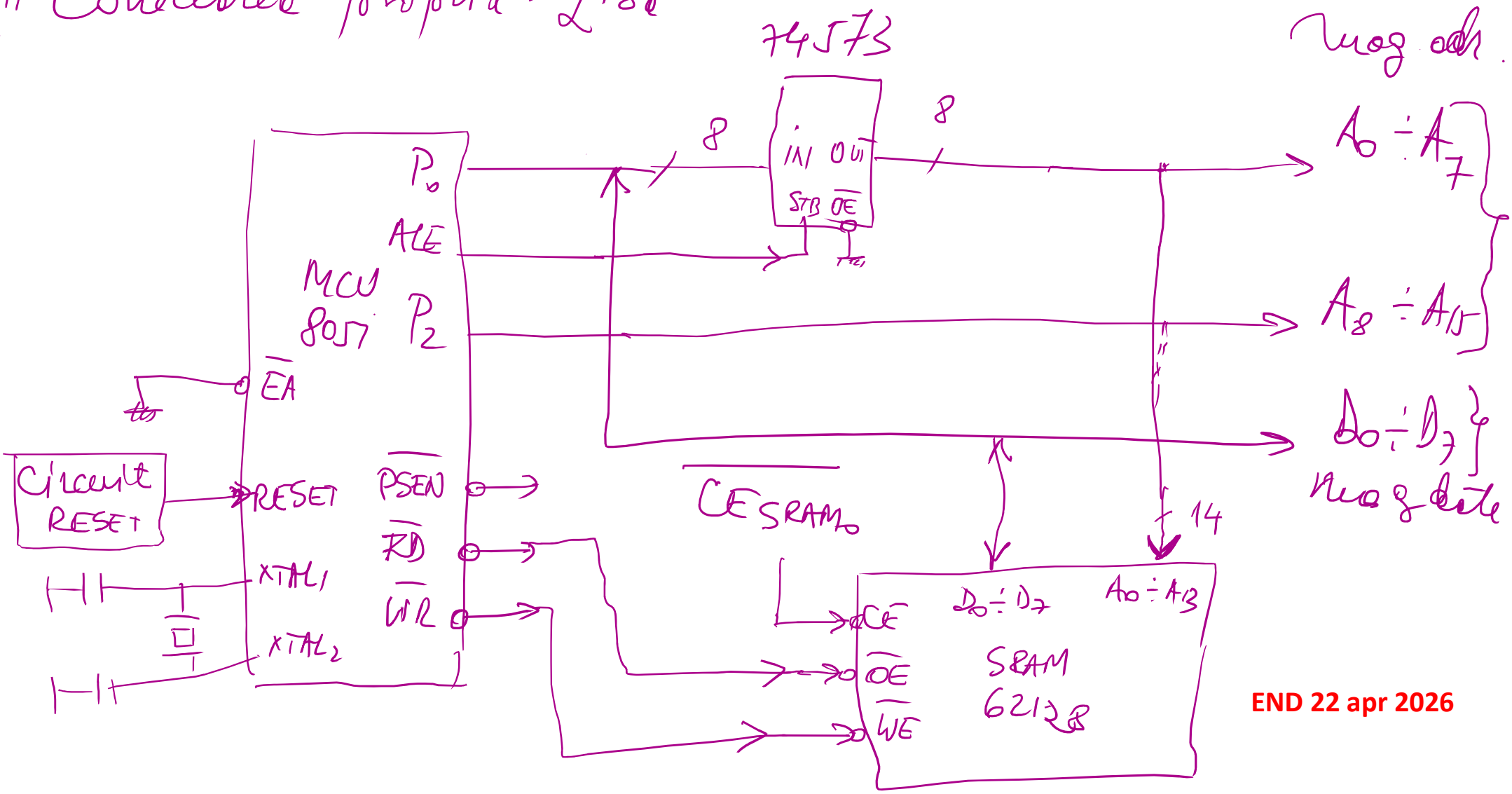
- 4 semnale selecție (conf II tabel)
- 4 combinații care rebuie identificate



$\Rightarrow$ DMUX { 4 semnale selecție / 4 ieșiri

$\Rightarrow$ DMUX 1:16 (are 4 intrări selecție)

III Conectarea proprie - zise



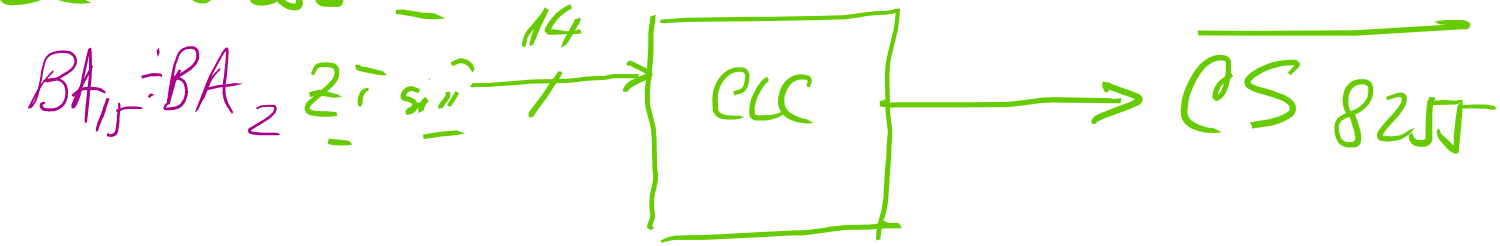
END 22 apr 2026

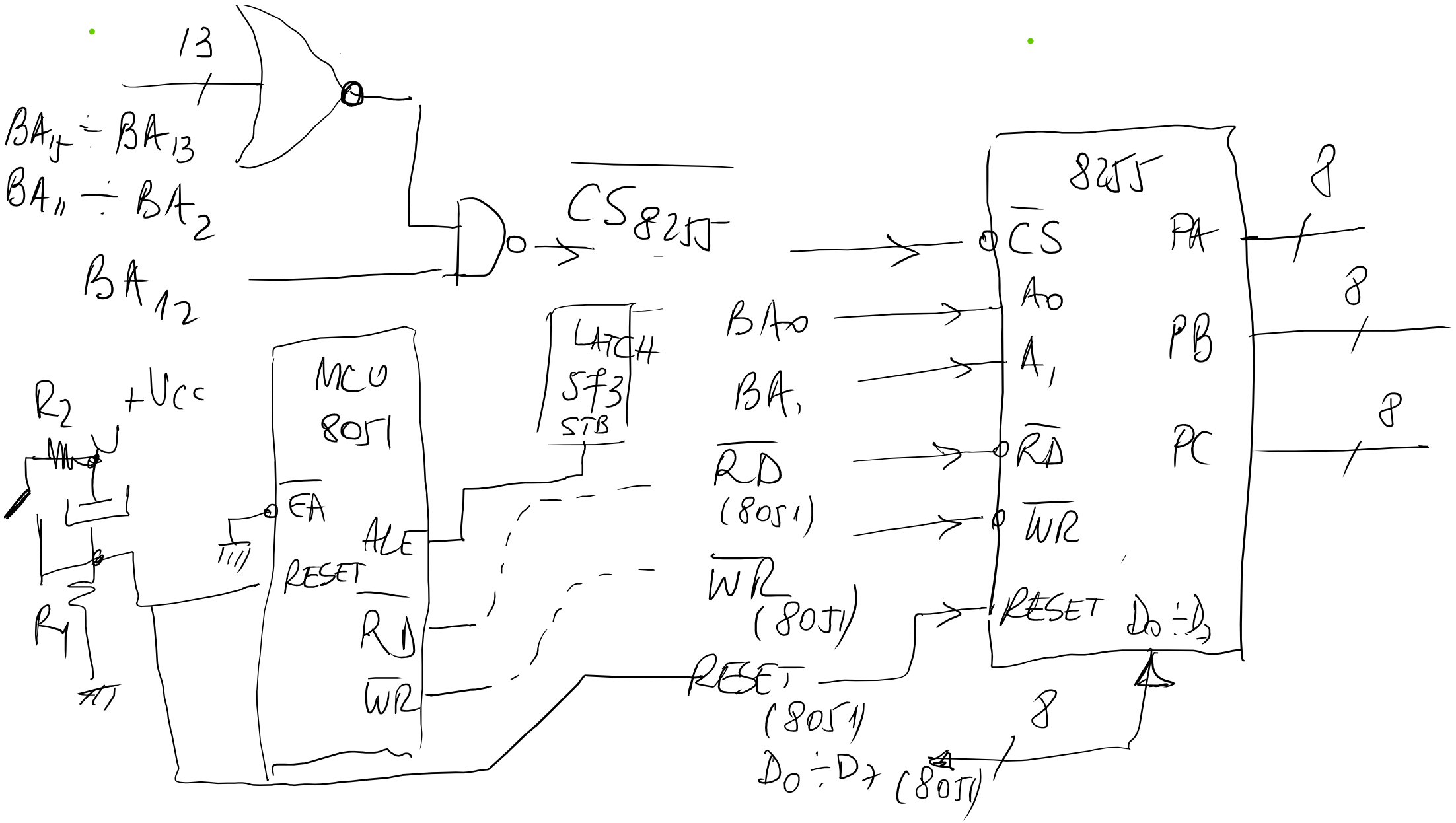
Conectarea circuitului 8255 în spațiul adreselor memoriei de date externe la MCU 8051 -

Pași

1/ se întocmește tabelul de adresare - se pun în evidență: - zonele de biți - biți nemodificați
- în particular în problemă biți selecție circuit Z_1, Z_2 și Z_3 conform - biți care se aplică circuitului Z_3

2/ Proiectarea circuitului de decodare a adreselor și selecție a circ 8255 -




13
BA15 ÷ BA13
BA11 ÷ BA2
BA12
CS8255
MCU
8051
EA
ALE
RESET
RD
WR
R2
+Vcc
R1
LATCH
573
STB
BA0
BA1
RD
(8051)
WR
(8051)
RESET
(8051)
D0 ÷ D7
(8051)
8255
CS
A0
A1
RD
WR
RESET
PA
PB
PC
D0 ÷ D7
8
8
8
8

Problema 1. Un MCU 8051, configurat cu /EA = 1 si avand cristalul de cuart de 12 MHz, are conectat la linia P1.1 un LED cu catodul la masa. Realizati programul in limbaj de asamblare 8051 prin intermediul caruia LED-ul se aprinde timp de 1 s si este stins 0,5 s, dupa care acest proces se reia.

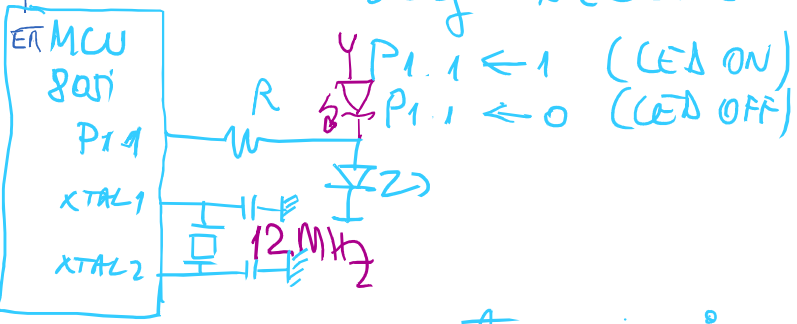
Solutie

Metodologie de rezolvare:

- Analiza problemei (conceptul de rezolvare)
- Programarea circuitelor I/O
- Elaborare Schema logica (organigrama)
- Implementare in limbaj de asamblare

1) ANALIZA PROBLEMEI

$\overline{EA} = 1$ => MCU 8051 - ne furnizeaza semnale externe



P1.1 ← 1 (LED ON)

P1.1 ← 0 (LED OFF)

- pentru obtinere temporizari folosim TIMER 0

TIMER 0

- regim lucru temporizator

TMOD C/$\overline{T}$ = 0

- 12MHz → (12) → 0,5 s (2Hz)
  → 1 s (1Hz)

1MHz → 1 μs

1MHz → 1Hz (10^6)

1MHz → 2Hz ($5 \cdot 10^5$)

=> mod de operare pe 16 biti

MOD 1 OPERARE TIMER 0

- operam in intreruperi generate de TIMER 0 cu frecventa:

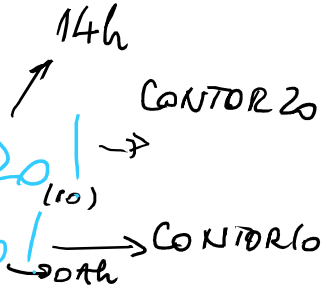
$\frac{1MHz}{50\,000} = 20Hz$ (50ms)

→ se impune divizare suplimentara (prin SW)

Alegem divizare la contor TIMER 50.000(10) = $C350_{(16)}$

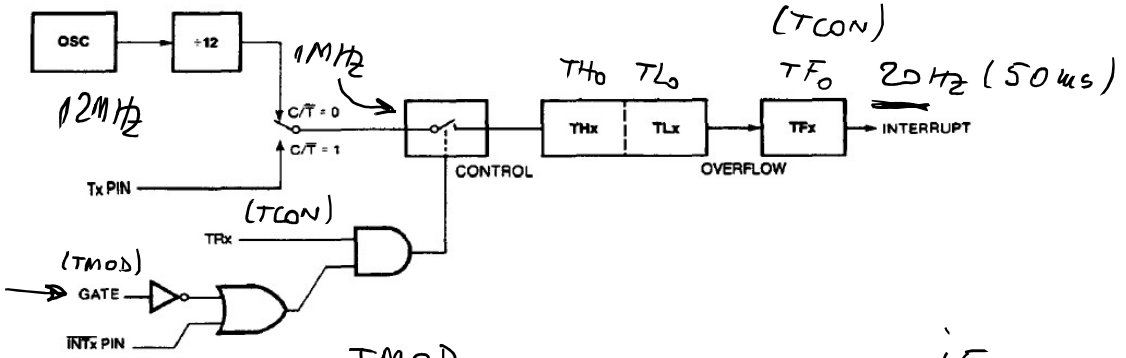
- pentru a ajunge la 1s se adauga divizare suplim cu $20_{(10)}$

- pentru a ajunge la 0,5s divizarea supl. este cu 10



2) Programare circ I/O

- TIMER 0
- LOGICA INTR
- $P_{1.1}$ / $P_{1.1} \leftarrow 1$ (LED ON)
  $P_{1.1} \leftarrow 0$ (LED OFF)



TH_0 TL_0 TF_0 20Hz (50ms)

TIMER

- TH_0, TL_0 ?

$\frac{1MHz}{20Hz} = \frac{10^6}{20} = 50.000_{(10)}$

$65536_{(10)} - 50.000_{(10)} = 15536_{(10)} = 3CB0H$

(stare finală) (nr stări) (stare inițială)

* $TH_0 \leftarrow$ #3Ch ; $TL_0 \leftarrow$ #B0h
* TMOD $\leftarrow$ #01h
* TCON $\leftarrow$ #10h

INTRERUPERI

IE $\leftarrow$ #82h

Subrutina tratare intr. TIMER 0: 000Bh

TMOD

	7	6	5	4	3	2	1	0
	GATE	C/$\overline{T}$	M_1	M_0	GATE	C/$\overline{T}$	M_1	M_0
	Timer 1				Timer 0			
	0	0	0	0	0	0	0	1

01 => 01h (mod 1)

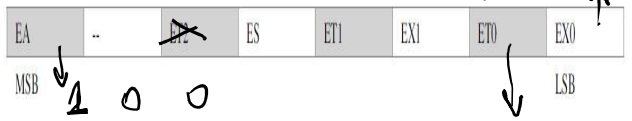
TCON

	7	6	5	4	3	2	1	0
	TF_1	TR_1	TF_0	TR_0	Folosiți la logica de întreruperi			
	0	0	0	1	0	0	0	0

=> 10h

15536 | 16
144 | 971 | 16
113 | 96 | 60 | 16
112 | 11 | 48 | 3
16 | 12
16
0

IE

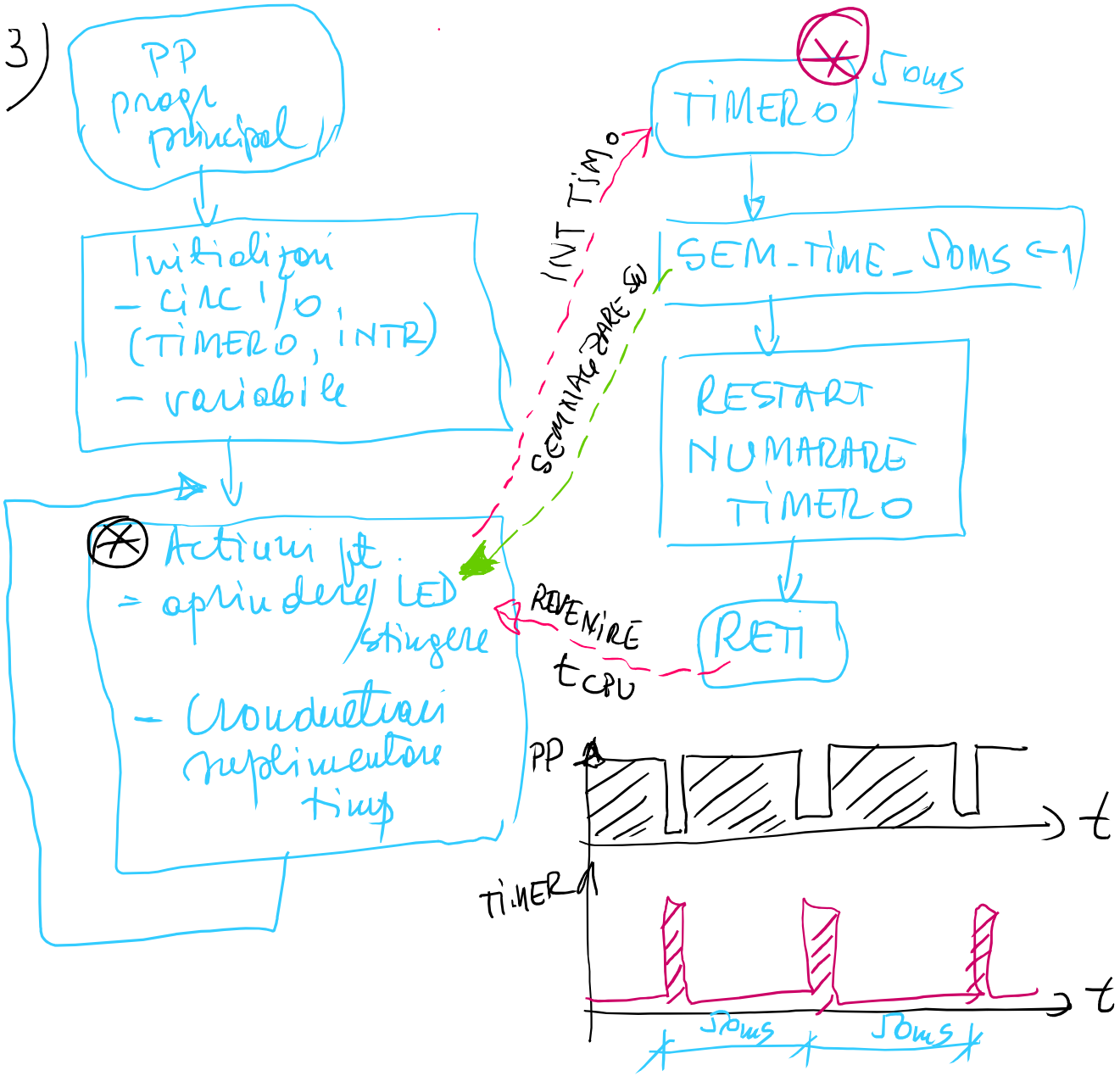


23h 1Bh 13h 000Bh 0003h

1 0 0

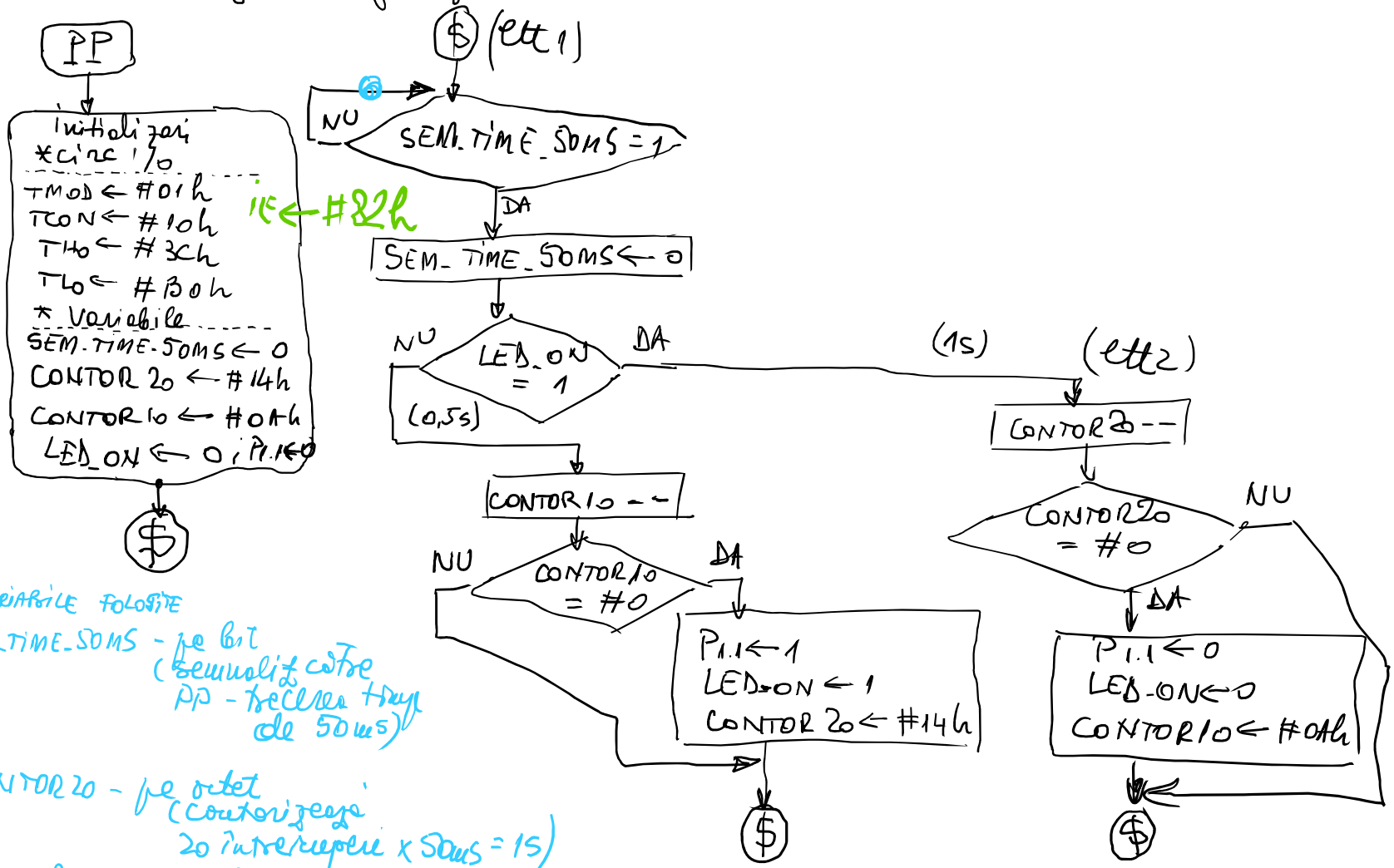
0 0 0 1 0 = 82h

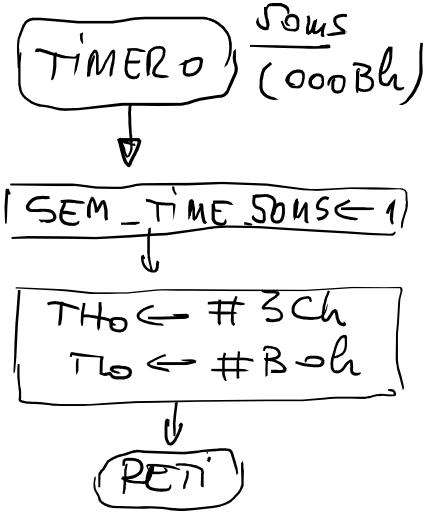
3)


PP
program principal
Inițializări
- CNC I/O, (TIMER0, INTR)
- variabile
Acțiuni pt.
= aprindere LED
stingere
- Coordonatori suplimentari timp
TIMER0
50ms
SEM_TIME_50MS ← 1
RESTART NUMĂRARE TIMER0
REȚI
INT TIM0
SEMNALARE SW
REVENIRE
tCPU

PP
t
TIMER
t
50ms
50ms

3) Schema logica (organigrama)





VARIABILE FOLOSITE

SEM_TIME_50MS - 1 bit (semnalizează catre PP - trecerea unui interval de 50ms)

CONTOR 20 - 1 octet (contorizează 20 întreruperi x 50ms = 1s)

CONTOR 10 - 1 octet (contorizează 10 într. x 50ms = 0,5s)

LED_ON - indică SW pt LED (1 - ON, 0 - OFF)

4] LIMBAJ ASM 51

```
        .ORG 0h
START:  LJMP 100h
        SEM_TIME_50MS   BIT  00h
        LED_ON          BIT  01h
        CONTOR10        DATA 30h
        CONTOR20        DATA 31h
        .ORG 100h
        MOV TMOD, #01h
        MOV TCON, #10h
        MOV TH0 , #3Ch
        MOV TL0 , #B0h
        MOV IE , #82h
        CLR SEM_TIME_50MS
        MOV CONTOR1, #0Ah
        MOV CONTOR2 , #14h
        CLR LED_ON
        CLR P1.1
ett0:   JNB SEM_TIME_50MS ett1
        CLR SEM_TIME_50MS
        JB LED_ON ett2
        DEC CONTOR10
        CJNE CONTOR10, #00, ett1
        SETB P1.1
        SETB LED_ON
        MOV CONTOR20. #14h
        SJMP ett1
ett2:   DEC CONTOR20
        CJNE CONTOR20, #00h, ett1
        CLR P1.1
        CLR LED_ON
        MOV CONTOR10, #0Ah
        SJMP ett1
        .ORG 000Bh
000Bh:  SETB SET_TIME_50MS
        MOV TH0, # 3Ch
        MOV TL0, # B0h
        RETI
        .END
```

END 22 apr 2026

PROBLEMA 2

Un circ. 8255 este conectat la mag. unui MCU 8051 în spațiul extern de memorie de date începând cu adresa 1000h și având conexiunile $BA_0 \leftrightarrow A_0$, $BA_1 \leftrightarrow A_1$, se cere realizarea unui program în ASM 8051 prin care (μproc. 8051) (μproc. 8255) PC$_3$ generează un semnal oscilatoriu digital cu frecv. max. posibilă și factor de umplere de exact 1/2.

Soluție

Metodologie de rezolvare:

1) - Analiza problemei (conceptul de rezolvare)
2) - Programarea circuitelor I/O
3) - Elaborare Schema logica (organigrama)
4) - Implementare in limbaj de asamblare

1) ANALIZA

PC$_3$ ∈ Gr. B

$\begin{cases} PC_3 \leftarrow 0 \\ PC_3 \leftarrow 1 \end{cases}$ Se repetă

$\begin{cases} PC_3 - \text{ieșire și} \\ \text{operare în} \\ \text{mod 0} \end{cases}$

$PC_3 \in PC_{inf}$ - ieșire și op în mod 0

2) Progr. circ I/O → a) identificare adrese circ I/O
→ b) cuv. com. 8255 → A_1 A_0 = 11

a)

μP	BA_{15} BA_{14} BA_{13} BA_{12} BA_{11} BA_{10} BA_9 BA_8	BA_7 BA_6 BA_5 BA_4 BA_3 BA_2 BA_1 BA_0	ADR HEXA	Explicit
	0 0 0 1	0 0 0 0 0 0 0 0	1000h	adr PA
	0 0 0 1	0 0 0 0 0 0 0 1	1001h	adr PB
	0 0 0 1	0 0 0 0 0 0 1 0	1002h	adr PC
	0 0 0 1	0 0 0 0 0 0 1 1	1003h	adr RCC

b) Cuv. de com.

1) Primul format

7 6 5 4 3 2 1 0

= 80h → (1003h)

Gr A mod 0 | PA ies | PC sau ies | Gr B mod 0 | PB ies | PCinf ieșire

OBS Trebuie realizată în HW o schemă prin care se generează $\overline{CS}_{8255}$ a.î. $\overline{CS} = 0$ pentru cele 4 adrese din tabel.

$PC_3 \leftarrow 1$ PC_3

VAR1

PC $0000\,1000 \equiv 08h \rightarrow (1002h)$

$PC_3 \leftarrow 0$

PC $0000\,0000 \equiv 00h \rightarrow (1002h)$

OBS se aspettano tutti bit di PC!

VAR2

$$\begin{cases} A \leftarrow PC \\ A \leftarrow A \vee 0000\,1000_{(2)} \equiv 08h \\ PC \leftarrow A \quad PC_3 \leftarrow 1 \end{cases}$$

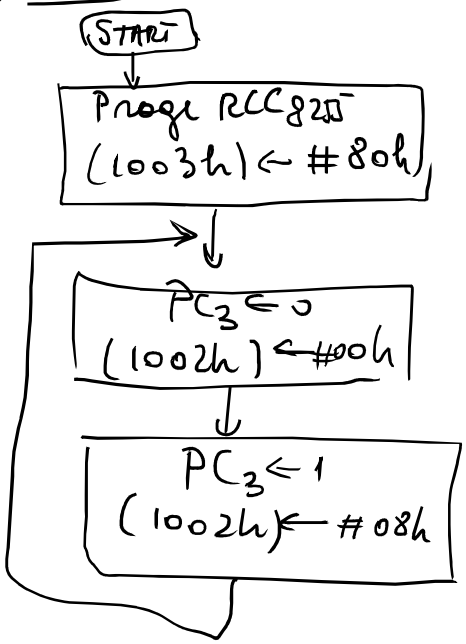
$$\begin{cases} A \leftarrow PC \\ A \leftarrow A \wedge 1111\,0111_{(2)} \equiv FFh \\ PC \leftarrow A \quad PC_3 \leftarrow 0 \end{cases}$$

VAR3 Ce format 2 al cui corr

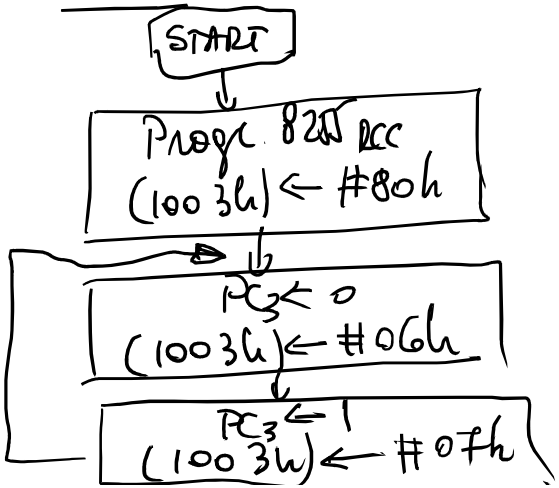
0000	0111	$\equiv 07h \rightarrow$ RCC $(PC_3 \leftarrow 1)$ (1003h)

0000	0110	$\equiv 06h \rightarrow$ RCC $(PC_3 \leftarrow 0)$ (1003h)

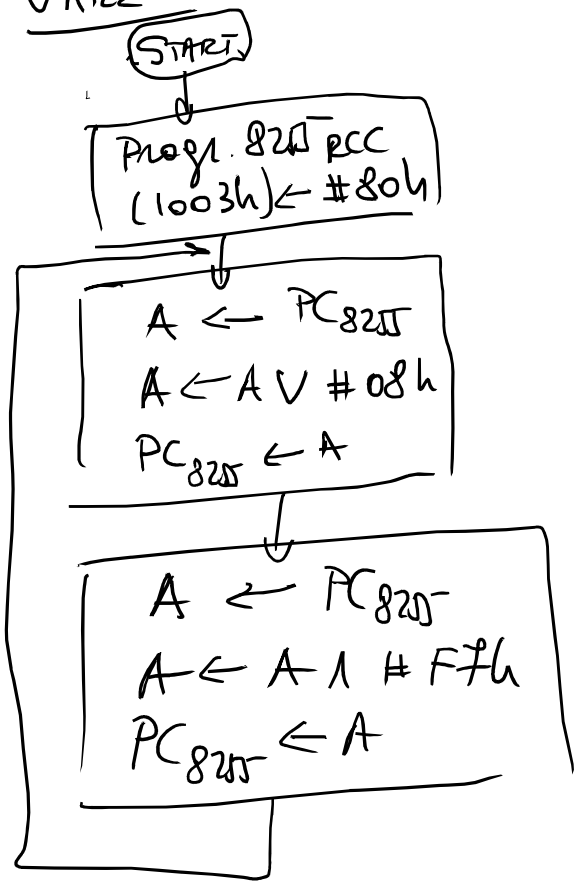
3) VAR1



VAR3

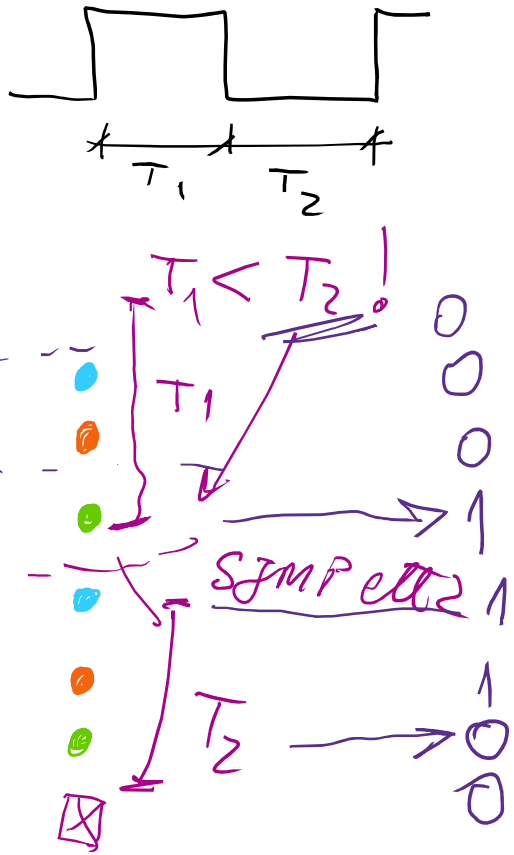


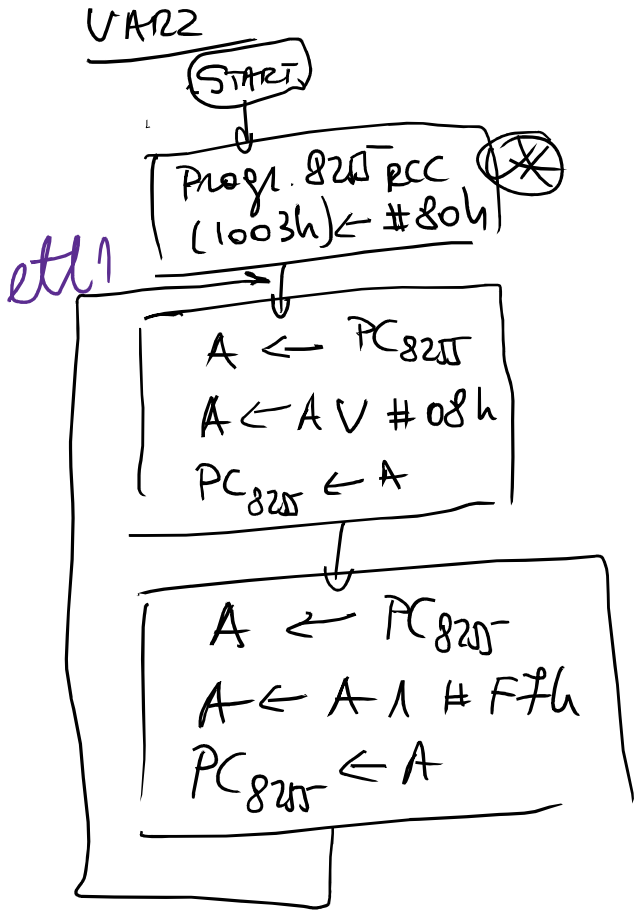
VAR2



4)

```
.ORG 0h
LJMP 100h
.ORG 100h
MOV A, #80h
MOV DPTR, #1003h ; adr RCC
MOVX @DPTR, A
ett1: MOV DPTR, #1002h ; adr PC
      MOVX A, @DPTR ; citire PC
      ORL A, #08h
      MOVX @DPTR, A ; scriere PC
ett2: MOVX A, @DPTR ; citire PC
      ANL A, #0F7h
      MOVX @DPTR, A ; scriere PC
      SJMP ett1
```





Problema 3. Un MCU 8051, configurat cu /EA = 1 si avand cristalul de cuart de 12 MHz, comanda prin liniile P1.6-P1.0 cei 7 anozi ai unui afisor LED 7 segmente cu catodul conectat la masa. Realizati programul in limbaj de asamblare 8051 prin intermediul caruia pe LED-urile afisorului se aprinde timp de 2s cifra 2 si apoi se aprinde timp de 3s cifra 3, dupa care acest proces se reia.

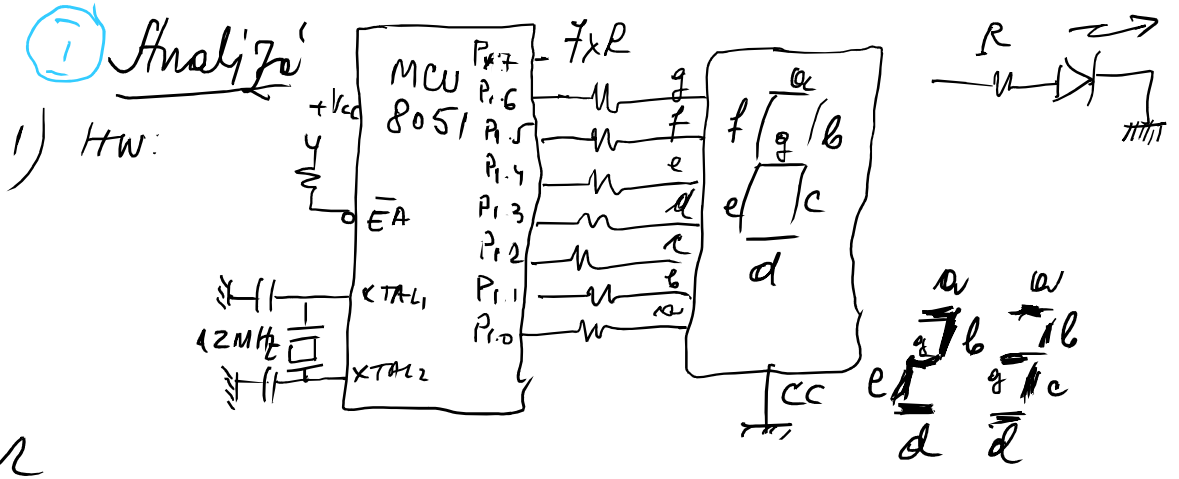
Solutie

Metodologie de rezolvare:

- Analiza problemei (conceptul de rezolvare)
- Programarea circuitelor I/O
- Elaborare Schema logica (organigrama)
- Implementare in limbaj de asamblare

1) Analiza

1) HW:



CIFRA AFIȘATĂ	g P1.6	f P1.5	e P1.4	d P1.3	c P1.2	b P1.1	a P1.0	COD HEXA
2	1	0	1	1	0	1	1	5Bh ✓
3	1	0	0	1	1	1	1	4Fh ✓

2) Apare necesitatea unui timer care să temporalizeze intervalele de timp (2s, 3s) la CPU.

⇒ Operare în întrerupere (TIMER 0)

f_{CLK} = 12MHz și TIMER 0 lucrează în REGIM TIMER MOD DE OPERARE

$f_{inTIMER0} = \frac{f_{CLK}}{12} = \frac{12MHz}{12} = 1MHz$ ($T = 1\mu s$)

0,5Hz ← ↓ → 0,33Hz

($T = 2s$) ($T = 3s$)

1MHz → 0,5Hz $\frac{1000000}{0,5} = 2 \cdot 10^6$

1MHz → 0,33Hz $\frac{1000000}{0,33} = 3 \cdot 10^6$

Este nevoie de divizări mari $< 2 \cdot 10^6$ / $3 \cdot 10^6$

⇒ mod 1 de operare la TIMER 0 (16 biți) $FFFF_h = 65535_{(10)}$

Aleg divizare la TIMER în modul 1 cu valoarea 50.000 (divizare la TIMER 0)

$f_{in\ TIMER0} = \frac{12MHz}{12} = 1MHz$ $\quad \frac{1MHz}{50000} = 20Hz \rightarrow \frac{1}{20Hz} = 0.05s = \underline{50ms}$

⊛ CPU trebuie să gestioneze și divizările suplimentare necesare contorizării timpilor de 2s și 3s. diviz SW

⊛ TIMER 0 va oferi semnalizări care indică trecerea intervalelor de timp de 50ms (20Hz) vom folosi o variabilă pe bit SEM_TIME_50MS

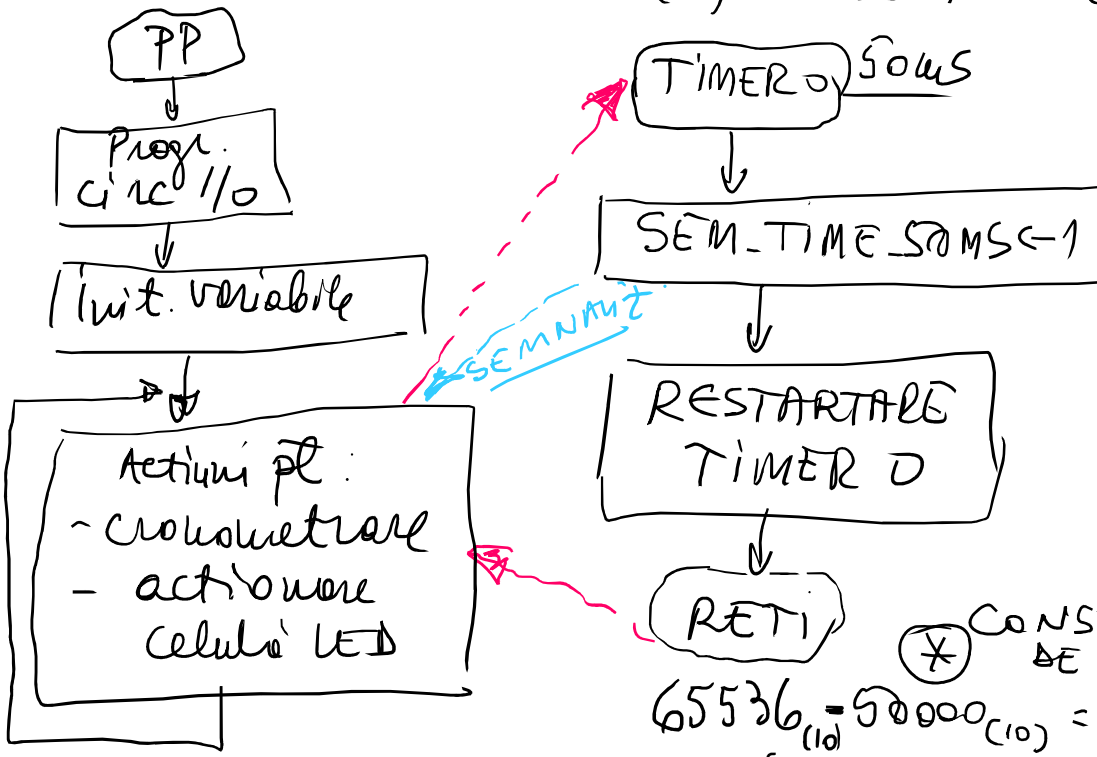
2s ⇓ $\quad \frac{40_{(10)}}{=28_h}$ $\qquad$ 3s ⇓ $\quad \frac{60_{(10)}}{=3C_h}$

⊛ În Prog Principal (PP):

- CPU testează semnalizările de la TIMER 0, contorizând timpii de așteptare pentru cifrele afișate pe celula LED corespunzătoare
- folosim variabila CIFRA pe bit / CIFRA = 0 → se afișează „2"
- SEM_TIME_50MS face legătura între cele / CIFRA = 1 → se afișează „3"

2 zone SOFTWARE PP - TIMER 0

- în PP se vor gestiona cei 2 contori pentru măsurarea timpului precizat de la semnalizarea de 50ms de la TIMER 0
CONTOR 2S ; CONTOR 3S - variabile pe octet cu valorile inițiale $40_{(10)} = 28h$; $60_{(10)} = 3Ch$ - variabilele vor fi actualizate prin decrementare



1) Programare circ. I/O
a) - TIMER 0 ← regim timer
mod operare 1
(16 biți ri blocare la nr. numărare)

(*) TMOD

	7	6	5	4	3	2	1	0
	GATE	C/$\overline{T}$	M_1	M_0	GATE	C/$\overline{T}$	M_1	M_0
	Timer 1				Timer 0			

01h ⇐ 0 0 0 0 0 0 0 1

(*) CONSTANTA DE DIVIZARE

$65536_{(10)} - 50000_{(10)} = 15536_{(10)} =$

{ TH0 ← #3Ch = 3CB0h
{ TL0 ← #B0h

(*) TCON

	7	6	5	4	3	2	1	0
	TF_1	TR_1	TF_0	TR_0	Folosiți la logica de întreruperi			

10h ⇐ 0 0 0 1 0 0 0 0

b) - LOGICA DE INTRERUPERI

- 1 singură sursă de întreruperi ⇒ ⇒ nu avem nevoie de reg IP.
- IE - interrupt enable

7	6	5	4	3	2	1	0
EA	-	-	ES	ET1	EX1	ET0	EX0
1	0	0	0	0	0	1	0

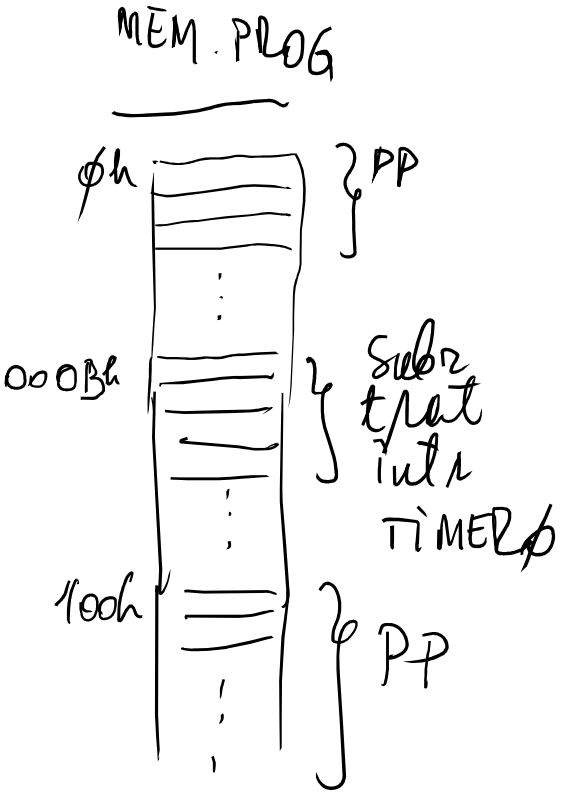
⇒ 82h

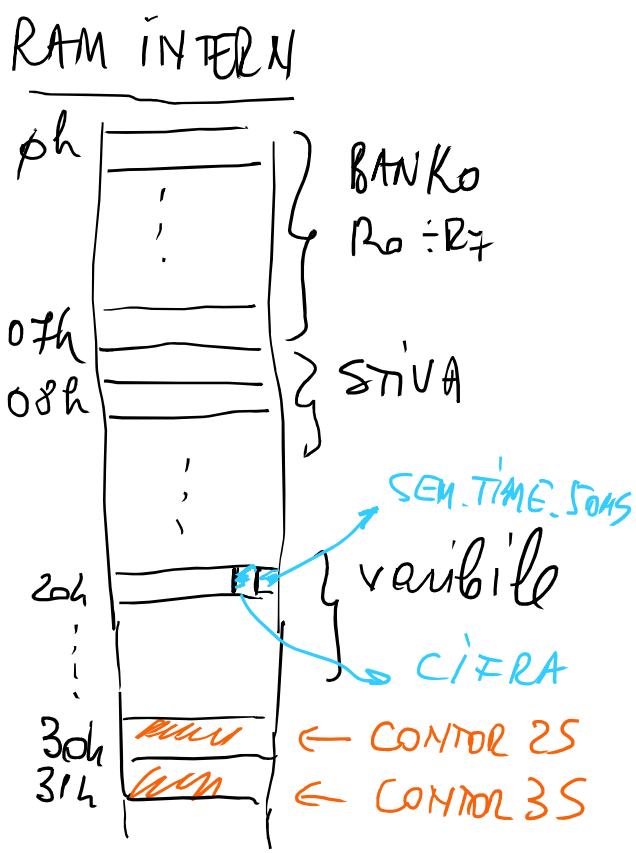
* Subrutina de tratare întrerupere TIMER 0 este plasată la adresa rezervată din mem. de program 000Bh ✓
* Stiva începe cea propusă de 8051 și anume cu baza la adresa RAM intern 07h (prima depunere în stivă va fi la adresa 08h)

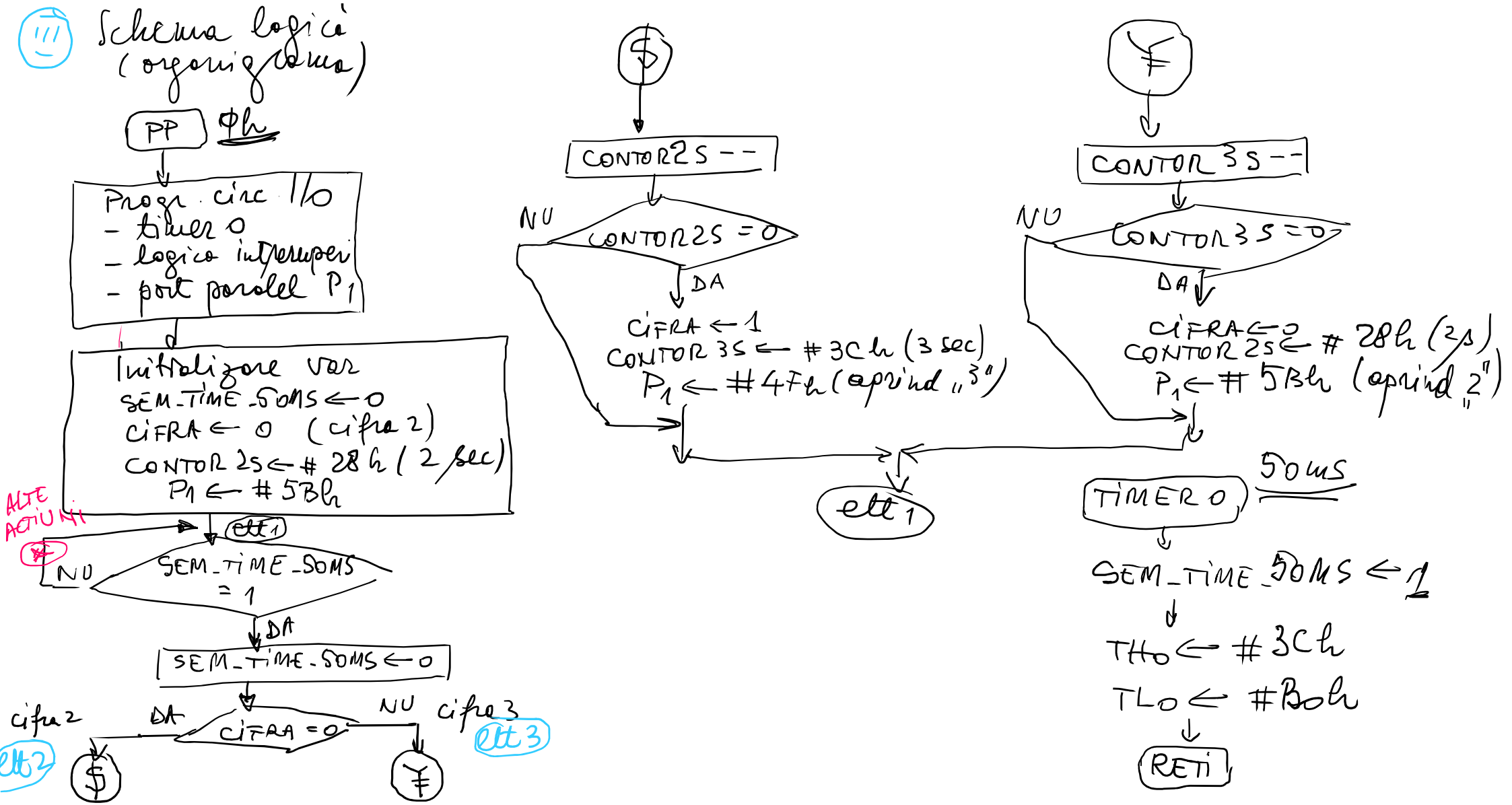
c) Portul paralel P1

P1 ← #5Bh (se aprinde „2")

P1 ← #4Fh (se aprinde „3")






Schema logică (organigrama)
PP
0h
Progr. circ I/O
- timer 0
- logica intreruperi
- port paralel P1
Initializare var
SEM_TIME_50MS ← 0
CIFRA ← 0 (cifra 2)
CONTOR 2S ← #28h (2 sec)
P1 ← #5Bh
ALTE ACTIUNI
ett1
NU
SEM_TIME_50MS = 1
DA
SEM_TIME_50MS ← 0
cifra 2
DA
CIFRA = 0
NU cifra 3
ett2
ett3
CONTOR 2S --
NU
CONTOR 2S = 0
DA
CIFRA ← 1
CONTOR 3S ← #3Ch (3 sec)
P1 ← #4Fh (aprind „3")
ett1
CONTOR 3S --
NU
CONTOR 3S = 0
DA
CIFRA ← 0
CONTOR 2S ← #28h (2s)
P1 ← #5Bh (aprind „2")
TIMER 0
50ms
SEM_TIME_50MS ← 1
TH0 ← #3Ch
TL0 ← #B0h
RETI

```
        ORG 0h
START:  LJMP etto
        ORG 000Bh
timo:   SETB SET_TIME_50MS
        MOV TH0, #3Ch
        MOV TL0, #B0h
        RETI

SETB_TIME_50MS  BIT   00h
CIFRA           BIT   01h
CONTOR25        DATA  30h
CONTOR35        DATA  31h

        ORG 100h
etto:   MOV TMOD, #01h
        MOV TCON, #10h
        MOV TH0, #3Ch
        MOV TL0, #B0h
        MOV IE, #82h
        MOV P1, #5Bh ; CIFRA "2"
INIT    CLR SET_TIME_50MS
VAR     CLR CIFRA
        MOV CONTOR25, #28h
ett1:   JNB SET_TIME_50MS, ett1
        CLR SET_TIME_50MS
        JB CIFRA, ett3
ett2:   DEC CONTOR25
        CJNE CONTOR25, #00h, ett1
        SETB CIFRA
        MOV CONTOR35, #3Ch
        MOV P1, #4Ch ; aprind "3"
        SJMP ett1
ett3:   DEC CONTOR35
        CJNE CONTOR35, #00h, ett1
        CLR CIFRA
        MOV CONTOR25, #28h
        MOV P1, #9Bh ; aprind "2"
        SJMP ett1
        END
```

END 29 apr 2026

```
SET_TIME_50MS   BIT   00h
CIFRA           BIT   01h
CONTOR25        DATA  30h
CONTOR35        DATA  31h

        XDATA
```